Practical Marketing Science

A field guide to measuring what marketing actually does

Christos Visvardis

visvardis.com

October 2026

Preface

This book is for people whose job is to answer a deceptively simple question: *what did our marketing actually do?*

You might sit in a growth team, an analytics team, an agency or a consultancy. You probably already know how to write SQL, fit a regression and read a confidence interval. What tends to be missing is not technique but judgment: which method fits which question, what each method can and cannot tell you, and how to turn a model output into a budget decision someone will act on. That is the gap this book tries to close.

How this book is organized

The book is organized around the decisions a measurement team supports, not around techniques. Most chapters follow the same shape:

1. **The decision the chapter supports**, stated up front.
2. **The method**, explained just far enough to use it correctly.
3. **A worked example** on simulated data, so you can see the method recover (or fail to recover) a known truth.
4. **Pitfalls**: the ways the method goes wrong in practice and how to spot them.
5. **A checklist** to run before you trust a result.
6. **Further reading**, an annotated selection from the resource library in Appendix A.

Simulated data is a deliberate choice. With real data you never know the true answer, so you cannot tell whether a method worked. With simulated data you set the truth yourself and then check what the method recovers. Every example in the book is reproducible from the code shown.

Contents at a glance

The parts build on each other but can be read out of order: each chapter states the decision it supports and points back to the earlier material it relies on.

Part	Chapters
I. Foundations	1. What marketing science is for · 2. Data and signal foundations · 3. Causal thinking for marketers · 4. Bayesian reasoning in practice
II. Experiments	5. A/B tests that hold up · 6. Geo experiments · 7. Quasi-experiments: DiD, CausalImpact, synthetic control · 8. Reading platform lift studies
III. Models	9. Marketing mix modeling end to end · 10. Bayesian MMM in practice · 11. Validating and calibrating MMM · 12. Response curves and budget optimization · 13. Attribution and its limits · 14. Triangulation: deciding under contradictory evidence
IV. Customers	15. Cohorts, retention, LTV and payback · 16. Churn and survival analysis · 17. Segmentation
V. Brand, pricing, demand	18. Brand measurement · 19. Pricing research and elasticities · 20. Forecasting and anomaly detection
VI. Channel playbooks	21. Paid search · 22. Paid social and creative fatigue · 23. SEO measurement · 24. CRM and email holdouts
VII. Practice	25. Readouts that change decisions · 26. The analyst's toolkit
Appendices	A. Resource library · B. Glossary · C. All checklists in one place

Code and tools

Examples are in Python. Parts I and II use only `numpy`, `pandas`, `scipy`, `statsmodels` and `matplotlib`, so that they run anywhere. Part III adds PyMC for Bayesian marketing mix models; those models are fitted once by `fit_models.py` (a few minutes on a laptop) and the chapters load the saved results. Where R has the better tool (GeoLift, Robyn, CausalImpact's original implementation), the text points to it.

The book itself is built with bookdown.

A note on sources

The further-reading sections draw on a curated library of public resources: package documentation, blog posts, courses and open-source code from the marketing science

community. The text of this book is original; the resources are pointers for going deeper. Links rot, so where a resource has moved, search for its title.

Contact

Website: visvardis.com

LinkedIn: linkedin.com/in/cvis

Chapter 1 What marketing science is for

> **The decision this chapter supports.**
> Whether a measurement effort is worth doing at all, which evidence to trust when sources disagree, and how to organize a measurement program so that it keeps feeding budget decisions instead of producing reports nobody acts on.

1.1 Measurement exists to change decisions

Marketing science is not the production of numbers. It is the reduction of uncertainty around decisions that cost money. If a measurement cannot plausibly change what someone does, it is not worth its cost, however elegant it is.

Marketing teams make four kinds of decisions, and each needs a different kind of evidence:

Decision	Typical question	Evidence that answers it	Precision needed
Level	How much should we spend in total next quarter?	Marketing mix model, aggregate experiments	Low to medium
Allocation	Where should the next euro go?	Response curves, marginal returns by channel and market	Medium
Keep or kill	Does this channel or tactic work at all?	Incrementality experiment	Medium to high
Execution	Which creative, audience or bid strategy?	A/B tests inside a platform, platform reporting	Relative ranking only

Two things follow from this table. First, no single method answers all four. An MMM is good at level and allocation, poor at creative choice. A platform A/B test is good at execution, silent on whether the channel is incremental at all. Second, the precision you need depends on the decision. If a channel's return would have to fall by half before you changed its budget, an estimate within plus or minus 30% is already enough. Spending another quarter to tighten that interval is wasted effort.

This is the idea of **value of information**: a measurement is worth what it saves you in expected bad decisions. A practical version of it is to ask, before any analysis, "what would

we do differently if the answer came out high, and what if it came out low?" If both answers lead to the same action, skip the analysis.

1.2 Three numbers that get confused

Most arguments about marketing performance are really arguments between three different quantities that share the same name.

Quantity	Definition	Where it comes from	What it is good for
Attributed	Conversions a tracking rule assigns to a channel	Platform reporting, analytics tools, multi-touch attribution	Relative comparisons within a channel; operational optimization
Incremental (average)	Conversions that would not have happened without the channel, divided over its total spend	Experiments, well-identified models	Keep-or-kill decisions; checking whether a channel works at all
Marginal	Extra conversions from the *next* unit of spend	Response curves from MMM or dose-response experiments	Allocation and budget level

Attributed and incremental numbers can differ by an order of magnitude. Average and marginal numbers differ whenever returns diminish, which is almost always. A channel can have an excellent average incremental return and a terrible marginal one because it is already saturated. Budget decisions are made at the margin, so the marginal number is the one that matters for allocation, and it is also the one hardest to measure.

1.3 Worked example: how attribution flatters retargeting

To see how far apart attributed and incremental numbers can be, we simulate a market where we know the truth. Each of 200,000 users has a baseline probability of converting with no advertising at all. Two channels run:

- **Retargeting** reaches users who have already shown interest, so its audience is selected on intent. It raises conversion probability by a true 0.4 percentage points.
- **Prospecting** reaches users roughly at random and raises conversion probability by a true 1.0 percentage point.

Conversions are credited with a last-touch rule: if a user saw both, retargeting gets the credit because it comes later in the journey.

```python
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 200_000

intent = rng.beta(1, 30, n)  # baseline conversion probability (mean ~3%)
# retargeting is selected on intent; prospecting reaches people roughly at random
exposed_rt = rng.random(n) < np.clip(intent * 6, 0, 0.9)
exposed_pr = rng.random(n) < 0.25

LIFT_RT, LIFT_PR = 0.004, 0.010   # true absolute lifts
u = rng.random(n)            # one draw per user, reused for the counterfactuals

def converts(rt_on=True, pr_on=True):
    p = intent + LIFT_RT * exposed_rt * rt_on + LIFT_PR * exposed_pr * pr_on
    return u < p

actual = converts()
incremental = {
    "Retargeting": actual.sum() - converts(rt_on=False).sum(),
    "Prospecting": actual.sum() - converts(pr_on=False).sum(),
}
attributed = {   # last touch: retargeting wins whenever both touched the user
    "Retargeting": (actual & exposed_rt).sum(),
    "Prospecting": (actual & exposed_pr & ~exposed_rt).sum(),
}
spend = {"Retargeting": 30_000, "Prospecting": 60_000}

tbl = pd.DataFrame({"Spend (EUR)": spend, "Attributed conv.": attributed,
                    "Incremental conv.": incremental})
tbl["Attributed CPA"] = (tbl["Spend (EUR)"] / tbl["Attributed conv."]).round(0)
tbl["Incremental CPA"] = (tbl["Spend (EUR)"] / tbl["Incremental conv."]).round(0)
tbl["Attributed / incr."] = (tbl["Attributed conv."] / tbl["Incremental
conv."]).round(1)
print(tbl.T.to_string())   # transposed to fit the page
```

```
##                     Retargeting  Prospecting
## Spend (EUR)             30000.0      60000.0
## Attributed conv.         2662.0       1395.0
## Incremental conv.         154.0        508.0
## Attributed CPA             11.0         43.0
## Incremental CPA           195.0        118.0
## Attributed / incr.         17.3          2.7
```

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 3.4))
x = np.arange(2)
ax.bar(x - 0.18, tbl["Attributed CPA"], 0.36, label="Attributed CPA")
ax.bar(x + 0.18, tbl["Incremental CPA"], 0.36, label="Incremental CPA (true)")
ax.set_xticks(x, tbl.index)
ax.set_ylabel("Cost per conversion (EUR)")
ax.set_yscale("log")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.1), ncol=2)
ax.set_title("Same campaigns, two very different answers")
plt.tight_layout()
plt.show()
```

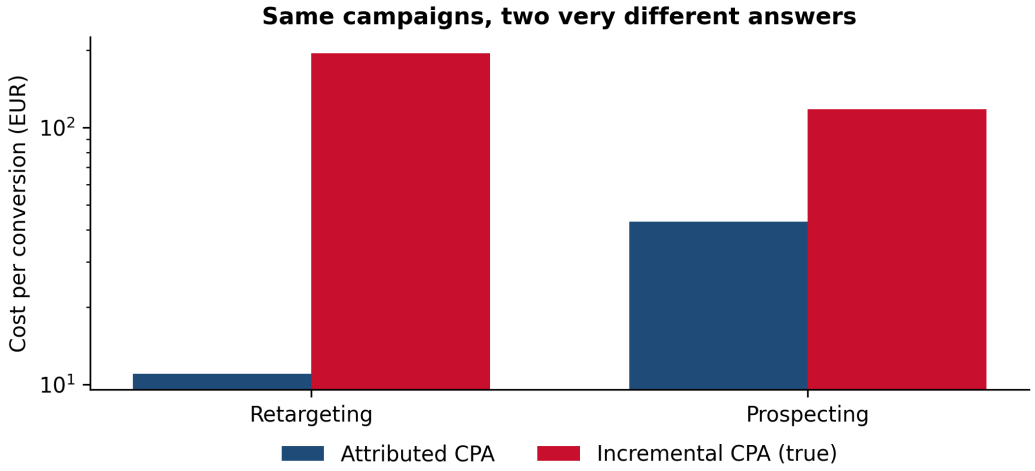


Figure 1.1: Attributed versus incremental CPA in the simulated market. Attribution ranks the channels in the wrong order.

Attribution makes retargeting look like the cheapest channel by a wide margin. In truth it is the most expensive per incremental conversion. Nothing in the attribution rule is broken; it does exactly what it says. The problem is that retargeting's audience would have converted at a high rate anyway, and last-touch credits all of those conversions to the last ad they saw. Any channel whose targeting is correlated with existing purchase intent (retargeting, branded search, CRM to engaged users, affiliates with coupon sites) will look better in attribution than it is.

The general lesson: **attribution measures who was touched before converting, not who was persuaded to convert.** Only a comparison against a counterfactual can measure the second.

> **Pitfall.**
> Presenting platform-reported ROAS or attributed CPA as if it were incremental. Platform numbers are useful for comparing ad sets within a platform, where the selection bias is roughly constant. They are not evidence that a channel pays for itself.

1.4 An evidence hierarchy

Because different sources answer different questions with different biases, a measurement program needs an explicit rule for which source wins when they disagree. One useful version, sometimes called the BEATS framework, ranks five layers from most to least authoritative:

1. **B**usiness outcomes: revenue, accounts, margin, blended CAC from finance or the data warehouse. This is ground truth for *what happened* but says nothing about *why*.
2. **E**xperiments: holdouts, geo tests, conversion lift studies. The strongest evidence for causality, but narrow in time and scope.
3. **A**nalytics and models: marketing mix models, leading indicators such as branded search and site traffic. Broad coverage, but every causal claim rests on modeling assumptions.
4. **T**racking: pixels, platform attribution, multi-touch attribution. Granular and fast, but measures correlation with conversion paths.
5. **S**urveys: self-reported attribution ("how did you hear about us?"). Useful for channels tracking cannot see (podcasts, word of mouth, offline), but subject to recall bias.

The ranking is about **authority, not usefulness**. Lower layers are faster, cheaper and more granular, which is why you use them day to day. The ranking tells you what to do when they conflict: a survey never overrides the finance numbers, and a model never overrides a well-run experiment on the same question. When a model and an experiment disagree, the right move is usually to recalibrate the model toward the experiment (Chapter 11), not to average the two.

Layer	Answers	Latency	Granularity	Main bias
Business outcomes	What happened in total	Days to months	Low	None for totals; no causal attribution
Experiments	Did X cause Y, here, now	Weeks	Low to medium	External validity (time, market, scale)
Models	How much did each driver contribute	Weeks	Medium	Specification and confounding
Tracking	Who touched what before converting	Real time	High	Selection on intent; signal loss
Surveys	What customers remember	Days	Medium	Recall and salience

1.5 Triangulation: agreement and divergence

When all layers tell the same story (blended CAC fell, an experiment shows good incremental CAC, leading indicators are up, platform CPAs fell, and more survey respondents name the channel), you can act with confidence. The interesting cases are divergences, and each pattern points to a specific diagnosis:

- **Platform says up, business says flat.** The platform is probably claiming credit for conversions that would have happened anyway, or modeled conversions have changed. Check for a tracking or attribution-window change before anything else.
- **Experiment says strong, model says weak.** The model may be under-crediting the channel because its spend is collinear with another driver, or the experiment ran in an unrepresentative period or market. Recalibrate the model with the experiment and check whether the test conditions match normal operation.
- **Model says strong, nothing else does.** Be suspicious. A channel whose spend rises with seasonal demand can absorb credit for the season (Chapter 3). Look for a test that can confirm it.
- **Survey says strong, tracking says weak.** Often a channel tracking cannot see, such as podcasts, video, word of mouth or offline. This is a candidate for a geo test, not a reason to dismiss either source.

Divergence is information. It tells you either that one source is broken or that you have learned something about how the channel works.

1.6 Running measurement as a program

Individual analyses rarely change much. A program that keeps a small set of methods in a feedback loop does. A minimal operating model has four parts:

1. **Always-on reporting** from the business layer, cut by market and channel, with a log of every tracking change, naming change and promotion so that nobody compares across a break unknowingly (Chapter 2).
2. **A model** (usually an MMM) refreshed on a fixed cadence, monthly or quarterly, used for level and allocation decisions.
3. **An experiment calendar** that targets the model's biggest uncertainties, not whatever a channel owner happens to want tested.
4. **A calibration loop**: each experiment result is fed back into the model as a prior or constraint, and the model's predictions are checked against the next experiment.

Deciding what to test next is itself a prioritization problem. A simple scoring rule works well in practice:

\[\text{priority} \;\propto\; \text{spend} \times \text{uncertainty in incremental return} \times P(\text{result changes the budget}) \]

A large channel with a well-established return scores low on uncertainty. A small channel scores low on spend. The best candidates are large channels whose incremental return nobody really knows, which in most companies means paid social prospecting, video and upper-funnel display.

> **Pitfall.**
> Measuring what is easy instead of what is uncertain. Teams often re-test paid search because it is easy to test, while the channels absorbing the most unexplained spend go untested for years.

1.7 Checklist

Checklist

- What decision will this analysis inform, and who will make it?
- What would we do if the answer came out high? If it came out low? If both are the same, stop.
- Is the question about attributed, incremental or marginal performance? Is the method able to answer that question?
- Which evidence layer does the method belong to, and does a higher layer already disagree with it?
- What precision is needed for the decision, and can the planned design deliver it?
- Is there a tracking change, naming change or promotion in the window that would break a comparison?

1.8 Further reading

- *Google's Measurement Playbook* and *Brand Measurement Playbook*. Clear vendor-side statements of the experiment, model and attribution triad.
- Kenneth Wilbur's *Advertising Measurement* lecture slides. A rigorous academic overview of why observational ad measurement is hard.
- *How to game attribution models* (Admend, archived). A short, sobering list of the ways attribution rules can be exploited.

Full links are in Appendix A.

Chapter 2 Data and signal foundations

> **The decision this chapter supports.**
> Whether your data can support the analysis you are about to run. Most failed measurement projects fail here, quietly, long before any model is fitted.

2.1 The measurement dataset

Almost every method in this book (MMM, geo tests, quasi-experiments, channel reporting) consumes the same underlying object: a panel of **spend, delivery and outcomes by date, market and channel**. Building that panel once, cleanly, and reusing it is the highest-leverage data work a measurement team can do.

Field group	Examples	Notes
Keys	date, market (country, region, DMA), channel, sub-channel, campaign	Fix the grain first. Daily by market by channel covers most needs.
Spend	media cost in a single currency, fees	Agree whether agency and tech fees are included, and keep it constant.
Delivery	impressions, clicks, reach, GRPs	Needed when spend and delivery decouple (CPM changes, auction shifts).
Outcomes (system of record)	sign-ups, accounts opened, first transactions, revenue	From the backend or warehouse, not from pixels.
Outcomes (tracked)	platform conversions, analytics conversions	Kept separately; used for diagnostics, never as the target.
Controls	price, promotions, product launches, holidays, competitor activity, macro indicators	Anything else that moves the outcome.
Events	tracking changes, naming changes, outages, policy changes	See the change log below.

Two rules save a lot of pain. First, **keep tracked and system-of-record outcomes in separate columns** and never let one silently replace the other. Second, **store the raw platform extracts** and build the panel from them with versioned code, so that a mapping change can be replayed over history instead of only applying going forward.

2.2 Naming and taxonomy

Channel-level analysis is only as good as the mapping from campaign to channel. A campaign called `DE_Search_Brand_Q3` is easy; one called `test_new_v2_final` is not. A workable taxonomy has three properties:

- **A naming convention enforced at creation**, with fixed positions for market, channel, funnel stage, objective and audience, separated by a fixed delimiter.
- **A mapping table** for anything that does not follow the convention, owned by a named person and versioned.
- **A fallback bucket** ("unmapped") that is monitored. If unmapped spend exceeds a small share of total spend, channel-level numbers are not reliable that week.

> **Pitfall.**
> Renaming or regrouping campaigns mid-period. If "Paid Social" is split into "Prospecting" and "Retargeting" in June, every year-on-year comparison at the sub-channel level is broken for twelve months unless history is remapped.

2.3 Tracked versus system-of-record outcomes

Tracked outcomes come from pixels, SDKs and tags: they are what the ad platforms and web analytics tools see. System-of-record outcomes come from your own backend: accounts actually opened, payments actually received. The two diverge for many reasons, most of which have nothing to do with marketing:

- users decline consent or block trackers;
- the same person converts on a different device or browser than the one they clicked on;
- the platform models conversions it cannot observe and adds them to the count;
- attribution windows and counting rules differ between tools;
- tags break during site releases.

Use the system of record as the outcome for every incrementality question. Use tracked outcomes for what they are good at: fast, granular, relative signals inside a channel. And monitor the **ratio** between the two, because a sudden change in that ratio is the most reliable early warning of a tracking problem.

2.4 Signal loss

Since around 2020, the amount of user-level data available to measure advertising has fallen sharply, and it is not coming back.

- **Mobile app tracking.** Apple's App Tracking Transparency (introduced with iOS 14.5) requires opt-in for cross-app tracking. Most users decline. App campaign measurement on iOS now runs largely through aggregated, delayed, privacy-thresholded postbacks.
- **Browsers.** Safari and Firefox block third-party cookies by default and limit the lifetime of some first-party cookies. Chrome's plans have shifted several times; do not build a measurement strategy that depends on any browser's behavior staying fixed.
- **Consent regimes.** In the EU and UK, GDPR and ePrivacy rules mean a meaningful share of users never consent to marketing tags. Consent rates vary by country, by device and by banner design, which means tracking coverage itself varies by market.
- **Modeled conversions.** Platforms fill the gap with statistical models of the conversions they can no longer observe. The reported number is therefore partly a model output whose method you cannot inspect.

The practical consequences are consistent. Platform-reported conversions are less complete and less comparable over time than they used to be. User-level methods such as multi-touch attribution have lost much of their input. Aggregate methods, which only need spend and outcomes by date and market, are largely unaffected. That is a large part of why marketing mix models and geo experiments have come back into fashion.

2.5 The change log

Every analysis that compares two periods implicitly assumes nothing else changed between them. In practice something always did. A **change log** is a single table, maintained by the measurement team, recording every event that could break a comparison:

Date	Market	Type	Description	Affects
2025-03-04	All	Tracking	New consent banner design	Tracked conversions, all web channels
2025-04-15	DE	Promo	Spring bonus offer launched	All outcomes in DE
2025-05-02	All	Naming	Paid social split into prospecting and retargeting	Sub-channel reporting
2025-06-10	FR	Outage	Sign-up form errors for 6 hours	Sign-ups in FR

The log costs a few minutes a week and prevents the most common analytical error in marketing: attributing a change in a metric to a campaign when it was really caused by a

tracking or product change in the same window.

2.6 Worked example: a campaign that seems to hurt

We simulate a year of daily data for one market. The system of record (backend sign-ups) has a weekly pattern, mild growth and noise. On day 200 a new campaign launches that truly lifts sign-ups by 4%. Ten days earlier, on day 190, a consent banner change silently reduces the share of sign-ups the tracking tools can see by 12%.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
days = np.arange(365)
weekly = 1 + 0.15 * np.sin(2 * np.pi * days / 7)
trend = 1 + 0.0004 * days
campaign = np.where(days >= 200, 1.04, 1.0)            # true +4% lift from day 200
backend = rng.poisson(400 * weekly * trend * campaign)  # system of record

coverage = np.where(days >= 190, 0.62, 0.70)            # tracking coverage drops on
day 190
tracked = rng.binomial(backend, coverage)

df = pd.DataFrame({"day": days, "backend": backend, "tracked": tracked})
pre, post = df.query("170 <= day < 200"), df.query("200 <= day < 230")
for col in ["tracked", "backend"]:
    change = post[col].mean() / pre[col].mean() - 1
    print(f"{col:8s}: 30 days after vs 30 days before launch = {change:+.1%}")
```

```
## tracked : 30 days after vs 30 days before launch = -4.1%
## backend : 30 days after vs 30 days before launch = +3.4%
```

Read on tracked conversions, a naive before-and-after comparison says the campaign coincided with a drop. Read on the backend, it says the opposite. Neither 30-day comparison is a valid estimate of the campaign effect (Chapter 3 explains why), but the tracked version is not even measuring the right thing.

The tracking break is easy to find if you look at the ratio of tracked to backend outcomes. Below, a simple least-squares search finds the single day that best splits the ratio series into two segments with different means.

```
import matplotlib.pyplot as plt

ratio = (df["tracked"] / df["backend"]).to_numpy()

def best_split(x, min_seg=14):
    """Single change point minimizing within-segment squared error."""
    costs = [((x[:k] - x[:k].mean())**2).sum() + ((x[k:] - x[k:].mean())**2).sum()
             for k in range(min_seg, len(x) - min_seg)]
    k = int(np.argmin(costs)) + min_seg
    return k, x[:k].mean(), x[k:].mean()

k, before, after = best_split(ratio)
print(f"Break detected at day {k}: ratio {before:.3f} -> {after:.3f}")
```

```
## Break detected at day 190: ratio 0.698 -> 0.620
```

```
fig, ax = plt.subplots(figsize=(7, 3.2))
ax.plot(days, ratio, lw=0.8, color="#6B6B6B", label="Daily tracked / backend")
ax.hlines([before, after], [0, k], [k, 364], color="#1F4E79", lw=2, label="Segment
means")
ax.axvline(k, ls="--", color="#C8102E", lw=1.2, label="Detected break")
ax.axvline(200, ls=":", color="#9A5B00", lw=1.5, label="Campaign launch")
ax.set_xlabel("Day"); ax.set_ylabel("Ratio")
ax.legend(loc="lower left", fontsize=8, ncol=2)
plt.tight_layout()
plt.show()
```

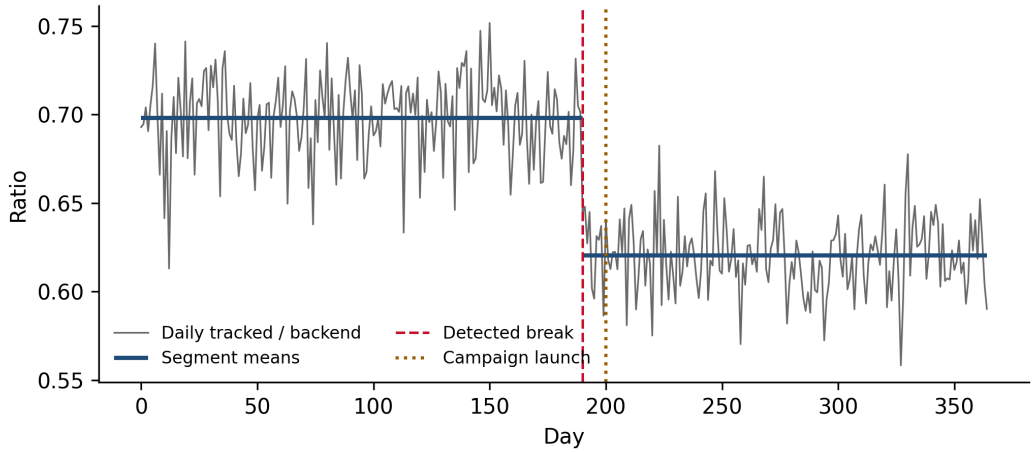


Figure 2.1: Tracked-to-backend ratio. The detected break (dashed) lands on the consent change, ten days before the campaign launch (dotted).

In real work, replace the toy search with a proper change-point library (the `ruptures` package in Python, `changepoint` in R) and run it as a scheduled check on the ratio for every market and

channel. A break in the ratio does not tell you what happened, only where to look; the change log usually tells you the rest.

2.7 Checks before any analysis

Before fitting anything, run a short battery of checks on the panel. Each one corresponds to a failure that has burned real projects.

Check	What you are looking for	Typical cause
Completeness	Missing days, markets or channels; zero spend that should not be zero	Failed extracts, API limits, new accounts not connected
Duplicates	The same spend counted twice	Overlapping extracts, agency and platform reporting the same campaign
Currency and time zone	Spend jumps at month boundaries, conversions shifted by a day	Mixed currencies, platform time zone differs from warehouse
Unmapped share	Spend not assigned to any channel	New campaigns that break the naming convention
Ratio stability	Breaks in tracked / backend, clicks / impressions, spend / impressions	Tracking changes, auction or pricing changes
Outliers	Single days far from the series	Outages, bot traffic, data errors, real but one-off events
Variation	Channels whose spend barely moves	Always-on channels; a model cannot learn their effect from history

The last check matters more than it looks. A model can only learn the effect of a channel from variation in its spend. A channel that ran at the same level every week for two years has no variation to learn from, and no amount of modeling will fix that. The fix is to create variation on purpose, which is one of the strongest arguments for running experiments (Part II).

2.8 What to request for a model or a test

When you start an MMM or plan a geo test, the data request is the first deliverable. A good request specifies:

- **History.** At least two years of weekly data for an MMM, three if you want to separate seasonality from trend with confidence. For a geo test, at least as many pre-period weeks as you plan to test, and preferably several times more.
- **Grain.** Weekly or daily, by the smallest geography at which media can be bought separately.
- **Media.** Spend and a delivery metric (impressions or GRPs) per channel, per geography, per period, including channels that are hard to collect such as TV, out-of-home, podcasts and sponsorships.
- **Outcomes.** The business outcome from the system of record, plus any intermediate outcome you want to model (site visits, sign-ups, activations).
- **Controls.** Prices, promotions, product changes, distribution changes, holidays, and any known external shocks.
- **Events.** The change log for the full period.

> **Pitfall.**
> Starting to model before the data request is complete. Missing channels do not disappear; their effect is absorbed by whichever included variable happens to be correlated with them, and the model gives that variable the credit.

2.9 Checklist

> **Checklist**
> - Is the outcome taken from the system of record, not from tracking?
> - Are tracked and backend outcomes stored side by side, with their ratio monitored?
> - Is there a change log, and does any entry overlap the analysis window?
> - Is unmapped spend below an agreed threshold for every week in the window?
> - Have currency, time zone and duplicate checks been run?
> - Does every channel you want to measure have enough spend variation to learn from?
> - Are all channels included, including offline ones, so that omitted spend does not bias the rest?

2.10 Further reading

- *How does IDFA deprecation impact ad prices?* (Mobile Dev Memo). How signal loss changes both measurement and media prices.

- `ruptures` (Python) for change-point detection, and exploratory tools such as `DataExplorer`, `naniar` and `sweetviz` for profiling a new dataset quickly.

Full links are in Appendix A.

Chapter 3 Causal thinking for marketers

> **The decision this chapter supports.**
> Whether an observed relationship between marketing and outcomes can be read as an effect, what you need to control for (and what you must not control for), and which identification strategy a question needs before anyone opens a dataset.

3.1 Every marketing question is a counterfactual

"Did the campaign work?" means "how many conversions happened that would not have happened without it?" The second half of that sentence describes a world that did not occur. Causal inference is the discipline of estimating outcomes in worlds you cannot observe.

The standard notation makes this precise. For a unit \(i\) (a user, a market, a week) let \(Y_i(1)\) be the outcome with the campaign and \(Y_i(0)\) the outcome without it. The effect is \(Y_i(1) - Y_i(0)\). We only ever observe one of the two, which is called the fundamental problem of causal inference. Every method in this book is a different way of constructing a credible stand-in for the missing half:

- **Randomized experiments** make the stand-in a randomly chosen control group.
- **Geo experiments and synthetic control** make it a weighted combination of untreated markets.
- **Difference-in-differences** makes it the treated group's own past, adjusted by the trend in a comparison group.
- **Regression and MMM** make it a model prediction with the treatment set to zero.

The methods differ in how much they rely on assumptions you cannot check. That is the axis on which to compare them, more than precision or convenience.

3.2 Why marketing data is especially confounded

Marketing data is close to a worst case for naive analysis, because spend is rarely set independently of the outcome it is supposed to drive.

- **Budgets follow demand.** Planners spend more in peak season because that is when customers buy. Spend and sales rise together even if the ads do nothing.
- **Targeting follows intent.** Retargeting, branded search and CRM reach people who were already likely to convert (Chapter 1).
- **Some channels respond to demand mechanically.** Search spend rises when more people search, which happens when demand is high for any reason.

- **Changes cluster.** Campaigns launch with promotions, product releases and price changes. Before-and-after comparisons cannot separate them.
- **Activity bias.** People who are online a lot both see more ads and do more of everything online, including converting.

Each of these creates a correlation between spend and outcomes that is not caused by the spend. The size of that spurious correlation is often larger than the true effect.

3.3 Drawing the problem: causal graphs

A causal graph (a DAG, for directed acyclic graph) is a diagram of what causes what. Drawing one before analysis forces assumptions into the open, and it tells you exactly which variables to adjust for. Three patterns cover most situations.

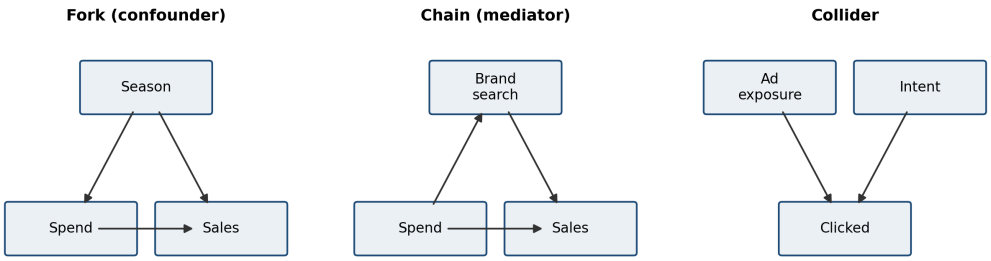


Figure 3.1: The three basic structures. Adjust for confounders; do not adjust for mediators when you want the total effect; never adjust for colliders.

Forks. Seasonality drives both spend and sales. It is a confounder: you must adjust for it, or the seasonal pattern will be credited to spend.

Chains. TV spend drives branded search, which drives sales. Branded search is a mediator: part of the TV effect flows through it. If you put branded search into the model as a control, you block that path and the model credits TV with only its direct effect. This is one of the most common errors in marketing mix models, and it systematically under-credits upper-funnel media.

Colliders. Ad exposure and existing intent both make a user more likely to click. If you restrict your analysis to clickers, you create a spurious negative correlation between exposure and intent among the people you study. Any analysis that filters on a post-treatment outcome ("users who engaged", "visitors who reached the pricing page") risks collider bias.

The rule that falls out is simple to state: **adjust for common causes of treatment and outcome, do not adjust for anything the treatment itself affects.**

3.4 Worked example: three regressions, three answers

We simulate three years of weekly data in which the truth is known.

- Underlying demand has a strong annual cycle.
- Planners set spend from a seasonal forecast, so spend is higher in peak season.
- Spend raises branded search, which raises sales, and spend also has a direct effect on sales.
- The **true total effect** of one extra unit of spend is 8 sales: 2 directly plus 1.5 × 4 through branded search.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

rng = np.random.default_rng(3)
t = np.arange(156)
season = 300 * np.sin(2 * np.pi * t / 52)                    # forecastable seasonal
demand
demand = 1000 + season + rng.normal(0, 40, t.size)
spend = 50 + 0.08 * season + rng.normal(0, 8, t.size)        # planners spend into the
peak
brand_search = 200 + 4 * spend + 0.3 * demand + rng.normal(0, 15, t.size)
sales = 0.5 * demand + 1.5 * brand_search + 2 * spend + rng.normal(0, 30, t.size)

df = pd.DataFrame({"sales": sales, "spend": spend, "brand_search": brand_search,
                   "s1": np.sin(2*np.pi*t/52), "c1": np.cos(2*np.pi*t/52)})

models = {
    "Naive: sales ~ spend": "sales ~ spend",
    "Adjusted: + seasonality": "sales ~ spend + s1 + c1",
    "Over-adjusted: + brand search": "sales ~ spend + s1 + c1 + brand_search",
}
rows = []
for name, f in models.items():
    fit = smf.ols(f, df).fit()
    lo, hi = fit.conf_int().loc["spend"]
    rows.append({"Model": name, "Estimate": fit.params["spend"],
                 "95% CI low": lo, "95% CI high": hi})
res = pd.DataFrame(rows).set_index("Model").round(1)
res["True effect"] = 8.0
print(res.to_string())
```

```
##                                Estimate  95% CI low  95% CI high  True effect
## Model
## Naive: sales ~ spend               17.5        16.7         18.3          8.0
## Adjusted: + seasonality             8.4         7.2          9.6          8.0
## Over-adjusted: + brand search      -0.8        -2.3          0.6          8.0
```

```python
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 2.6))
y = np.arange(len(res))[::-1]
err = [res["Estimate"] - res["95% CI low"], res["95% CI high"] - res["Estimate"]]
ax.errorbar(res["Estimate"], y, xerr=err, fmt="o", color="#1F4E79", capsize=4)
ax.axvline(8, color="#C8102E", ls="--", lw=1.2, label="True total effect")
ax.set_yticks(y, res.index); ax.set_xlabel("Sales per unit of spend")
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

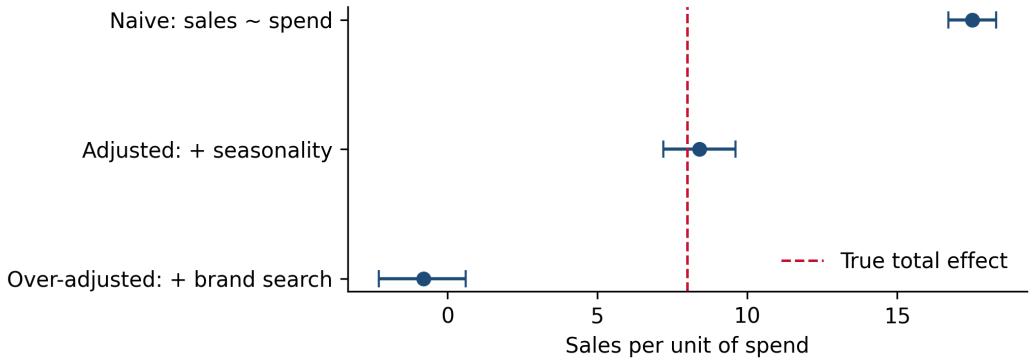


Figure 3.2: Estimated effect of spend under three specifications. Only the model that adjusts for the confounder and leaves the mediator alone recovers the truth.

The naive regression roughly doubles the true effect, because spend rides on the seasonal peak and gets credit for it. Adding seasonal terms removes the confounding and lands near the truth with an interval that contains it.

Adding branded search does something worse than expected. You might guess it would cut the estimate to the direct effect of 2, since it blocks the path through search. Instead the estimate falls to around zero. The reason is that branded search is not only a mediator: it is also driven by demand fluctuations the model cannot see. That makes it a collider between spend and unmeasured demand, and conditioning on it opens a spurious path that pushes the spend coefficient down further. In real data, where branded search responds to everything from news coverage to competitor activity, this is the normal situation, not a contrived one.

Notice that all three regressions fit the data well and all three produce tight confidence intervals. **Statistical precision tells you nothing about whether the specification is right.** Only the causal structure, which comes from domain knowledge rather than from the data, tells you which estimate to believe.

The example is also generous to the analyst in one respect: planners reacted only to the *forecastable* seasonal pattern, which the sine and cosine terms capture completely. If planners also reacted to unexpected demand (a competitor exits, a news story breaks), seasonal controls would not remove that part of the confounding, and the adjusted estimate would still be biased upward. That is the general weakness of adjustment methods: they only remove confounding you have measured.

> **Pitfall.**
> Including branded search, site visits or any other downstream metric as a control in a model meant to measure the total effect of upper-funnel media. If the variable sits between the media and the outcome, controlling for it removes part of the effect you are trying to measure.

3.5 A ladder of identification strategies

Each strategy for estimating a causal effect rests on an assumption. The higher up the ladder, the weaker and more checkable the assumption.

Strategy	Key assumption	How checkable	Covered in
Randomized experiment (user-level)	Randomization was done correctly and nothing leaks between groups	High: balance checks, A/A tests	Chapter 5
Geo experiment	Control markets would have followed the treated markets' path	Medium-high: pre-period fit	Chapter 6
Synthetic control, CausalImpact	Weighted controls track the treated unit, and no other shock hits it in the test window	Medium: pre-period fit, placebo tests	Chapter 7
Difference-in-differences	Parallel trends between treated and comparison groups	Medium: pre-trend checks	Chapter 7
Regression adjustment, MMM	All confounders are measured and the functional form is right	Low: relies on domain knowledge	Chapters 9 to 11
Raw correlation, before-and-after	Nothing else changed	Essentially none	Avoid

Two practical consequences. First, if a decision is large and the question can be answered with a method higher up the ladder, it usually should be. Second, when you must use a method lower down, say so in the readout and name the confounders you could not rule out.

3.6 Reporting an observational result honestly

An observational analysis is not worthless; it is often the only option. But the reader needs to know what kind of evidence it is. A short template:

1. **Design.** "This is an observational before-and-after comparison with seasonal controls. It is not an experiment."
2. **Estimate with uncertainty.** The point estimate, its interval, the sample size and the time window.
3. **Confounders not ruled out.** "A price change in week 14 and a competitor's campaign in weeks 15 to 18 overlap the window and could explain part of the change."
4. **Direction of likely bias.** "Both would push the estimate up, so the true effect is more likely below the estimate than above it."

5. **What would settle it.** "A four-week geo holdout in two matched markets would give a direct estimate."

3.7 Checklist

Checklist

- Have you written down the counterfactual: what exactly would have happened without the treatment?
- Have you drawn the causal graph, including how the budget was set?
- Which variables are confounders (adjust) and which are mediators or colliders (do not adjust)?
- Does the analysis filter on anything that happens after treatment?
- Where on the identification ladder does the method sit, and is the decision large enough to justify climbing higher?
- Does the readout state the design, the uncertainty and the confounders that remain?

3.8 Further reading

- *Causal Inference: The Mixtape* (Scott Cunningham) and *Causal Inference for the Brave and True* (Matheus Facure). The two most practical free introductions; the second is written in Python.
- *Causal Inference in R* (r-causal.org) and *Applied Causal Inference Powered by ML and AI* (causalml-book.org). Deeper treatments with code.
- DAGitty, a browser tool for drawing causal graphs and finding adjustment sets.
- DoWhy and EconML (py-why), CausalML (Uber) and CausalPy (PyMC Labs). Python libraries that implement the methods in Part II.

Full links are in Appendix A.

Chapter 4 Bayesian reasoning in practice

> **The decision this chapter supports.**
> How to express what you know about a marketing effect as a probability, how to combine evidence from different sources, and how to turn uncertainty into a decision instead of a significance verdict.

4.1 Why Bayesian thinking fits marketing

This chapter is not an argument that every analysis should be Bayesian. Frequentist methods are fine, and many results in later chapters are reported with ordinary confidence intervals. It is an argument that the *way of thinking* fits marketing decisions unusually well, for four reasons.

1. **Stakeholders ask Bayesian questions.** "How likely is it that this channel pays for itself?" is a question about the probability of a hypothesis given the data. A p-value answers a different question (the probability of data this extreme if the effect were zero), and the mismatch causes endless confusion.
2. **Prior knowledge exists and should be used.** You usually know something before the data arrives: past experiments, industry benchmarks, the physical impossibility of a channel driving 300% of revenue. Ignoring it is not neutral; it is a choice to pretend you know nothing.
3. **Data is often small and noisy.** Weekly data for a two-year MMM has about 100 observations and a dozen or more parameters. Priors are what keep such models from producing absurd answers.
4. **Evidence accumulates.** A measurement program produces a stream of experiments and model refreshes. Bayesian updating gives a principled way to combine them instead of treating each one in isolation.

The mechanics are compact. Start with a **prior** distribution over the quantity of interest. Combine it with the **likelihood** of the observed data. The result is the **posterior**, which is what you now believe. Every summary you report (an estimate, an interval, a probability that the effect exceeds a threshold) is read off the posterior.

4.2 Worked example: an A/B test, read two ways

A landing page test runs with 12,000 users per arm. The control converts 360 users (3.00%) and the variant converts 402 (3.35%). The frequentist reading:

```
import numpy as np
from statsmodels.stats.proportion import proportions_ztest

conv = np.array([402, 360]); n = np.array([12_000, 12_000])
z, p = proportions_ztest(conv, n, alternative="two-sided")
rel_lift = (conv[0] / n[0]) / (conv[1] / n[1]) - 1
print(f"Observed relative lift: {rel_lift:+.1%}, two-sided p = {p:.3f}")
```

```
## Observed relative lift: +11.7%, two-sided p = 0.122
```

The result is "not significant at 5%", and in many organizations the conversation ends there, with the variant discarded. The Bayesian reading starts from a weak prior (Beta(1, 1), uniform over conversion rates) and draws from the posterior of each arm:

```
rng = np.random.default_rng(11)
draws = 200_000
post_a = rng.beta(1 + 360, 1 + 12_000 - 360, draws)   # control
post_b = rng.beta(1 + 402, 1 + 12_000 - 402, draws)   # variant
lift = post_b / post_a - 1

print(f"P(variant better than control) = {(post_b > post_a).mean():.1%}")
```

```
## P(variant better than control) = 93.9%
```

```
print(f"Relative lift: median {np.median(lift):+.1%}, 95% interval "
      f"[{np.quantile(lift, .025):+.1%}, {np.quantile(lift, .975):+.1%}]")
```

```
## Relative lift: median +11.6%, 95% interval [-3.0%, +28.5%]
```

```
# Expected loss: how much conversion rate we give up, on average, if we pick the wrong
arm
loss_ship_b = np.maximum(post_a - post_b, 0).mean()
loss_keep_a = np.maximum(post_b - post_a, 0).mean()
print(f"Expected loss if we ship the variant: {loss_ship_b*100:.3f} pp")
```

```
## Expected loss if we ship the variant: 0.006 pp
```

```
print(f"Expected loss if we keep the control: {loss_keep_a*100:.3f} pp")
```

```
## Expected loss if we keep the control: 0.356 pp
```

The data are identical; what changes is what you report. "There is roughly a 94% chance the variant is better, the likely lift is around 12% but could plausibly be anywhere from slightly

negative to nearly 30%, and the expected cost of shipping it if we are wrong is under a hundredth of a percentage point of conversion, while the expected cost of keeping the control is over a third of a point" is a statement a product owner can act on. It also makes the real trade-off visible: if the variant is cheap to ship and easy to revert, a 94% probability with a small expected loss is a good bet; if shipping it is costly or irreversible, it may be worth running longer.

Two cautions keep this honest. First, with a weak prior and a reasonable sample size, the Bayesian interval is numerically close to the frequentist one. The Bayesian framing does not create evidence that is not there. Second, the decision rule must be fixed before the test, exactly as with significance testing. "Ship if the probability of being better exceeds 95% and expected loss is below X" is a rule; looking at the posterior every day and stopping when it looks good is peeking, and it inflates false positives in Bayesian and frequentist analysis alike (Chapter 5).

4.3 Worked example: combining a model and an experiment

A marketing mix model estimates the incremental return on ad spend (iROAS) of a paid social channel at 1.8, with posterior standard deviation 0.6. A geo experiment on the same channel then estimates 1.1, with standard error 0.3. Which do you believe?

Neither on its own. If both estimates are roughly normal and measure the same quantity, the combined estimate weights each by its precision (the inverse of its variance):

$$\mu_{\text{post}} = \frac{\mu_1/\sigma_1^2 + \mu_2/\sigma_2^2}{1/\sigma_1^2 + 1/\sigma_2^2}, \qquad \sigma_{\text{post}} = \left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right)^{-1/2}$$

```
from scipy import stats

mmm = (1.8, 0.6); geo = (1.1, 0.3)
prec = [1/mmm[1]**2, 1/geo[1]**2]
mu = (mmm[0]*prec[0] + geo[0]*prec[1]) / sum(prec)
sd = sum(prec) ** -0.5

for name, (m, s) in {"MMM alone": mmm, "Geo test alone": geo, "Combined": (mu,
sd)}.items():
    p_win = 1 - stats.norm.cdf(1, m, s)
    print(f"{name:15s} iROAS {m:.2f} (sd {s:.2f})   P(iROAS > 1) = {p_win:.0%}")
```

```
## MMM alone       iROAS 1.80 (sd 0.60)   P(iROAS > 1) = 91%
## Geo test alone  iROAS 1.10 (sd 0.30)   P(iROAS > 1) = 63%
## Combined        iROAS 1.24 (sd 0.27)   P(iROAS > 1) = 81%
```

```
import matplotlib.pyplot as plt
x = np.linspace(-0.5, 4, 600)
fig, ax = plt.subplots(figsize=(7, 2.9))
curves = [(mmm, "MMM", "--"), (geo, "Geo test", ":"), ((mu, sd), "Combined", "-")]
for (m, s), lab, ls in curves:
    ax.plot(x, stats.norm.pdf(x, m, s), ls=ls, lw=2, label=f"{lab}: {m:.2f}")
ax.axvline(1, color="#6B6B6B", lw=0.8)
ax.text(1.03, ax.get_ylim()[1] * 0.9, "break-even", fontsize=8)
ax.set_xlabel("iROAS")
ax.set_yticks([])
ax.legend()
plt.tight_layout()
plt.show()
```

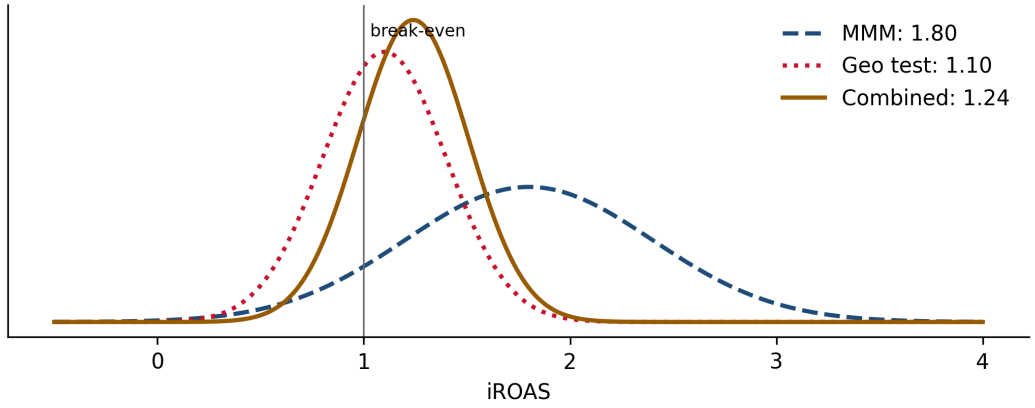


Figure 4.1: Precision-weighted combination of a model estimate and an experiment. The experiment dominates because it is more precise.

The combined estimate sits much closer to the experiment because the experiment is four times as precise. This is the core logic of **calibrating an MMM with experiments** (Chapter 11): the experiment becomes a prior, or a likelihood term, that pulls the model's channel estimate toward what was measured directly.

The formula assumes both numbers measure the same thing. In practice they often do not: the experiment ran in two markets for four weeks at a particular spend level, while the MMM coefficient is an average over two years and all markets. Before combining, check that the spend level, time period and outcome definition match closely enough. If they do not, the honest move is to model the difference (for example, by widening the experiment's uncertainty to reflect the gap) rather than pretending it is not there.

4.4 Choosing priors

Priors attract suspicion because they look subjective. In marketing models they are usually the opposite: they encode hard constraints everyone agrees on and stop the model from wandering into impossible territory. Three kinds are useful.

- **Weakly informative priors** rule out the absurd without favoring any particular plausible value. Example: a channel's ROI is positive and very unlikely to exceed 20.
- **Informative priors from evidence**, such as a past experiment or a meta-analysis of similar channels. These should be documented with their source.
- **Structural priors** on things like how quickly an ad effect decays (adstock) or how fast returns saturate, where the data alone rarely pins down the answer.

A useful habit is the **prior predictive check**: before seeing the data, simulate what the prior implies about observable quantities and ask whether that looks sane. Suppose a business spends 50,000 per week on media and makes 100,000 per week in revenue, and you are choosing a prior for the media coefficient.

```
import pandas as pd
rng = np.random.default_rng(5)
spend, revenue = 50_000, 100_000
priors = {
    "Normal(0, 10) on coefficient": rng.normal(0, 10, 100_000),
    "LogNormal(log 1, 0.5) on ROI": rng.lognormal(np.log(1.0), 0.5, 100_000),
}
rows = {}
for name, beta in priors.items():
    share = beta * spend / revenue              # implied share of revenue driven by
media
    rows[name] = {"5%": np.quantile(share, .05), "median": np.median(share),
                  "95%": np.quantile(share, .95),
                  "P(impossible)": ((share < 0) | (share > 1)).mean()}
pct = (pd.DataFrame(rows).T * 100).round(0).astype(int).astype(str) + "%"
print(pct.to_string())   # P(impossible): share below 0% or above 100%
```

```
##                                5% median   95% P(impossible)
## Normal(0, 10) on coefficient  -824%     1%  823%           92%
## LogNormal(log 1, 0.5) on ROI    22%    50%  114%            8%
```

The "uninformative" Normal(0, 10) prior says it is entirely plausible that media drives minus 800% or plus 800% of revenue, and it puts over 90% of its mass on impossible values. It is not neutral; it is wrong. The lognormal prior on ROI keeps media's share between roughly 20% and 110% of revenue, which is still generous but at least physically possible, and it encodes the belief that advertising does not destroy revenue.

The second habit is **sensitivity analysis**: refit with a meaningfully different reasonable prior and see whether the decision changes. If it does, the data are not strong enough to settle the

question, and the honest conclusion is that you need more data or an experiment, not that you should pick the prior that gives the preferred answer.

> **Pitfall.**
> Choosing priors after seeing which answer they produce. Priors must be set and documented before the model is fitted, with a source for each informative one. Otherwise the prior is just a dial for producing the result someone wanted.

4.5 From uncertainty to decisions

The point of a posterior is to make decisions, and a decision needs two ingredients a significance test does not provide: the size of the possible gain and loss, and how much each outcome matters.

Consider a budget decision on the paid social channel from the example above, where break-even is an iROAS of 1. The combined posterior says the channel probably pays back, but not certainly. The expected value of an extra 100,000 in spend is

\[E[\text{profit}] = 100{,}000 \times (E[\text{iROAS}] - 1), \]

and you can also compute the probability of losing money and the expected size of that loss. A useful habit is to report all three: expected gain, probability of loss, and expected loss if wrong. Two channels with the same 80% probability of being profitable can be very different bets if one has a long left tail.

Bayesian decision analysis also gives a clean answer to "should we test this?": run the experiment if the expected reduction in bad-decision losses exceeds the cost of the test (including the revenue lost in the holdout). This is the value of information idea from Chapter 1, made quantitative.

4.6 Checklist

Checklist

- Are priors set and documented before fitting, with a source for each informative prior?
- Has a prior predictive check confirmed that the priors imply plausible outcomes?
- Has a sensitivity analysis shown whether a different reasonable prior changes the decision?
- When combining a model and an experiment, do they measure the same quantity (spend level, period, market, outcome)?
- Was the decision rule (thresholds on probability and expected loss) fixed before looking at results?
- Does the readout report probability of loss and expected loss, not just a point estimate?

4.7 Further reading

- *Bayes Rules!* (Johnson, Ott and Dogucu) and *Statistical Thinking for the 21st Century* (Poldrack). Accessible introductions with code.
- *Calibration with experiments* (Juan Orduz). The bridge from this chapter to Bayesian MMM calibration in Part III.
- PyMC and PyMC-Marketing documentation, which Part III uses throughout.

Full links are in Appendix A.

Chapter 5 A/B tests that hold up

> **The decision this chapter supports.**
> Whether a change (a page, an offer, a message, an onboarding step) improved the outcome enough to ship, and how to design the test so that the answer is trustworthy whichever way it comes out.

5.1 Design before data

An A/B test is the strongest evidence in marketing science because randomization breaks every confounding path at once. It is also the easiest to ruin, because every decision made after the data starts arriving is a chance to fool yourself. The fix is to write the design down before launch. A one-page test plan should fix:

Element	What to decide	Common mistake
Hypothesis	What change, for whom, expected direction and mechanism	"Let's see what happens"
Randomization unit	User, account, session, device, household	Randomizing sessions when the outcome is per user
Primary metric	One metric that decides the test, defined precisely, with its window	Picking the metric that moved after the fact
Guardrail metrics	Metrics that must not get worse (e.g. activation, complaints, unsubscribes)	No guardrails, so a "win" damages something downstream
Minimum detectable effect	The smallest effect worth acting on	Not computing it, then calling a null result "a small positive lift"
Sample size and duration	From the MDE, baseline rate and traffic; whole weeks	Stopping when it "looks significant"
Decision rule	What result leads to ship, iterate or stop	Deciding the rule after seeing the result
Segments	Which cuts will be examined, declared in advance	Slicing until something is significant

The plan does not have to be long. It has to exist before the data does.

5.2 Power and the minimum detectable effect

The minimum detectable effect (MDE) is the smallest true effect the test has a good chance (usually 80%) of detecting at the chosen significance level. It is the single most useful number in test planning, because it tells you in advance what a null result will mean.

For a conversion rate p and n users per arm, a good approximation of the relative MDE for a two-sided test at level α with power $1-\beta$ is

$$\text{MDE}_{\text{rel}} \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})\sqrt{2p(1-p)/n}}{p}.$$

```
import numpy as np
import pandas as pd
from scipy import stats

def rel_mde(p, n, alpha=0.05, power=0.8):
    z = stats.norm.ppf(1 - alpha / 2) + stats.norm.ppf(power)
    return z * np.sqrt(2 * p * (1 - p) / n) / p

ns = [5_000, 10_000, 25_000, 50_000, 100_000, 250_000]
tbl = pd.DataFrame({f"baseline {p:.0%}": [rel_mde(p, n) for n in ns] for p in [0.01,
0.03, 0.10]},
                   index=pd.Index(ns, name="users per arm"))
print((tbl * 100).round(1).astype(str).add("%").to_string())
```

```
##               baseline 1% baseline 3% baseline 10%
## users per arm
## 5000                55.8%       31.9%        16.8%
## 10000               39.4%       22.5%        11.9%
## 25000               24.9%       14.2%         7.5%
## 50000               17.6%       10.1%         5.3%
## 100000              12.5%        7.1%         3.8%
## 250000               7.9%        4.5%         2.4%
```

Read the table before running any test. With 10,000 users per arm and a 3% baseline, the test can only reliably detect a relative lift of roughly 20% or more. Most real changes to a landing page move conversion by a few percent. Such a test is almost guaranteed to come back null, and when it does, the honest summary is "we could not detect an effect; effects smaller than about 20% were undetectable with this sample", not "no effect".

Three levers improve the MDE, in roughly increasing order of usefulness:

1. **More traffic or a longer test.** The MDE shrinks with the square root of the sample, so halving it takes four times the data.

2. **A metric closer to the change.** A change to the pricing page is easier to detect on "started sign-up" than on "first payment 60 days later", although the closer metric may not be the one you ultimately care about.
3. **Variance reduction** (CUPED, below), which is often worth the equivalent of 20% to 50% more traffic for free.

5.3 Peeking: the most common way to ruin a test

A fixed-sample test controls the false positive rate at 5% only if you look at the result once, at the planned end. If you check every day and stop as soon as the result is significant, the false positive rate climbs far above 5%. The simulation below runs 2,000 A/A tests, where there is no true difference, for 28 days and checks significance each day.

```
import matplotlib.pyplot as plt

rng = np.random.default_rng(1)
sims, days, per_day, p = 2_000, 28, 500, 0.03
a = rng.binomial(per_day, p, (sims, days)).cumsum(1)
b = rng.binomial(per_day, p, (sims, days)).cumsum(1)
n = per_day * np.arange(1, days + 1)
pooled = (a + b) / (2 * n)
z = (b / n - a / n) / np.sqrt(pooled * (1 - pooled) * 2 / n)

daily_sig = np.abs(z) > 1.96
fp_by_day = np.maximum.accumulate(daily_sig, axis=1).mean(0)   # stopped at first
"win"

# Group sequential: 4 planned looks with O'Brien-Fleming boundaries
looks = np.array([7, 14, 21, 28]) - 1
bounds = 2.024 * np.sqrt(4 / np.arange(1, 5))
fp_gs = (np.abs(z[:, looks]) > bounds).any(1).mean()

print(f"Single look at day 28:       {daily_sig[:, -1].mean():.1%}")
```

```
## Single look at day 28:       5.5%
```

```
print(f"Daily peeking for 28 days:   {fp_by_day[-1]:.1%}")
```

```
## Daily peeking for 28 days:   28.4%
```

```
print(f"4 planned looks, OBF bounds: {fp_gs:.1%}")
```

```
## 4 planned looks, OBF bounds: 5.7%
```

```
fig, ax = plt.subplots(figsize=(7, 2.9))
ax.plot(np.arange(1, days + 1), fp_by_day, lw=2, label="Stop at first significant
daily look")
ax.axhline(0.05, color="#C8102E", ls="--", lw=1, label="Nominal 5%")
ax.set_xlabel("Days of peeking"); ax.set_ylabel("False positive rate")
ax.yaxis.set_major_formatter(plt.matplotlib.ticker.PercentFormatter(1))
ax.legend(loc="upper left")
plt.tight_layout()
plt.show()
```

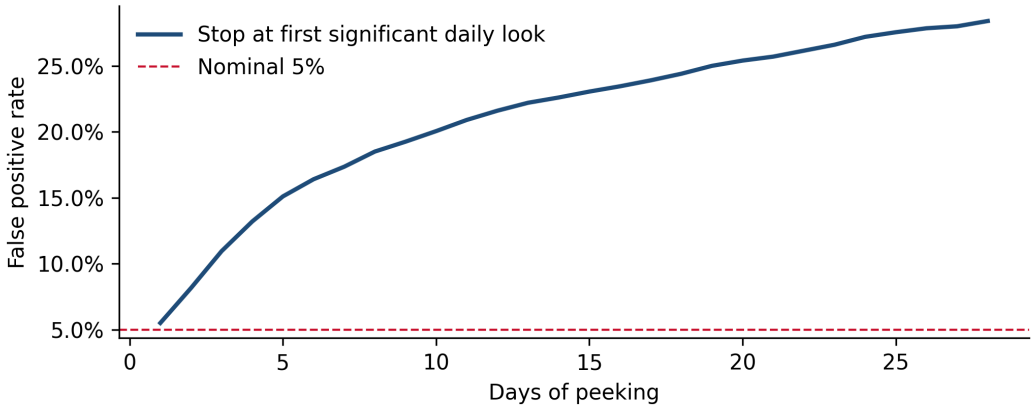


Figure 5.1: False positive rate of A/A tests when stopping at the first significant daily look, compared with a single look at the end and with a four-look group sequential design.

Daily peeking for four weeks turns a 5% false positive rate into something close to 30%: more than one in four "winning" tests is noise. The four-look group sequential design stays close to the nominal 5%. There are two honest fixes.

- **Do not peek at the decision metric.** Look at guardrails and data quality during the test, but decide only at the planned end.
- **Plan the peeks.** Group sequential designs fix a small number of interim looks in advance and use stricter thresholds at the early ones. O'Brien-Fleming boundaries, used above, demand very strong evidence early and close to the usual threshold at the end, which keeps the overall error near 5% while still allowing an early stop for a large effect. Always-valid inference (mixture sequential tests) goes further and allows continuous monitoring, at some cost in power.

If someone asks for an interim look at a fixed-sample test, give it, but say plainly that a significant interim result is not a reason to stop early under that design.

5.4 Multiple comparisons

Every extra comparison is another chance for noise to look like a finding. If a test is read on one primary metric, three secondary metrics and five countries, that is up to 20 comparisons, and at a 5% threshold you should expect about one false positive even if nothing works.

```
k = np.array([1, 3, 5, 10, 20, 50])
print(pd.DataFrame({"comparisons": k,
                    "P(at least one false positive)": (1 - 0.95 **
k).round(2)}).to_string(index=False))
```

```
##  comparisons  P(at least one false positive)
##            1                            0.05
##            3                            0.14
##            5                            0.23
##           10                            0.40
##           20                            0.64
##           50                            0.92
```

Practical rules:

- **One primary metric decides.** Everything else is supporting evidence or a guardrail.
- **Declare segments in advance** and correct for them. Holm's method controls the chance of any false positive; the Benjamini-Hochberg procedure controls the share of false discoveries and is less conservative. Both are one line with `statsmodels.stats.multitest.multipletests`.
- **Treat unplanned segment findings as hypotheses** for the next test, not as results. "The variant worked in Italy only" found after the fact is far more often noise than a real interaction.

5.5 Variance reduction with CUPED

CUPED (controlled-experiment using pre-experiment data) uses a pre-period measure of the same metric to remove predictable variation from the outcome. If \(X\) is a user's pre-period value and \(Y\) the in-test outcome, analyze \(Y - \theta(X - \bar X)\) with \(\theta = \text{cov}(X, Y)/\text{var}(X)\). Because \(X\) is measured before randomization, the adjustment cannot bias the estimate; it only shrinks the variance, by a factor of \(1 - \rho^2\) where \(\rho\) is the correlation between \(X\) and \(Y\).

```
rng = np.random.default_rng(8)
n = 20_000
pre = rng.gamma(2, 25, n)                                  # pre-period spend per user
treat = rng.random(n) < 0.5
post = 0.7 * pre + rng.gamma(2, 12, n) + 2.0 * treat   # true effect: +2.0 per user

theta = np.cov(pre, post)[0, 1] / pre.var()
adj = post - theta * (pre - pre.mean())

for name, y in [("Raw", post), ("CUPED", adj)]:
    d = y[treat].mean() - y[~treat].mean()
    se = np.sqrt(y[treat].var() / treat.sum() + y[~treat].var() / (~treat).sum())
    print(f"{name:6s} effect {d:5.2f}  95% CI [{d - 1.96*se:5.2f}, {d + 1.96*se:5.2f}]
SE {se:.3f}")
```

```
## Raw    effect  3.25  95% CI [ 2.42,  4.08]  SE 0.423
## CUPED  effect  2.37  95% CI [ 1.90,  2.84]  SE 0.242
```

```
print(f"Correlation pre/post: {np.corrcoef(pre, post)[0, 1]:.2f}")
```

```
## Correlation pre/post: 0.82
```

Two things happen. The CUPED standard error is a bit over half the raw one, which is equivalent to running the raw test with about three times the traffic. And the point estimate moves: in this particular sample the treatment arm happened to contain heavier spenders before the test started, which inflated the raw estimate well above the true 2.0. CUPED corrects for that chance imbalance and lands much closer to the truth. Randomization guarantees balance on average, not in every sample; CUPED fixes the sample you actually got. CUPED works best for metrics with stable individual behavior (spend, sessions, transactions) and does nothing for brand-new users with no history, which is a real limitation for acquisition tests.

5.6 Checks that catch broken tests

Sample ratio mismatch (SRM). If the design splits traffic 50/50, the observed split should be close to 50/50. A significant deviation means something is wrong with assignment or logging (redirects that drop users, bot filtering applied to one arm, a variant that crashes on some browsers), and the result cannot be trusted whatever it shows.

```
observed = np.array([50_600, 49_400])
chi2, p = stats.chisquare(observed)
print(f"Split {observed[0] / observed.sum():.2%} / {observed[1] / observed.sum():.2%}:
SRM p-value = {p:.5f}")
```

```
## Split 50.60% / 49.40%: SRM p-value = 0.00015
```

A 50.6/49.4 split looks harmless but is very unlikely by chance at this sample size. Investigate before reading the outcome.

A/A tests. Running the testing setup with two identical arms is the cheapest way to verify that randomization, logging and analysis produce about 5% false positives. Run one whenever the tooling changes.

Novelty and learning effects. A new design can win for two weeks because it is new, or lose for two weeks because users have to relearn something. Plot the daily effect; if it trends, the end-of-test average does not describe the long-run effect.

Interference. If treated and control users interact (referrals, shared accounts, marketplaces, limited inventory), the control is contaminated and the estimate is biased. Randomize at a higher level (company, region) when this matters.

Long outcome windows. Many marketing outcomes (activation, revenue, retention) arrive weeks or months after exposure. A test read at two weeks measures the short-term outcome only. When the decision depends on revenue or retention, define the readout as a cohort metric at a fixed horizon (for example 30, 60 or 90 days after assignment) and plan the timeline around it.

> **Pitfall.**
> Treating an ad platform's split test as an incrementality test. A platform A/B test compares two versions of an ad against each other; both arms are still advertising. It tells you which version performs better relative to the other, not whether either one causes conversions that would not have happened anyway. For that you need a holdout (Chapter 8) or a geo test (Chapter 6).

5.7 Tests before the product exists

Some of the most valuable tests happen before anything is built.

- **Smoke tests and fake doors** present an offer, feature or product that does not exist yet and measure how many people try to take it. They size demand cheaply, at some cost in user trust if overused.
- **Shadow brands** run a test campaign under an unrelated brand name to measure response to an offer or positioning without exposing the main brand.
- **Creative pre-testing** with fractional factorial designs (sometimes called Taguchi or orthogonal-array testing) estimates the effect of several creative elements at once with far

fewer cells than a full factorial, at the cost of assuming most interactions are small.

5.8 Reporting a test

A readout that survives scrutiny reports, in this order: the decision and how confident it is; the design (unit, split, dates, sample size, primary metric and window); the estimate with its interval; the MDE the test had; guardrails; and any data quality issues found (SRM, logging gaps). A non-significant result is reported as "we could not detect an effect larger than X", which is a real and useful result.

5.9 Checklist

Checklist

- Is there a written plan with hypothesis, unit, primary metric and window, MDE, sample size, duration and decision rule, dated before launch?
- Does the MDE correspond to an effect size worth acting on? If not, redesign or do not run the test.
- Does the test run in whole weeks and long enough for the outcome window?
- Was the sample ratio checked, and were A/A tests run on this setup?
- Is the decision metric read only at the planned end, or under a pre-planned sequential design?
- How many comparisons does the readout make, and are they corrected or labeled as exploratory?
- Is a null result reported with the effect size that could have been detected?

5.10 Further reading

- *abtest.design*. A gallery of real A/B tests, useful for calibrating what effect sizes are realistic.

Full links are in Appendix A.

Chapter 6 Geo experiments

> **The decision this chapter supports.**
> Whether a channel that cannot be randomized at the user level (TV, out-of-home, audio, paid social or search measured on business outcomes) is incremental, and by how much per euro, measured on the system-of-record outcome rather than on tracked conversions.

6.1 When geo tests are the right tool

A geo experiment randomizes **places** instead of people. Some regions get a change in media (more, less or none) and the rest continue as usual. Because the outcome is measured as an aggregate per region and day, the method needs no user-level tracking at all. That makes it the natural choice when:

- the channel cannot be randomized at the user level (TV, radio, out-of-home, print);
- you want the outcome from your own backend, including offline and cross-device conversions, instead of the platform's tracked conversions;
- you want to check a platform's own lift study with an independent method (Chapter 8);
- you need an experiment to calibrate a marketing mix model (Chapter 11), which works at the same aggregate level.

The price is statistical power. You have tens of units, not millions, and regions are noisy and different from each other. Most of the craft of geo testing is about getting enough power out of few, heterogeneous units.

6.2 Design choices

Unit. The smallest geography at which media can be bought separately and outcomes can be measured reliably. In the US this is often the DMA. In Europe, whole countries are almost always too few and too different to serve as units; use administrative regions (NUTS-2 or NUTS-3), postal-code areas or commuting zones, which group areas where people live and work together and so reduce spillover.

Number of units and treated share. More units means more power. With 30 to 60 regions, treating 20% to 50% of them is typical; the best split depends on how much of the business you are willing to put at risk.

Assignment. Two approaches work:

- *Stratified randomization*: sort regions by size (or by a pre-period outcome), form blocks of similar regions, and randomly pick treated regions within each block. This keeps the design-based guarantee of randomization while balancing size.
- *Matched selection*: choose treated regions and controls to maximize pre-period similarity (as GeoLift and Google's matched markets tools do). It can increase power, but the guarantee now rests on the pre-period fit carrying over into the test period.

Treatment type.

Treatment	What changes	What it measures
Go-dark holdout	Media switched off in treated regions	Average incremental return of current spend
Heavy-up	Extra spend in treated regions	Marginal return of extra spend at the current level
Scale-back (inverse)	Spend reduced in treated regions	Marginal return of the last units of spend

These answer different questions. A go-dark test tells you whether the channel works at all; a heavy-up or scale-back tells you whether the next or last euro works, which is what an allocation decision needs (Chapter 1).

Duration and cooldown. Long enough to accumulate power and to cover at least two full weekly cycles; for channels with long-lasting effects (TV, brand campaigns), add a cooldown period after the test during which the outcome is still measured, because effects persist after spend stops.

6.3 Worked example: a heavy-up test on 40 regions

We simulate 120 days of daily conversions in 40 regions of very different sizes, with a shared weekly pattern, a shared trend, a national random walk and slow region-specific drift. After 90 pre-period days, 8 treated regions receive extra spend for 30 days, with a true lift of 6%.

Assignment is stratified: regions are sorted by pre-period size into 8 blocks of 5, and one region per block is randomly treated.

```python
import numpy as np
import pandas as pd
from scipy.optimize import minimize

rng = np.random.default_rng(21)
G, T, T0 = 40, 120, 90                      # regions, days, first test day
t = np.arange(T)
size = rng.lognormal(np.log(80), 0.7, G)    # baseline daily conversions per region
national = 1 + 0.12 * np.sin(2 * np.pi * t / 7) + 0.0015 * t + np.cumsum(rng.normal(0,
0.006, T))
regional = 1 + np.cumsum(rng.normal(0, 0.004, (G, T)), axis=1)   # slow region-
specific drift
mu = size[:, None] * national[None, :] * regional

# Stratified randomization: 8 blocks of 5 similar-sized regions, one treated per block
pre_size = mu[:, :T0].mean(1)
blocks = np.argsort(pre_size).reshape(8, 5)
treated = np.array([rng.choice(b) for b in blocks])
is_t = np.isin(np.arange(G), treated)

LIFT = 0.06
mu_obs = mu.copy()
mu_obs[np.ix_(is_t, t >= T0)] *= 1 + LIFT
y = rng.poisson(mu_obs).astype(float)
print(f"Treated regions: {is_t.sum()}, share of pre-period conversions: "
      f"{y[is_t, :T0].sum() / y[:, :T0].sum():.0%}")
```

```
## Treated regions: 8, share of pre-period conversions: 19%
```

The estimator is a **synthetic control**: a weighted combination of control regions, with non-negative weights summing to one, chosen to track the treated regions' total as closely as possible in the pre-period. The test-period gap between the treated total and its synthetic twin is the effect estimate.

```
def synth_effect(y, treated_mask, T0):
    target = y[treated_mask].sum(0)
    X = y[~treated_mask]
    X = X * (target[:T0].mean() / X[:, :T0].mean(1))[:, None]     # put controls on
treated scale
    k = X.shape[0]
    sse = lambda w: ((target[:T0] - w @ X[:, :T0]) ** 2).sum()
    w = minimize(sse, np.full(k, 1 / k), method="SLSQP", bounds=[(0, 1)] * k,
                 constraints={"type": "eq", "fun": lambda w: w.sum() - 1}).x
    cf = w @ X
    lift = target[T0:].sum() / cf[T0:].sum() - 1
    return lift, target, cf

est, target, cf = synth_effect(y, is_t, T0)
pre_mape = np.mean(np.abs(target[:T0] / cf[:T0] - 1))
print(f"Estimated lift: {est:+.1%} (true {LIFT:+.0%}); pre-period MAPE
{pre_mape:.1%}")
```

```
## Estimated lift: +3.8% (true +6%); pre-period MAPE 3.4%
```

A point estimate alone is not a result. To get its uncertainty we use **placebo tests**: repeat the exact same procedure many times on sets of 8 control regions that received no treatment. The spread of those placebo "effects" is the noise the method produces when the truth is zero.

```
controls = np.where(~is_t)[0]
placebo = []
for _ in range(300):
    fake = np.zeros(G, bool)
    fake[rng.choice(controls, 8, replace=False)] = True
    keep = ~is_t                                          # truly treated regions
excluded
    placebo.append(synth_effect(y[keep], fake[keep], T0)[0])
placebo = np.array(placebo)

lo, hi = est - np.quantile(placebo, [0.975, 0.025])
p_val = (np.abs(placebo) >= abs(est)).mean()
print(f"Lift {est:+.1%}, 95% interval [{lo:+.1%}, {hi:+.1%}], placebo p-value
{p_val:.3f}")
```

```
## Lift +3.8%, 95% interval [+0.8%, +6.6%], placebo p-value 0.010
```

The estimate is below the true 6%, but the interval contains it and excludes zero. This is what one run of a sound design looks like: any single test lands somewhere in its interval, which is why the interval, not the point, is the result.

```
import matplotlib.pyplot as plt
roll = lambda s: pd.Series(s).rolling(7, center=True).mean()
fig, ax = plt.subplots(figsize=(7, 3))
ax.plot(t, roll(target), lw=2, label="Treated regions")
ax.plot(t, roll(cf), lw=2, ls="--", label="Synthetic control")
ax.axvspan(T0, T - 1, color="#9A5B00", alpha=0.08, label="Test period")
ax.set_xlabel("Day"); ax.set_ylabel("Daily conversions")
ax.legend(loc="upper left")
plt.tight_layout()
plt.show()
```

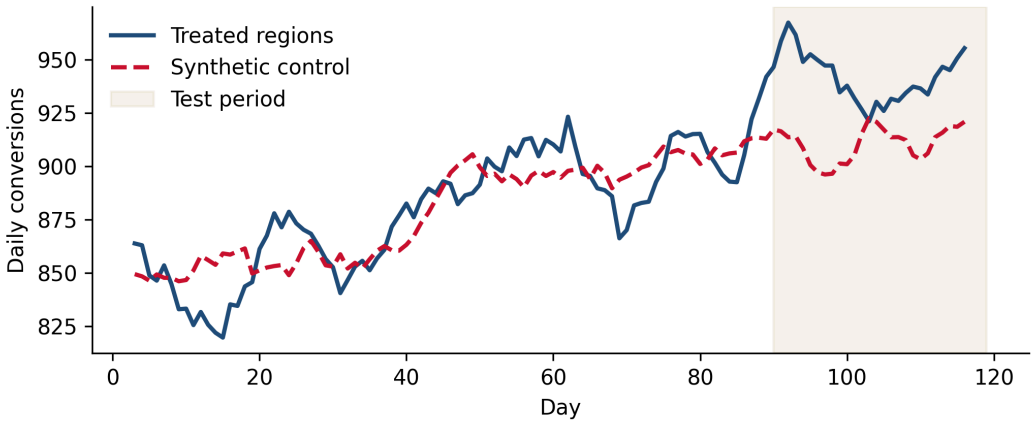


Figure 6.1: Treated regions (total) versus their synthetic control, 7-day averages. The gap after day 90 is the estimated effect of the heavy-up.

6.3.1 Power: what this design could have detected

The same placebo distribution answers the planning question. If the true lift were \(L\), each placebo estimate would shift to roughly \((1+L)(1+e)-1\), where \(e\) is its null estimate, because the injected lift scales the treated total but not the synthetic control fitted on the pre-period. The power at \(L\) is the share of those shifted estimates that clear the critical value.

```
crit = np.quantile(np.abs(placebo), 0.95)
lifts = np.linspace(0, 0.10, 41)
power = [(np.abs((1 + L) * (1 + placebo) - 1) > crit).mean() for L in lifts]
mde = lifts[np.argmax(np.array(power) >= 0.8)]
print(f"Critical |lift| at 5%: {crit:.1%};  MDE at 80% power: about {mde:.1%}")
```

```
## Critical |lift| at 5%: 2.9%;  MDE at 80% power: about 4.2%
```

```
fig, ax = plt.subplots(figsize=(7, 2.6))
ax.plot(lifts * 100, power, lw=2)
ax.axhline(0.8, ls=":", color="#6B6B6B")
ax.set_xlabel("True lift (%)")
ax.set_ylabel("Power")
plt.tight_layout()
plt.show()
```

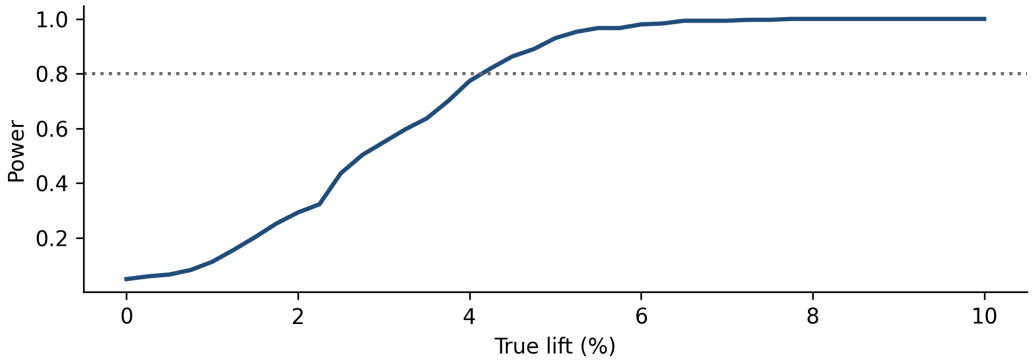


Figure 6.2: Power of the design by true lift, from the placebo distribution. The dotted line marks 80% power.

In real work this analysis runs **before** the test, on historical data only: pick candidate treated sets, run placebos on the pre-period, and see whether the MDE is small enough to be worth the cost of the test. If it is not, change the design (more regions, longer duration, a bigger spend change) before spending any money.

6.3.2 From lift to iROAS

The decision needs a return, not a lift. Suppose the heavy-up added 90,000 EUR of spend in the treated regions over the 30 days and each conversion is worth 120 EUR.

```
extra_spend, value = 90_000, 120
cf_post = cf[T0:].sum()
for label, L in [("estimate", est), ("95% low", lo), ("95% high", hi)]:
    incr = cf_post * L
    print(f"{label:9s} incremental conversions {incr:7.0f}   iROAS {incr * value /
extra_spend:4.2f}")
```

```
## estimate  incremental conversions    1040   iROAS 1.39
## 95% low   incremental conversions     214   iROAS 0.29
## 95% high  incremental conversions    1804   iROAS 2.41
```

The point estimate says the heavy-up paid back, with an iROAS around 1.4. The interval runs from about 0.3 to 2.4, so the test cannot rule out that the extra spend lost money. That is a typical geo-test outcome and still useful: it rules out very high marginal returns, it gives a

measured anchor that can be combined with a model estimate (Chapter 4), and it tells you how much bigger the next test would need to be to settle the question. What it does not support is a confident claim either way, and the readout should say so.

6.4 Analysis methods in practice

- **Difference-in-differences ratio**: the treated regions' post/pre ratio divided by the controls' post/pre ratio. Transparent, but sensitive to regions drifting apart.
- **Synthetic control**, as above, and its augmented variants (GeoLift uses an augmented synthetic control with a ridge correction for imperfect pre-period fit).
- **Time-based regression** (Google's GeoexperimentsResearch): regress the treated aggregate on the control aggregate in the pre-period and predict forward.
- **Trimmed match** (Google): a robust estimator for paired designs that trims the most extreme pairs, useful when a few regions behave very differently.

With a good design, these methods usually agree within their uncertainty. When they disagree sharply, check the pre-period fit and look for a region with an unusual event during the test.

> **Pitfall.**
> Reading the test while it is running and extending it "until it becomes significant". The peeking problem from Chapter 5 applies to geo tests too, and with few units it is worse. Fix the duration from the power analysis.

6.5 Things that break geo tests

- **Spillover.** People in control regions see the treated media (national TV, regional TV with overlapping coverage, people commuting into treated cities). Spillover biases the estimate toward zero. Commuting zones and buffer regions excluded from both groups reduce it.
- **Imprecise geo-targeting.** Digital platforms target by inferred location, which leaks across borders. Check delivery reports by region; a "go-dark" region still getting 15% of its normal impressions is a weaker treatment than designed.
- **One region dominating.** If one metropolitan region holds a third of the business, it will dominate both the treated total and the synthetic control. Consider splitting it, excluding it, or assigning it to control.
- **Other changes during the test.** A regional promotion, a sales team push, a local news story. Freeze other regional activity where possible and log anything that happens (Chapter 2).

- **Unrepresentative timing.** A test run in a seasonal low measures returns at low demand. Results transfer best to periods that look like the test period.

6.6 Checklist

Checklist

- Is the geo unit the smallest one at which media and outcomes can both be separated, with spillover considered?
- Was assignment randomized (ideally stratified), and is the method fixed in advance?
- Does a pre-test power analysis on historical data give an MDE worth testing for?
- Is the treatment type (go-dark, heavy-up, scale-back) matched to the decision (average versus marginal return)?
- Is delivery verified by region, so that the treatment actually happened as designed?
- Is the outcome taken from the system of record, with a cooldown window for lasting effects?
- Is the result reported as an iROAS range from placebo-based or model-based intervals, with the MDE?

6.7 Further reading

- *GeoLift* (Meta) documentation, walkthrough and the *Inverse GeoLift* post. The most complete open-source geo testing workflow, in R.
- *Commuting Zones* (Meta). Geographic units designed to reduce spillover.
- Google's *matched_markets* (with design and post-analysis notebooks), *trimmed_match* and *GeoexperimentsResearch* (time-based regression).
- *tidysynth* (R) and *pysyncon*, *mlsynth* (Python) for synthetic control.
- *openlift* (GitHub). A lighter-weight open-source geo lift tool.

Full links are in Appendix A.

Chapter 7 Quasi-experiments: DiD, CausalImpact and synthetic control

> **The decision this chapter supports.**
> What an intervention that was not designed as an experiment actually did: a launch in one market, a price change, a site redesign, an SEO change, an outage, a campaign limited to some regions for operational reasons.

7.1 Making the best of things that already happened

Most of the "did X work?" questions a measurement team receives are about things that already happened without a control group. You cannot randomize retroactively. What you can often do is find a comparison that plausibly shows what would have happened otherwise: other markets, other products, other customer groups, or the treated unit's own history adjusted for what happened to everyone else.

These designs sit in the middle of the identification ladder (Chapter 3). They are much better than a raw before-and-after comparison and much weaker than a randomized test, because their key assumption, that the comparison would have moved like the treated unit, can be supported but never proven.

7.2 Difference-in-differences

Difference-in-differences (DiD) compares the change in the treated group with the change in a comparison group over the same period. Anything that moved both groups equally (seasonality, macro conditions, a national campaign) cancels out. The key assumption is **parallel trends**: without the treatment, the two groups would have followed parallel paths.

Parallel trends cannot be tested for the post-period, but it can be checked in the pre-period with an **event study**: estimate the treated-versus-control gap for each period relative to the treatment date. Before treatment, the gaps should hover around zero. After treatment, they trace out the effect over time.

7.2.1 Worked example: an event study

Thirty regions are observed for 52 weeks. In week 30, ten of them get a new local sales and marketing program. The true effect builds up gradually, reaching 8% after four weeks. Outcomes are modeled in logs so that coefficients read as approximate percentage effects.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

rng = np.random.default_rng(12)
units, weeks, start = 30, 52, 30
treated = rng.choice(units, 10, replace=False)
rows = []
unit_fe = rng.normal(np.log(500), 0.5, units)
week_fe = 0.1 * np.sin(2 * np.pi * np.arange(weeks) / 52) + 0.002 * np.arange(weeks)
for u in range(units):
    for w in range(weeks):
        rel = w - start
        effect = min(0.02 * (rel + 1), 0.08) if (u in treated and rel >= 0) else 0.0
        rows.append({"unit": u, "week": w, "treated": int(u in treated), "rel": rel,
                     "log_y": unit_fe[u] + week_fe[w] + effect + rng.normal(0, 0.03)})
df = pd.DataFrame(rows)

# Event-time dummies for treated units, binned at the ends; week -1 is the reference
df["k"] = df["rel"].clip(-10, 15)
for k in range(-10, 16):
    if k != -1:
        df[f"e{k + 10}"] = ((df["k"] == k) & (df["treated"] == 1)).astype(int)
terms = " + ".join(c for c in df.columns if c.startswith("e") and c[1:].isdigit())
fit = smf.ols(f"log_y ~ C(unit) + C(week) + {terms}", df).fit(
    cov_type="cluster", cov_kwds={"groups": df["unit"]})

es = pd.DataFrame({"k": [int(c[1:]) - 10 for c in terms.split(" + ")],
                   "coef": [fit.params[c] for c in terms.split(" + ")],
                   "se": [fit.bse[c] for c in terms.split(" + ")]})
pre = es[es.k < -1]
print(f"Mean pre-period coefficient: {pre.coef.mean():+.3f} (max |coef|
{pre.coef.abs().max():.3f})")
```

```
## Mean pre-period coefficient: -0.010 (max |coef| 0.025)
```

```
print(f"Mean effect, weeks 4+ after start: {es[es.k >= 3].coef.mean():+.3f} (true
+0.080)")
```

```
## Mean effect, weeks 4+ after start: +0.074 (true +0.080)
```

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 2.9))
ax.errorbar(es.k, es.coef, yerr=1.96 * es.se, fmt="o", ms=4, capsize=2,
color="#1F4E79")
ax.axhline(0, color="#6B6B6B", lw=0.8); ax.axvline(-0.5, color="#C8102E", ls="--",
lw=1)
ax.set_xlabel("Weeks relative to start (endpoints binned)"); ax.set_ylabel("Effect on
log outcome")
plt.tight_layout()
plt.show()
```

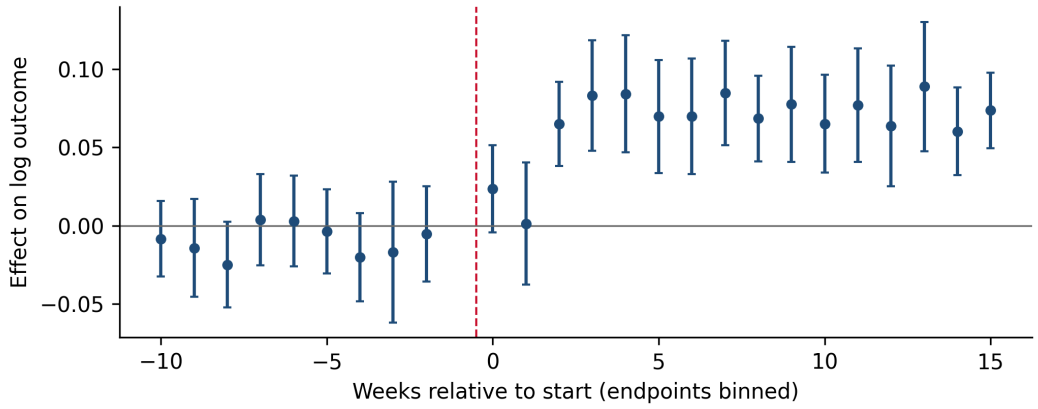


Figure 7.1: Event study: estimated treated-versus-control gap by week relative to the program start, with 95% intervals clustered by region. Flat pre-period coefficients support parallel trends.

The pre-period coefficients sit around zero, the effect ramps up after the start and settles near the true 8%. Two features of the code matter in practice: standard errors are **clustered by region**, because weekly observations within a region are not independent, and the reference period is the week just before treatment.

> **Pitfall.**
> Staggered rollouts analyzed with an ordinary two-way fixed effects regression. When different units start treatment at different times and effects change over time, the standard regression compares late-treated units with already-treated units and can produce badly biased estimates, even with the wrong sign. Use estimators designed for staggered adoption (Callaway and Sant'Anna, Sun and Abraham, or the imputation approach), available in packages such as `differences`, `diff-diff` and `DRDID`.

7.3 CausalImpact: a model-based counterfactual

When there is a single treated unit (one country, one product, the whole website) and several unaffected time series that move with it, a **structural time series model** can predict the

counterfactual. This is the approach behind Google's CausalImpact package. The model is fitted on the pre-period, with the control series as regressors and a time-varying level to absorb slow drift, then projected through the post-period. The gap between actual and predicted is the effect, and the model's forecast uncertainty gives its interval.

The assumptions are specific and worth stating in any readout:

1. **The controls were not affected by the intervention.** Other countries' sign-ups are good controls for a country-specific change; branded search volume is a bad control for a campaign, because the campaign moves it.
2. **The relationship between the treated series and the controls is stable** from the pre-period into the post-period.
3. **Nothing else happened to the treated unit alone** during the post-period.

7.3.1 Worked example: a site change in one market

A conversion-flow change goes live in one market on day 100 and truly lifts daily sign-ups by 10%. Three other markets, unaffected, serve as controls.

```
import warnings, statsmodels.api as sm
warnings.filterwarnings("ignore")

rng = np.random.default_rng(5)
T, T0 = 140, 100
t = np.arange(T)
common = 100 + 0.15 * t + 8 * np.sin(2 * np.pi * t / 7)
X = np.column_stack([common * a + rng.normal(0, 2.5, T) for a in (0.8, 1.1, 0.6)])
y = common + np.cumsum(rng.normal(0, 0.4, T)) + rng.normal(0, 2.5, T)
y[T0:] *= 1.10                                    # true effect: +10% from day 100

def impact(y, X, T0, T1=None, reps=2000):
    """Fit on [0, T0), predict [T0, T1); return predicted mean, simulated paths."""
    T1 = T1 or len(y)
    res = sm.tsa.UnobservedComponents(y[:T0], level="llevel",
exog=X[:T0]).fit(disp=False)
    pred = res.get_forecast(T1 - T0, exog=X[T0:T1]).predicted_mean
    sims = np.asarray(res.simulate(T1 - T0, repetitions=reps, anchor="end",
                                   exog=X[T0:T1])).reshape(T1 - T0, -1)
    return pred, sims

pred, sims = impact(y, X, T0)
cum = y[T0:].sum() - sims.sum(0)
true_cum = (y[T0:] - y[T0:] / 1.10).sum()
print(f"Cumulative effect: {y[T0:].sum() - pred.sum():.0f} sign-ups, 95% interval "
      f"[{np.quantile(cum, .025):.0f}, {np.quantile(cum, .975):.0f}]  (true
{true_cum:.0f})")
```

```
## Cumulative effect: 319 sign-ups, 95% interval [149, 489]  (true 473)
```

```
print(f"Relative effect: {y[T0:].sum() / pred.sum() - 1:+.1%};  "
      f"simulated paths with effect > 0: {(cum > 0).mean():.1%}")
```

```
## Relative effect: +6.5%;  simulated paths with effect > 0: 100.0%
```

```
band = np.quantile(sims, [0.025, 0.975], axis=1)
fig, ax = plt.subplots(figsize=(7, 2.9))
ax.plot(t, y, lw=1.2, color="#333", label="Actual")
ax.plot(t[T0:], pred, lw=2, ls="--", label="Counterfactual")
ax.fill_between(t[T0:], band[0], band[1], alpha=0.2, label="95% interval")
ax.axvline(T0, color="#C8102E", lw=1)
ax.set_xlabel("Day"); ax.set_ylabel("Sign-ups"); ax.legend(loc="upper left")
plt.tight_layout()
plt.show()
```

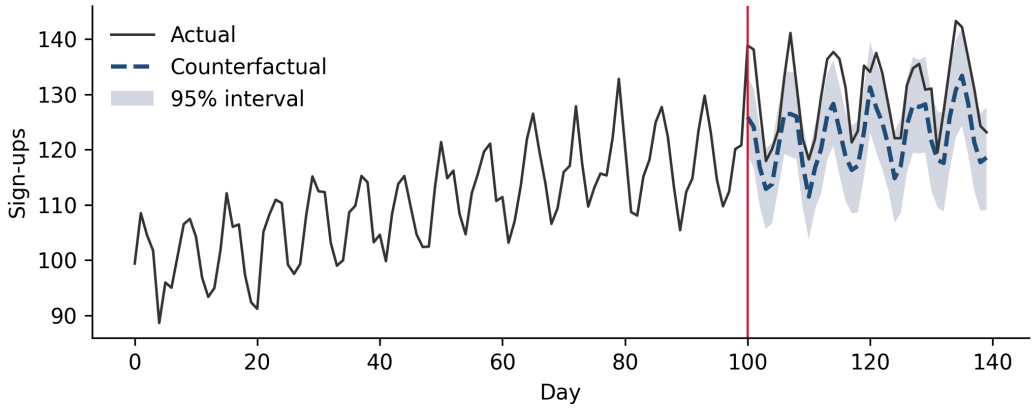


Figure 7.2: Actual sign-ups in the treated market versus the model's counterfactual and its 95% interval, fitted on the pre-period only.

The analysis detects the effect clearly, but the estimate (around 6 to 7%) is well below the true 10%, and the interval only just contains the truth. The treated market has its own slow drift that the controls do not explain. The model's interval accounts for that drift on average, but any single analysis can land well off target. This is why one quasi-experimental estimate should be treated as one noisy measurement, and why placebo checks matter.

7.3.2 Placebo in time

The most useful robustness check costs one line: pretend the intervention happened earlier, inside the pre-period, and run the same analysis on data where nothing happened. A sound setup should find nothing.

```
pred_p, sims_p = impact(y, X, T0=70, T1=100)
cum_p = y[70:100].sum() - sims_p.sum(0)
print(f"Placebo (fake start day 70): effect {y[70:100].sum() / pred_p.sum() - 1:+.1%},
95% interval "
      f"[{np.quantile(cum_p, .025):.0f}, {np.quantile(cum_p, .975):.0f}] sign-ups")
```

```
## Placebo (fake start day 70): effect +0.3%, 95% interval [-118, 146] sign-ups
```

If the placebo finds a "significant" effect, the model is not tracking the treated series well enough to trust the real result. Run several placebo dates; the share that come out significant is a direct estimate of the method's false positive rate on your data.

> **Pitfall.**
> Choosing control series by trying combinations until the post-period effect looks good. Pick controls on substantive grounds and on pre-period fit only, and fix them before looking at the post-period.

7.4 Synthetic control for a single unit

The synthetic control method from Chapter 6 also works for a single treated unit with a pool of untreated donors (other countries, other products). It is more transparent than a structural time series model, because the counterfactual is an explicit weighted average of named donors, and its inference comes from in-space placebos: run the method on each donor as if it had been treated and see whether the real effect stands out. With fewer than about ten donors the placebo distribution is too coarse for a meaningful p-value, and the analysis should be presented as descriptive.

7.5 When there is no control at all

Sometimes the only data is the treated series itself: an **interrupted time series**. You can model trend and seasonality in the pre-period and project forward, but any other change at the same time (a competitor move, a seasonal anomaly, a tracking change) is indistinguishable from the effect. Present the result as weak evidence, show the forecast interval, and look hard at the change log.

7.6 Threats and how to probe them

Threat	What it looks like	Probe
Concurrent shock	Another change hit the treated unit in the same window	Change log, placebo outcomes that should not move
Anticipation	Behavior changed before the official start (announcements, pre-launch PR)	Event-study leads, earlier start date
Contaminated controls	The intervention also affected the comparison units	Check spillover channels; drop suspect controls
Unstable relationship	Controls tracked well historically but diverged for unrelated reasons	Placebo-in-time tests; shorter pre-period
Selection into treatment	The treated unit was chosen because it was doing badly (or well)	Look for regression to the mean in the pre-period
Specification search	Many models tried, one reported	Pre-register controls and windows; report all variants tried

7.7 Checklist

Checklist

- Is there a plausible comparison that was not affected by the intervention?
- For DiD: are event-study pre-period coefficients flat, and are standard errors clustered at the unit level?
- For staggered rollouts: is a staggered-adoption estimator used instead of plain two-way fixed effects?
- For CausalImpact: are the controls unaffected, chosen on pre-period grounds only, and fixed before looking?
- Do placebo-in-time (and, where possible, placebo-in-space) tests come out null?
- Does the change log show anything else hitting the treated unit in the window?
- Does the readout name the design as quasi-experimental and list the threats that remain?

7.8 Further reading

- Google's *CausalImpact* (R) and *tfp-causalimpact* (Python).

- *CausalPy* (PyMC Labs). Bayesian synthetic control, DiD, interrupted time series and regression discontinuity in one package.
- *Causal Inference: The Mixtape*, chapters on DiD and synthetic control.
- `differences`, `diff-diff` and `DRDID` for modern DiD with staggered adoption.
- *Bootstrap random walks for causal inference on time series* (Carlos Agostini). An alternative way to build intervals.

Full links are in Appendix A.

Chapter 8 Reading platform lift studies

The decision this chapter supports.
How much to trust an ad platform's own incrementality study, what exactly it measured, and how to turn its result into numbers you can use for budgeting and for correcting the platform's everyday reporting.

8.1 What a lift study is

The large ad platforms offer incrementality studies on their own inventory: conversion lift studies that hold out a random share of the target audience from seeing your ads, geo-based variants that hold out regions, and brand lift studies that survey exposed and unexposed users. The details and names change often, so check your platform's current documentation, but the logic is stable.

User-level conversion lift is a real randomized experiment. The platform splits the eligible audience at random into a test group, which can be shown your ads, and a holdout, which is not (holdout users see other advertisers' ads instead). Conversions in the two groups are compared. Because assignment is random, the comparison is causal in the same sense as any A/B test.

That makes a platform lift study much stronger evidence than the platform's attribution reporting, which measures who converted after an ad, not who converted because of one (Chapter 1). It does not make it a complete answer. The randomization is sound; the questions are about **what outcome was measured, on whom, under which conditions**.

8.2 What exactly was measured

Read every lift study against these questions before using its result.

Question	Why it matters	What to look for
What is the outcome?	The platform can only count conversions it observes	Pixel / SDK events versus conversions uploaded from your backend
Who was randomized?	Results apply to the audience in the test, not to everyone	Targeting, exclusions, campaigns included
How is the effect scaled?	Intent-to-treat versus effect on users who actually saw ads	"Reach" or "exposed" share; whether results are per eligible or per reached user
How big was the holdout?	Holdout size drives precision	Holdout share, number of conversions in the holdout
Which interval is reported?	Not every platform reports 95% intervals	Confidence or credibility level; Bayesian probabilities of lift
Was delivery normal?	Campaign changes during the test change what was measured	Budget, bidding, creative changes in the test window
What window?	Long-lag outcomes may fall outside the study	Study duration, post-period, conversion window

The single most important item is the first. If the study counts only pixel-tracked conversions, it inherits all the signal loss of tracking (Chapter 2): users who convert on another device or without consent are missing from both groups. That usually understates the absolute incremental count while leaving the relative lift roughly intact. Wherever the platform allows it, feed system-of-record conversions into the study (server-side events or offline conversion uploads), so the study measures the outcome you actually care about.

8.3 Worked example: from a study readout to usable numbers

A study ran for four weeks with a 10% holdout. The platform reports these totals for the period:

```python
import numpy as np
import pandas as pd

N_t, N_c = 2_000_000, 200_000      # users in test and holdout groups (10% holdout)
conv_t, conv_c = 6_300, 560        # conversions observed in each group
spend = 150_000                    # media spend during the study (EUR)
attributed = 2_100                 # conversions the platform attributed to the
campaign

p_t, p_c = conv_t / N_t, conv_c / N_c
incr = (p_t - p_c) * N_t                                        # incremental
conversions
se = np.sqrt(p_t * (1 - p_t) / N_t + p_c * (1 - p_c) / N_c) * N_t
lo, hi = incr - 1.96 * se, incr + 1.96 * se

out = pd.DataFrame({
    "estimate": [p_t / p_c - 1, incr, spend / incr, incr / attributed],
    "95% low":  [lo / (p_c * N_t), lo, spend / hi, lo / attributed],
    "95% high": [hi / (p_c * N_t), hi, spend / lo, hi / attributed],
}, index=["Relative lift", "Incremental conversions", "Incremental CPA (EUR)",
"Incrementality factor"])
print(out.round(2).to_string())
```

```
##                          estimate  95% low  95% high
## Relative lift                0.12     0.04      0.21
## Incremental conversions    700.00   211.48   1188.52
## Incremental CPA (EUR)      214.29   126.21    709.29
## Incrementality factor        0.33     0.10      0.57
```

```python
print(f"\nPlatform-attributed CPA: {spend / attributed:.0f} EUR")
```

```
##
## Platform-attributed CPA: 71 EUR
```

Three things stand out.

The relative lift is modest and the interval is wide. A 12.5% lift with an interval from roughly 4% to 21% is typical. The precision is limited by the holdout: only 560 conversions in the control group.

The incremental CPA is several times the attributed CPA. The platform's reporting says each conversion costs about 71 EUR. The study says each *incremental* conversion costs around 214 EUR, and plausibly anywhere from about 125 to over 700. Note that the interval on a cost ratio is skewed: it is computed by inverting the interval on incremental conversions, not by adding and subtracting a margin.

The incrementality factor connects the two. About a third of attributed conversions are incremental here, with a wide range. This factor is how a lift study gets used day to day (below).

8.3.1 Intent-to-treat versus reached users

The calculation above compares everyone randomized to the test group with everyone in the holdout, including test-group users who never actually saw an ad. That is the **intent-to-treat** effect, and it is what matters for budgeting, because it answers "what did running this campaign do?".

Some studies instead report the effect per **reached** user, comparing exposed users with holdout users who would have been exposed. If only 45% of the test group was reached, and unreached users were unaffected, the effect per reached user is the intent-to-treat effect divided by 0.45. Both numbers are legitimate. Mixing them up overstates or understates the effect by a factor of two or more, so check which one the report gives before comparing studies.

8.4 Sizing the holdout

Holdout size trades precision against cost: holdout users are not advertised to, so a bigger holdout means more conversions forgone during the study. The table shows the minimum detectable relative lift (80% power, 5% two-sided) for the audience and baseline rate above, by holdout share.

```
from scipy import stats
N, p = 2_200_000, 0.0028
z = stats.norm.ppf(0.975) + stats.norm.ppf(0.8)
rows = []
for h in [0.05, 0.10, 0.20, 0.30, 0.50]:
    n_c, n_t = N * h, N * (1 - h)
    mde = z * np.sqrt(p * (1 - p) * (1 / n_t + 1 / n_c)) / p
    rows.append({"holdout share": f"{h:.0%}", "holdout conversions": round(n_c * p),
                 "MDE (relative lift)": f"{mde:.1%}"})
print(pd.DataFrame(rows).to_string(index=False))
```

```
## holdout share  holdout conversions MDE (relative lift)
##            5%                  308               16.4%
##           10%                  616               11.9%
##           20%                 1232                8.9%
##           30%                 1848                7.8%
##           50%                 3080                7.1%
```

Going from a 10% to a 20% holdout narrows the MDE noticeably; beyond about 30%, the gains are small and the cost keeps growing. If the expected lift is around 10%, a 10% holdout on this audience is marginal: there is a real chance the study comes back "inconclusive". Size the study before launching it, the same way as any test (Chapter 5).

8.5 Using a lift study

Incrementality factors. The most practical use is to calibrate everyday platform reporting. Divide incremental conversions by attributed conversions in the same study and apply that factor to the platform's attributed numbers going forward. Platform CPA of 71 EUR and a factor of 0.33 imply an incremental CPA of about 214 EUR, which is the number to compare with other channels and with the value of a conversion. Factors are specific to a platform, campaign type, market and attribution setting, and they drift as targeting and tracking change. Refresh them at least a few times a year and whenever the setup changes materially.

MMM calibration. A lift study is also an input for a marketing mix model, as a prior or a calibration target for that channel (Chapter 11), with the caveats above about outcome definition and scaling.

Cross-checks. Because the platform designs, runs and reports the study, an occasional independent check with a geo test on backend outcomes (Chapter 6) is good practice for the largest channels. The point is not that platforms report dishonestly; it is that their study measures their tracked outcome under their conditions, and you want to know how well that transfers to your business outcome.

> **Pitfall.**
> Extending a lift result to another market, campaign type or spend level without saying so. A study on prospecting campaigns in one country, at one budget, says little about retargeting in another country at double the budget. When a result is used outside the conditions it was measured in, write that down as an assumption.

8.6 Brand lift studies

Brand lift studies survey users in exposed and holdout groups about ad recall, awareness, consideration or intent. They measure stated attitudes, not behavior, and typically have small response samples, so their intervals are wide. They are useful for comparing creative or formats on upper-funnel goals, and as a sanity check that a brand campaign registers at all. They are not evidence of incremental conversions or revenue; for that, use conversion lift or geo tests on business outcomes.

8.7 Checklist

Checklist

- Is the outcome the platform's tracked conversion or your system-of-record conversion?
- Is the result intent-to-treat or per reached user, and does any comparison mix the two?
- How many conversions were in the holdout, and what MDE did the design have?
- Which confidence or credibility level is reported?
- Did budget, bidding or creative change during the study?
- Is the incrementality factor computed with its interval and stored with the conditions it was measured under (platform, campaign type, market, attribution setting, date)?
- If the result is applied to another market, campaign or spend level, is that stated as an assumption?

8.8 Further reading

- *Google's Measurement Playbook*. The vendor view of how lift studies, MMM and attribution fit together.
- *Calibration with experiments* (Juan Orduz). How lift results enter a Bayesian MMM, previewing Part III.

Full links are in Appendix A.

Chapter 9 Marketing mix modeling end to end

The decision this chapter supports.
How much each channel contributes to the business outcome and at what return, as the basis for setting the total budget and splitting it across channels. This chapter builds a classical MMM from scratch so that every moving part is visible, and shows where it fails.

9.1 What an MMM is, and what it is not

A marketing mix model is a time-series regression of a business outcome (revenue, sign-ups, accounts) on media activity and everything else that drives the outcome: trend, seasonality, prices, promotions, macro conditions. Its output is a decomposition of the outcome into a base and a contribution per channel, from which you get return on spend, response curves and budget scenarios.

Its strengths follow from working with aggregates. It needs no user-level tracking, so it is unaffected by signal loss (Chapter 2). It covers every channel, online and offline, in one consistent framework. And it naturally produces the marginal returns that allocation decisions need.

Its weaknesses follow from being observational. It learns only from variation in spend that already happened, so a channel whose spend never moved cannot be measured, and a channel whose spend moves with demand will be credited with that demand (Chapter 3). An MMM is a causal claim made with regression, and it is only as good as its assumptions. That is why Chapter 11 is about validating and calibrating it against experiments.

9.2 The building blocks

Most MMMs, from a spreadsheet to Google's Meridian, share the same structure:

\[y_t = \text{base}_t + \sum_{c} \beta_c \, f_c\!\left(\text{adstock}_c(x_{c,t})\right) + \varepsilon_t, \]

where \(x_{c,t}\) is spend (or impressions) in channel \(c\) in week \(t\), and the base contains intercept, trend, seasonality and controls.

Adstock captures carryover: an ad seen this week still affects behavior next week. The simplest form is geometric decay, where a share \(\delta\) of each week's effect carries into the following week, then \(\delta^2\) into the week after, and so on. TV and brand campaigns typically decay slowly; search and promotional activity decay fast. Delayed

adstock forms (for example Weibull-shaped) allow the effect to peak a week or two after the spend.

Saturation captures diminishing returns: the tenth million spent in a channel buys less than the first. Common shapes are the Hill function, the negative exponential and the logistic form used here, \(f(x) = (1 - e^{-\lambda x}) / (1 + e^{-\lambda x})\), where larger \(\lambda\) means faster saturation.

```
import sys; sys.path.insert(0, ".")
import numpy as np
import matplotlib.pyplot as plt
from pms_sim import geometric_adstock, logistic_saturation

pulse = np.r_[1.0, np.zeros(9)]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(8, 2.5))
for d in [0.1, 0.3, 0.6]:
    a1.plot(geometric_adstock(pulse, d, l_max=10), marker="o", ms=3, label=f"decay
{d}")
a1.set_xlabel("Weeks after spend"); a1.set_title("Adstock"); a1.legend()
x = np.linspace(0, 2, 100)
for lam in [0.5, 1.5, 3.0]:
    a2.plot(x, logistic_saturation(x, lam), label=f"lambda {lam}")
a2.set_xlabel("Spend (scaled)"); a2.set_title("Saturation"); a2.legend()
plt.tight_layout()
plt.show()
```

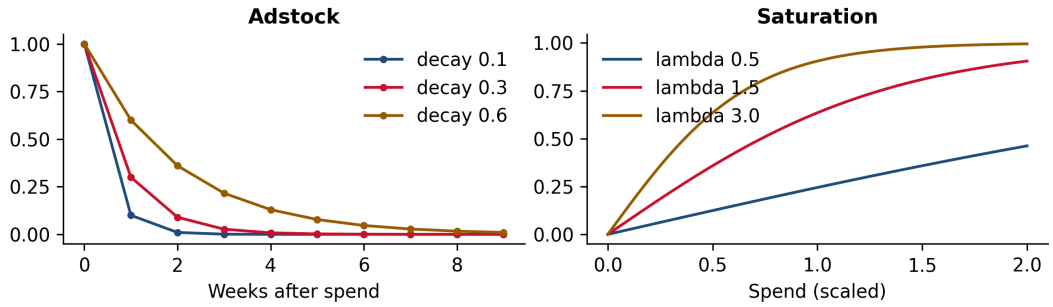


Figure 9.1: Left: geometric adstock spreads a single week of spend over the following weeks. Right: logistic saturation for different values of lambda (spend scaled to the channel maximum).

Order matters. Applying adstock before saturation (the usual choice) means carryover from past weeks and spend this week saturate together, which suits channels with continuous presence. Saturation before adstock suits bursty channels where each week's exposure saturates on its own.

The base is everything the media does not explain. Getting it right matters more than any media specification, because whatever the base misses, the media terms will absorb. Trend,

yearly seasonality (Fourier terms or holiday dummies), promotions, price changes, product launches and known shocks all belong here.

9.3 The data for this part of the book

All of Part III uses one simulated dataset so that results can be compared across chapters and against the truth. It has three years of weekly revenue and four channels, built to reproduce the situations real MMMs struggle with:

- **TV** runs in flights: six weeks on, twelve weeks off. Plenty of variation, slow decay.
- **Social** is planned into the seasonal peak, so its spend is correlated with seasonality.
- **Search** spend follows demand, including demand shocks the model cannot observe.
- **Display** is always on at an almost constant level.

The full simulation is in `pms_sim.py`:

```python
"""Simulated weekly marketing data with a known ground truth (Part III)."""
import numpy as np
import pandas as pd

CHANNELS = ["tv", "social", "search", "display"]


def geometric_adstock(x, decay, l_max=8):
    """Carry over a share `decay` of each week's effect into the following weeks."""
    w = decay ** np.arange(l_max)
    return np.convolve(x, w)[: len(x)]


def logistic_saturation(x, lam):
    """Concave response: ~linear for small x, flattening out as x grows."""
    return (1 - np.exp(-lam * x)) / (1 + np.exp(-lam * x))


TRUTH = {  # per channel: adstock decay, saturation lambda (on spend / channel max),
effect size (EUR)
    "tv":      {"decay": 0.60, "lam": 1.5, "beta": 140_000},
    "social":  {"decay": 0.30, "lam": 2.0, "beta": 90_000},
    "search":  {"decay": 0.10, "lam": 2.5, "beta": 85_000},
    "display": {"decay": 0.30, "lam": 1.0, "beta": 12_000},
}


def simulate(seed=2026, n_weeks=156):
    rng = np.random.default_rng(seed)
    t = np.arange(n_weeks)
    season = np.sin(2 * np.pi * t / 52) + 0.4 * np.cos(4 * np.pi * t / 52)
    shock = np.zeros(n_weeks)                       # unobserved demand shocks (AR(1))
    for i in range(1, n_weeks):
        shock[i] = 0.7 * shock[i - 1] + rng.normal(0, 0.35)
    promo = (rng.random(n_weeks) < 0.08).astype(float)

    spend = pd.DataFrame({
        # TV runs in flights: on for some weeks, off for others
        "tv": np.where((t // 6) % 3 == 0, rng.uniform(40_000, 80_000, n_weeks), 0.0),
        # Social is planned into the seasonal peak
        "social": np.clip(30_000 + 9_000 * season + rng.normal(0, 4_000, n_weeks),
5_000, None),
        # Search spend follows demand, including the unobserved shocks
        "search": np.clip(20_000 + 4_000 * season + 6_000 * shock + rng.normal(0,
1_500, n_weeks), 3_000, None),
        # Display is always on at an almost constant level
        "display": 10_000 + rng.normal(0, 400, n_weeks),
    })

    contrib = pd.DataFrame(index=t)
    for c in CHANNELS:
```

```
        p = TRUTH[c]
        x = geometric_adstock(spend[c].to_numpy(), p["decay"]) / spend[c].max()
        contrib[c] = p["beta"] * logistic_saturation(x, p["lam"])

    base = 300_000 + 400 * t + 45_000 * season + 30_000 * shock + 25_000 * promo
    revenue = base + contrib.sum(1) + rng.normal(0, 9_000, n_weeks)

    # Category-level search interest: a noisy, observable proxy for demand (not driven
by our media)
    category_demand = 100 + 20 * season + 20 * shock + rng.normal(0, 4, n_weeks)

    df = pd.DataFrame({"week": pd.date_range("2023-01-02", periods=n_weeks, freq="W-
MON"),
                       "revenue": revenue, "promo": promo, "category_demand":
category_demand, **spend})
    roi = {c: contrib[c].sum() / spend[c].sum() for c in CHANNELS}
    return df, {"contrib": contrib, "roi": roi, "base": base}
```

```
from pms_sim import simulate, CHANNELS, TRUTH
import pandas as pd

df, truth = simulate()
summary = df[CHANNELS].agg(["mean", "std"]).T
summary["cv"] = summary["std"] / summary["mean"]
summary["true ROI"] = pd.Series(truth["roi"])
print(summary.round(2).to_string())
```

```
##              mean       std    cv  true ROI
## tv       21345.83  30231.52  1.42      2.43
## social   30086.51   8504.20  0.28      2.04
## search   20389.93   4830.19  0.24      2.89
## display  10017.12    411.24  0.04      0.68
```

The coefficient of variation (cv) is worth a look before any modeling. Display's spend varies by about 4% around its mean; there is almost nothing to learn its effect from.

9.4 Fitting a classical MMM

The model is linear in the effect sizes \(\beta\) once the adstock and saturation parameters are fixed. That suggests a simple two-level fit: an outer search over the eight nonlinear parameters (a decay and a \(\lambda\) per channel), and for each candidate, an inner least-squares fit of the effect sizes and base terms, with media effects constrained to be non-negative. This is the same idea, minus the ridge penalty and multi-objective search, that Meta's Robyn uses.

```
from scipy.optimize import lsq_linear, differential_evolution

n = len(df); t = np.arange(n)
base_terms = np.column_stack([np.ones(n), t / n,
                              np.sin(2*np.pi*t/52), np.cos(2*np.pi*t/52),
                              np.sin(4*np.pi*t/52), np.cos(4*np.pi*t/52),
df["promo"]])
X = df[CHANNELS].to_numpy(); x_max = X.max(0); y = df["revenue"].to_numpy()

def media_terms(theta):
    decay, lam = theta[:4], theta[4:]
    return np.column_stack([logistic_saturation(geometric_adstock(X[:, i], decay[i]) /
x_max[i], lam[i])
                            for i in range(4)])

def inner_fit(theta):
    M = np.column_stack([media_terms(theta), base_terms])
    lower = np.r_[np.zeros(4), np.full(base_terms.shape[1], -np.inf)]   # media
effects >= 0
    return lsq_linear(M, y, bounds=(lower, np.inf)), M

bounds = [(0, 0.9)] * 4 + [(0.2, 6)] * 4
opt = differential_evolution(lambda th: (inner_fit(th)[0].fun ** 2).sum(), bounds,
                             seed=1, maxiter=80, tol=1e-8)
fit, M = inner_fit(opt.x)
contrib_hat = M[:, :4] * fit.x[:4]
fitted = M @ fit.x

r2 = 1 - ((y - fitted) ** 2).sum() / ((y - y.mean()) ** 2).sum()
mape = np.mean(np.abs(y / fitted - 1))
print(f"In-sample R^2 = {r2:.3f}, MAPE = {mape:.1%}")
```

```
## In-sample R^2 = 0.979, MAPE = 1.8%
```

```
res = pd.DataFrame({"true decay": [TRUTH[c]["decay"] for c in CHANNELS], "est. decay":
opt.x[:4],
                    "true ROI": [truth["roi"][c] for c in CHANNELS],
                    "est. ROI": contrib_hat.sum(0) / X.sum(0)}, index=CHANNELS)
print(res.round(2).to_string())
```

```
##          true decay  est. decay  true ROI  est. ROI
## tv              0.6        0.55      2.43      2.28
## social          0.3        0.10      2.04      1.64
## search          0.1        0.14      2.89      6.99
## display         0.3        0.00      0.68     23.18
```

```
fig, ax = plt.subplots(figsize=(7, 2.7))
xx = np.arange(4)
ax.bar(xx - 0.18, res["true ROI"], 0.36, label="True ROI")
ax.bar(xx + 0.18, res["est. ROI"].clip(upper=8), 0.36, label="Estimated ROI (capped at
8)")
ax.set_xticks(xx, CHANNELS); ax.set_ylabel("Revenue per EUR"); ax.legend()
plt.tight_layout()
plt.show()
```

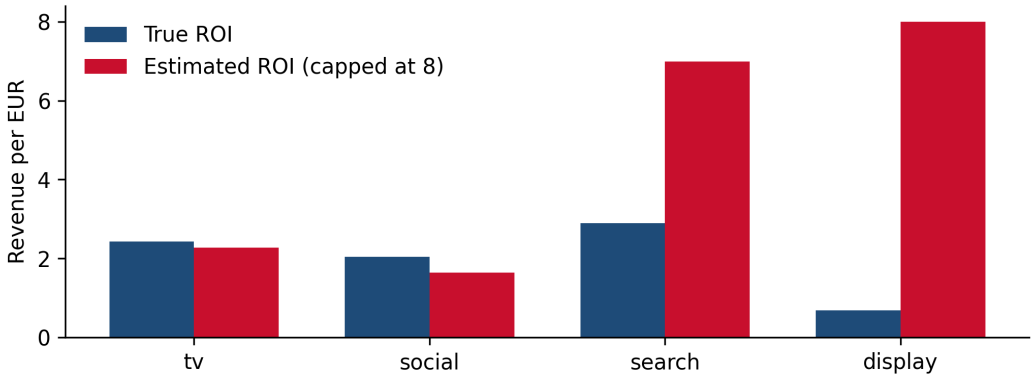


Figure 9.2: Classical MMM: estimated versus true ROI by channel. Excellent fit, wrong answers for two of four channels.

9.5 Reading the result

The model fits the data almost perfectly, and two of its four ROIs are badly wrong. Each error has a specific cause, and each cause is common in real data.

TV is recovered well. Its spend switches on and off independently of everything else, which is close to a natural experiment. Flighted channels are the easiest to measure.

Social is somewhat underestimated. Its spend moves with seasonality, so the seasonal terms and the social coefficient compete to explain the same pattern. The data cannot fully separate them.

Search is overstated by more than double. Search spend rises when demand rises, including demand shocks that are not in the model. The search coefficient absorbs those shocks. This is confounding in exactly the sense of Chapter 3, and no amount of fit quality will reveal it.

Display is absurd. With almost no variation in spend, display is nearly a constant, so it competes with the intercept. The optimizer happily assigns it a large chunk of base revenue, and the ROI explodes. The model has no information about display at all; it just does not say so.

The general lesson is the most important one in this part of the book: **goodness of fit is not evidence that the decomposition is right.** An MMM can have an R^2 of 0.98 and a MAPE of 2% while attributing base demand to media. Validation has to come from somewhere other than in-sample fit: from priors that encode what is plausible (Chapter 10), and above all from experiments (Chapter 11).

> **Pitfall.**
> Presenting MMM fit statistics as validation. R^2 and MAPE tell you the model reproduces the outcome series; they do not tell you the split between base and media, or between channels, is right. Several very different decompositions usually fit equally well.

9.6 Modeling decisions that matter

Decision	Options	Guidance
Outcome	Revenue, sign-ups, accounts, first transactions	The outcome closest to value that still has enough weekly volume; from the system of record
Granularity	Weekly or daily; national or by region	Weekly is standard. Regional (geo-level) data multiplies the observations and is usually worth the effort
Media variable	Spend, impressions, GRPs	Impressions or GRPs when media prices change over time; spend for budgeting outputs
Number of channels	Every line item, or grouped	Roughly one channel per 15 to 20 weeks of data at most; group small channels
Base	Trend, seasonality, holidays, promotions, price, macro	Include every major non-media driver; missing ones leak into media
History	2 to 3 years	Long enough to see seasons repeat; short enough that the business is comparable
Long-term effects	Usually not captured	Standard MMMs measure short-term response (weeks); brand-building effects over years need other methods

Geo-level data deserves emphasis. A national weekly model has around 150 observations. The same three years by 20 regions have 3,000, and regional variation in spend (from regional TV, from different market sizes, from tests) is often much less confounded than national variation over time. Hierarchical models that share information across regions (Meridian supports this directly) are the main route to more precise MMMs.

9.7 The tool landscape

- **Robyn** (Meta, R): ridge regression with evolutionary search over adstock and saturation hyperparameters, producing a set of Pareto-optimal models that trade off fit against how far the decomposition is from current spend shares. Strong on automation; requires judgment in choosing among models.
- **Meridian** (Google, Python): a Bayesian MMM with hierarchical geo-level modeling, reach and frequency inputs, and calibration with experiments through ROI priors. It replaces Google's earlier LightweightMMM.
- **PyMC-Marketing** (PyMC Labs, Python): a Bayesian MMM with flexible adstock and saturation choices, time-varying effects, lift-test calibration and a budget optimizer. Chapter 10 builds the same kind of model from scratch.
- **Orbit** (Uber), **Prophet-based** approaches, **Cassandra** and commercial tools (for example Recast) cover other points on the trade-off between flexibility, automation and transparency.

Choose a tool for its handling of priors, calibration and uncertainty, not for its fit statistics.

9.8 Checklist

Checklist

- Is the outcome from the system of record, and are all major non-media drivers in the base?
- Does every modeled channel have meaningful spend variation (check the coefficient of variation), and are channels without it grouped or handled with priors?
- Is there a channel whose spend responds to demand (search, retargeting)? Treat its estimate as an upper bound until tested.
- Are adstock and saturation choices justified per channel type?
- Is the ratio of channels to observations reasonable, or is geo-level data available?
- Is anyone treating R^2 or MAPE as validation?

9.9 Further reading

- *Bayesian MMM*, *MMM checklist*, *Adstock rates* and *Seasonality* (Recast). Clear, critical explanations of the main modeling choices.
- *Google Meridian* documentation .
- *siMMMulator* (Meta). Data to practice on, and a simulator for testing whether a model recovers known truth.
- *HelloFresh Bayesian MMM*, and *Uber's MMM with Orbit* as published case studies.

Full links are in Appendix A.

Chapter 10 Bayesian MMM in practice

> **The decision this chapter supports.**
> The same allocation and budget-level questions as Chapter 9, answered with a model that states its assumptions as priors, reports honest uncertainty, and says plainly when the data cannot tell you something.

10.1 What priors buy you

The classical model in Chapter 9 had two failure modes. For display it produced an absurd answer with no warning. For search it produced a confident wrong answer. A Bayesian model fixes the first and makes the second easier to diagnose, but it does not fix the second on its own.

Priors help in three concrete ways:

1. **They rule out the impossible.** Media effects are non-negative, adstock decays are between 0 and 1, and no channel plausibly drives half of revenue on a tenth of the budget. Encoding this stops the model from assigning base revenue to a constant like display.
2. **They make "the data says nothing" visible.** When a channel's posterior looks just like its prior, the data contributed no information. That is a finding, and it should be reported as one.
3. **They are the entry point for experiments.** A lift test result can enter the model as a prior or as an extra likelihood term (Chapter 11). This is the main route to fixing confounded channels like search.

10.2 The model

The model below is a plain PyMC implementation of the structure from Chapter 9: geometric adstock with an eight-week maximum lag, logistic saturation, and a base with intercept, trend, two pairs of yearly Fourier terms and a promotion dummy. Revenue is scaled by its mean and spend by each channel's maximum, so that priors can be stated on comparable scales. Writing it out by hand, rather than calling a library, makes every assumption explicit; the library equivalents come at the end of the chapter.

```
"""A compact Bayesian MMM in plain PyMC: geometric adstock + logistic saturation."""
import numpy as np
import pymc as pm
import pytensor.tensor as pt

L_MAX = 8


def lagged(X, l_max=L_MAX):
    """Array of shape (l_max, n_weeks, n_channels): X shifted by 0..l_max-1 weeks."""
    out = np.zeros((l_max,) + X.shape)
    for l in range(l_max):
        out[l, l:] = X[: len(X) - l]
    return out


def build_mmm(df, channels, lift=None, controls=()):
    """df: weekly data with revenue, promo and spend columns.
    lift: optional dict(channel, start, end, roi, se) from an experiment, used to
calibrate.
    controls: optional extra control columns (standardized inside the model)."""
    n = len(df)
    t = np.arange(n)
    y_scale = df["revenue"].mean()
    y = df["revenue"].to_numpy() / y_scale
    X = df[channels].to_numpy()
    x_scale = X.max(0)
    Xlag = lagged(X / x_scale)
    fourier = np.column_stack([f(2 * np.pi * k * t / 52) for k in (1, 2) for f in
(np.sin, np.cos)])
    spend_share = X.sum(0) / X.sum()

    coords = {"channel": channels, "week": t, "fourier": ["s1", "c1", "s2", "c2"]}
    with pm.Model(coords=coords) as model:
        # Carryover: TV is expected to last longer than digital channels
        decay = pm.Beta("decay", alpha=np.where(np.array(channels) == "tv", 3.0, 2.0),
                        beta=np.where(np.array(channels) == "tv", 2.0, 3.0),
dims="channel")
        lam = pm.Gamma("lam", alpha=4, beta=2, dims="channel")
        # Effect sizes, parameterized as each channel's average weekly contribution
        # (share of mean revenue); prior scale proportional to the channel's share of
spend
        beta = pm.HalfNormal("beta", sigma=0.12 * spend_share / spend_share.max(),
dims="channel")

        weights = decay[None, :] ** np.arange(L_MAX)[:, None]           # (lag,
channel)
        adstocked = (Xlag * weights[:, None, :]).sum(axis=0)             # (week,
channel)
        sat = (1 - pt.exp(-lam * adstocked)) / (1 + pt.exp(-lam * adstocked))
        sat = sat / sat.mean(axis=0)            # rescale so that beta = average
```

```
contribution
        contrib = pm.Deterministic("contrib", sat * beta, dims=("week", "channel"))

        # Prior on the overall level (intercept plus average media contribution), not
on the
        # intercept alone: a near-constant channel can then trade off against the
intercept
        # without moving the level, which removes a ridge the sampler otherwise
struggles with
        level = pm.Normal("level", 0.9, 0.3)
        intercept = pm.Deterministic("intercept", level - beta.sum())
        trend = pm.Normal("trend", 0, 0.3)
        gamma = pm.Normal("gamma", 0, 0.2, dims="fourier")
        promo = pm.Normal("promo", 0, 0.2)
        mu = (intercept + trend * t / n + pt.dot(fourier, gamma)
              + promo * df["promo"].to_numpy() + contrib.sum(axis=1))
        if controls:
            Z = df[list(controls)].to_numpy()
            Z = (Z - Z.mean(0)) / Z.std(0)
            delta = pm.Normal("delta", 0, 0.2, shape=len(controls))
            mu = mu + pt.dot(Z, delta)
        sigma = pm.HalfNormal("sigma", 0.1)
        pm.Normal("y", mu, sigma, observed=y, dims="week")

        # ROI per channel over the whole period, in EUR revenue per EUR spend
        pm.Deterministic("roi", contrib.sum(axis=0) * y_scale / X.sum(0),
dims="channel")

        if lift is not None:   # experiment-implied ROI over the test window, as an
extra likelihood term
            i = channels.index(lift["channel"])
            w = slice(lift["start"], lift["end"])
            model_roi = contrib[w, i].sum() * y_scale / X[w, i].sum()
            pm.Normal("lift_obs", mu=model_roi, sigma=lift["se"],
observed=lift["roi"])
    return model, {"y_scale": y_scale, "x_scale": x_scale}
```

The priors deserve a word each:

- **Decay**: Beta(3, 2) for TV (mean 0.6) and Beta(2, 3) for the digital channels (mean 0.4). This encodes the common finding that TV carries over longer, without forcing it.
- **Saturation**: Gamma(4, 2) on λ, mean 2, which puts the curve's bend somewhere within the observed spend range rather than far outside it.
- **Effect sizes**: each channel's β is its *average weekly contribution* as a share of mean revenue (the saturation curve is rescaled to average 1 over the data). Its prior is half-normal with scale proportional to the channel's share of spend, a mild belief that bigger budgets go to channels that do more. This mirrors PyMC-Marketing's default. It is a choice, not a law; Chapter 11 shows how to check what it does.

- **Level**: the prior is placed on the overall level (intercept plus average media contribution) rather than on the intercept, for a reason explained under diagnostics below.

10.3 Prior predictive check

Before fitting, simulate from the priors alone and look at what they imply for quantities you can judge: the ROI of each channel and the share of revenue attributed to media.

```
import sys; sys.path.insert(0, ".")
import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, pymc as pm
from pms_sim import simulate, CHANNELS
from pms_mmm import build_mmm

df, truth = simulate()
model, scales = build_mmm(df, CHANNELS)
with model:
    prior = pm.sample_prior_predictive(1000, random_seed=7)
```

```
## Sampling: [beta, decay, gamma, lam, level, promo, sigma, trend, y]
```

```
roi_prior = prior.prior["roi"].stack(s=("chain", "draw")).values          # (channel,
draws)
media_share = (prior.prior["contrib"].sum(("week", "channel")).values.ravel()
               / prior.prior_predictive["y"].sum("week").values.ravel())
q = lambda a: np.quantile(a, [0.05, 0.5, 0.95], axis=-1)
tbl = pd.DataFrame(q(roi_prior).T, index=CHANNELS, columns=["5%", "median", "95%"])
print("Prior ROI by channel:\n" + tbl.round(2).to_string())
```

```
## Prior ROI by channel:
##            5%  median   95%
## tv       0.14    1.45  4.14
## social   0.15    1.38  4.04
## search   0.11    1.34  3.80
## display  0.13    1.45  4.04
```

```
lo, med, hi = q(media_share)
print(f"\nPrior share of revenue from media: median {med:.0%}, 90% range [{lo:.0%},
{hi:.0%}]")
```

```
##
## Prior share of revenue from media: median 29%, 90% range [11%, 89%]
```

Because each channel's effect prior scales with its share of spend, the implied ROI prior is similar for every channel, centered a little above 1 with a long right tail: a neutral starting

point that does not favor any channel. Total media is expected to drive somewhere between a tenth and most of revenue, with a small chance of more than all of it.

This is the second version of the prior, and the first one is worth describing because it shows what the check is for. Initially the effect-size prior had two and a half times the scale used here. It looked reasonable written down, but its prior predictive check implied a median media share of about 72% of revenue and a 90% range reaching over 200%: a belief, before seeing any data, that media plausibly drives twice the business. Tightening the scale brought the implied quantities back into a defensible range. Nothing about the data was used to make that change; it was a correction of the prior to match what we actually believe. That is the purpose of the prior predictive check, and it is why the check comes before fitting.

10.4 Fitting and diagnostics

Fitting is a single call. Three years of weekly data with four channels samples in a few minutes on a laptop.

```
with model:
    idata = pm.sample(1000, tune=1500, chains=4, target_accept=0.95, random_seed=3)
    pm.sample_posterior_predictive(idata, extend_inferencedata=True, random_seed=3)
```

Before reading any result, check that the sampler worked. Three checks cover most problems: no (or very few) divergent transitions, R-hat close to 1 for every parameter (chains agree), and an effective sample size in the hundreds or more (enough independent draws).

```
post = idata.posterior
n_div = int(idata.sample_stats["diverging"].sum())
rhat = az.rhat(post, var_names=["decay", "lam", "beta", "level", "sigma"])
ess = az.ess(post, var_names=["decay", "lam", "beta", "level", "sigma"])
max_rhat = max(float(rhat[v].max()) for v in rhat.data_vars)
min_ess = min(float(ess[v].min()) for v in ess.data_vars)
print(f"Divergences: {n_div};  max R-hat: {max_rhat:.3f};  min ESS: {min_ess:.0f}")
```

```
## Divergences: 0;  max R-hat: 1.004;  min ESS: 2702
```

Getting to a clean run took one change worth describing, because it is typical. The first version of this model put the prior on the intercept and produced a few dozen divergent transitions. Comparing divergent draws with the rest showed where they came from: they clustered where display's effect was large and the intercept small. Display spend is nearly constant, so the data identifies only the *sum* of the intercept and display's contribution; the sampler was exploring a long, narrow ridge between them, bounded by display's effect being non-negative. Placing the prior on the overall level and deriving the intercept from it

(`intercept = level - sum(beta)`) turns that ridge into a direction along which the level does not move, and the divergences disappear. When divergences appear, the fix is almost always a reparameterization or a better prior, not a higher `target_accept`.

Then check that the model reproduces the data it was fitted on. The posterior predictive check overlays the observed series on the model's predictive interval. Like R^2, this is a necessary condition, not evidence that the decomposition is right.

```
import matplotlib.pyplot as plt
ppc = idata.posterior_predictive["y"].stack(s=("chain", "draw")).values *
scales["y_scale"]
band = np.quantile(ppc, [0.05, 0.95], axis=1)
cover = np.mean((df["revenue"] >= band[0]) & (df["revenue"] <= band[1]))
fig, ax = plt.subplots(figsize=(7, 2.8))
ax.fill_between(df["week"], band[0] / 1e3, band[1] / 1e3, alpha=0.25, label="90%
predictive interval")
ax.plot(df["week"], df["revenue"] / 1e3, lw=1, color="#333", label="Observed")
ax.set_ylabel("Revenue (EUR k)"); ax.legend(loc="upper left", fontsize=8)
plt.tight_layout(); plt.show()
```

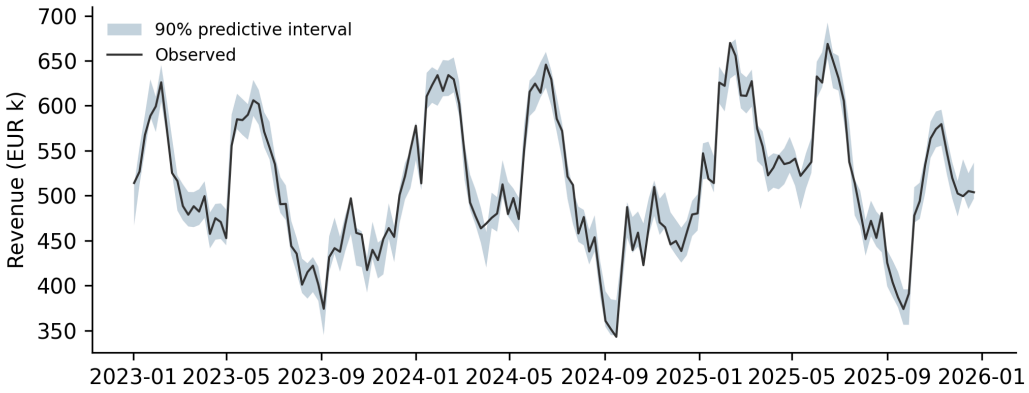


Figure 10.1: Posterior predictive check: observed weekly revenue against the model's 90% predictive interval.

```
print(f"Share of weeks inside the 90% interval: {cover:.0%}")
```

```
## Share of weeks inside the 90% interval: 92%
```

10.5 Results: what the posterior says

```
roi_post = post["roi"].stack(s=("chain", "draw")).values
classical = {"tv": 2.28, "social": 1.64, "search": 6.99, "display": 23.18}   # from
Chapter 9
res = pd.DataFrame({
    "true": [truth["roi"][c] for c in CHANNELS],
    "classical": [classical[c] for c in CHANNELS],
    "posterior median": np.median(roi_post, 1),
    "90% low": np.quantile(roi_post, 0.05, 1), "90% high": np.quantile(roi_post, 0.95,
1),
    "prior 90% width": np.diff(np.quantile(roi_prior, [0.05, 0.95], 1), axis=0)[0],
    "posterior 90% width": np.diff(np.quantile(roi_post, [0.05, 0.95], 1), axis=0)[0],
}, index=CHANNELS)
print(res.round(2).to_string())
```

```
##          true  classical  posterior median  90% low  90% high  prior 90% width
posterior 90% width
## tv       2.43       2.28              2.27     2.18      2.36             4.00
0.18
## social   2.04       1.64              1.52     0.76      2.60             3.89
1.83
## search   2.89       6.99              6.54     5.78      7.49             3.69
1.71
## display  0.68      23.18              1.04     0.08      3.39             3.91
3.31
```

```
nums = {col: {c: float(v) for c, v in res[col].items()} for col in res.columns}   #
for inline text
```

```
fig, ax = plt.subplots(figsize=(7, 2.9))
yy = np.arange(4)[::-1]
ax.errorbar(res["posterior median"], yy,
            xerr=[res["posterior median"] - res["90% low"], res["90% high"] -
res["posterior median"]],
            fmt="o", capsize=4, label="Posterior (90%)")
ax.scatter(res["true"], yy, marker="x", s=60, color="#C8102E", zorder=3,
label="Truth")
ax.set_yticks(yy, CHANNELS); ax.set_xlabel("ROI (revenue per EUR)");
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

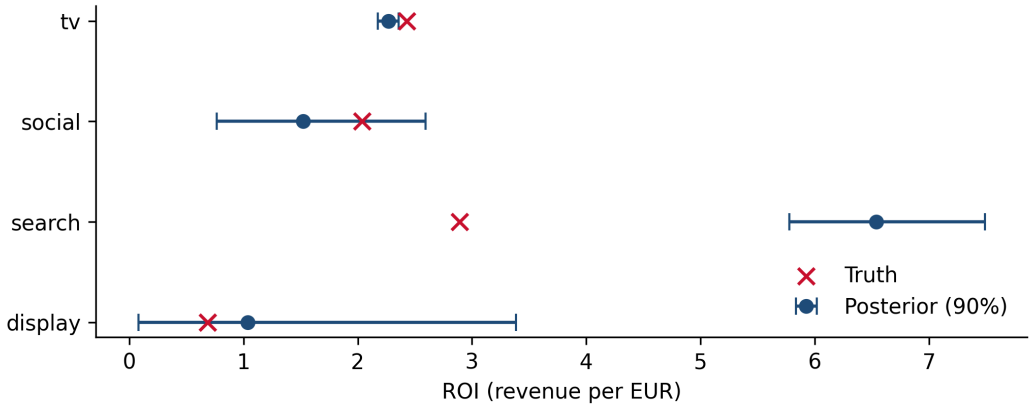


Figure 10.2: Posterior ROI by channel (median and 90% interval) against the truth. Display's interval is mostly prior; search's is precise and wrong.

Compare this with the classical fit, channel by channel.

TV is close to the truth, and its interval is narrow (2.18 to 2.36). Look closely, though: the true value of 2.43 sits just above it. Even the best-identified channel absorbs a little bias from what the model gets wrong elsewhere, and a 90% interval is a statement about uncertainty given the model, not a guarantee.

Display is no longer absurd. Its posterior median is 1.04 against a true 0.68, but look at the widths: its posterior interval is nearly as wide as its prior interval. The data has told the model almost nothing about display, and the posterior is mostly the prior speaking. The correct readout is "we cannot measure display with this data", and the correct action is an experiment, not a budget decision based on this number.

Search is still badly overstated, with a median of 6.54 against a true 2.89, and its interval is narrow. This is the dangerous case. Priors regularize; they do not remove confounding. The model sees search spend rising when revenue rises and has no way to know that both are driven by demand shocks it cannot observe. A Bayesian model is just as fooled as a classical one, only with tighter intervals around the wrong answer.

Social sits between the two: its estimate competes with the seasonal terms, and the interval reflects that.

The lesson for practice: read the posterior for each channel against its prior. A narrow posterior far from the prior means the data is informative, which is good news only if the channel's spend is not confounded with demand. A posterior that looks like the prior means the data is uninformative. Both cases point to the same next step for the channels that matter: an experiment, which is where Chapter 11 picks up.

Pitfall.
Taking a narrow posterior interval as proof of a well-measured effect. Interval width reflects the information in the data given the model; it says nothing about bias from a missing confounder. Search spend that follows demand will produce a tight, wrong interval in any MMM, Bayesian or not.

10.6 The same model with PyMC-Marketing

Libraries wrap this structure with sensible defaults, plotting and optimization. With PyMC-Marketing (version 1.x at the time of writing), the equivalent model is set up roughly as follows; check the current documentation, because the API evolves.

```
from pymc_marketing.mmm import MMM, GeometricAdstock, LogisticSaturation

mmm = MMM(
    date_column="week",
    channel_columns=["tv", "social", "search", "display"],
    control_columns=["promo"],
    adstock=GeometricAdstock(l_max=8),
    saturation=LogisticSaturation(),
    yearly_seasonality=2,
)
X = df.drop(columns="revenue")
mmm.fit(X, df["revenue"], target_accept=0.95, random_seed=3)

# Calibration with a lift test (Chapter 11) and budget optimization (Chapter 12)
# are built in: see mmm.add_lift_test_measurements() and the budget optimizer.
```

Google's **Meridian** takes a similar approach with a different emphasis: it is built for geo-level hierarchical data, models reach and frequency directly, and is typically parameterized with priors on ROI rather than on raw coefficients, which makes it natural to plug in experiment results as ROI priors.

10.7 Checklist

Checklist

- Is every prior written down with a reason, and does the prior predictive check imply plausible ROIs and media shares?
- Were divergences, R-hat and effective sample sizes checked before reading results?
- Does the posterior predictive check cover the data at roughly the nominal rate?
- For each channel, how does the posterior compare with the prior? Which channels are data-driven and which are prior-driven?
- Which channels' spend responds to demand? Are their estimates flagged as likely upward-biased until tested?

10.8 Further reading

- *PyMC-Marketing* documentation.
- *Budgeting with Bayesian models* (Bolt) and the *HelloFresh* Bayesian MMM post as production case studies.

Full links are in Appendix A.

Chapter 11 Validating and calibrating MMM

> **The decision this chapter supports.**
> Whether an MMM's channel estimates can be trusted for budget decisions, which checks actually catch the errors that matter, and how to use experiments to correct the model where it is wrong.

11.1 What validation has to catch

Chapters 9 and 10 showed two kinds of error in the same dataset. Display was **unidentified**: the data said nothing about it, and the classical model produced nonsense while the Bayesian one returned its prior. Search was **confounded**: its spend follows demand, and both models credited it with that demand, precisely and wrongly.

A validation procedure is only useful if it can catch both. The table below lists the common checks and what each can and cannot detect.

Check	Catches	Does not catch
In-sample fit (R^2, MAPE, posterior predictive)	Gross misspecification	Wrong decomposition; confounding
Out-of-sample forecast	Overfitting; unstable base	Wrong decomposition; confounding
Stability across refits and windows	Unidentified channels (estimates jump)	Stable confounding
Prior sensitivity	Prior-driven channels	Confounding
Parameter recovery on simulated data	Method that cannot recover truth even in principle	Problems specific to your real data
Business sense checks (base share, ROI ranges)	Absurd results	Plausible but wrong results
Agreement with experiments	**Confounding and bias of all kinds**	Only covers channels and periods that were tested

Only the last row catches confounding. Everything else is useful, but none of it can tell you that a precise, stable, plausible, well-fitting estimate is biased.

11.2 Out-of-sample forecasts and stability

A common first check is to refit the model on a shorter history and forecast the following quarter. We do this for the classical model from Chapter 9, at four cut-off points, and record both the forecast error and each channel's estimated ROI.

```
import sys; sys.path.insert(0, ".")
import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd
from pms_sim import simulate, CHANNELS
from pms_classical import fit_classical

df, truth = simulate()
X, y, promo = df[CHANNELS].to_numpy(), df["revenue"].to_numpy(),
df["promo"].to_numpy()
rows = []
for end in [104, 117, 130, 143]:
    _, _, predict = fit_classical(X[:end], y[:end], promo[:end])
    yhat, contrib = predict(X[:end + 13], promo[:end + 13])
    rows.append({"fit on weeks": f"1–{end}",
                 "next-13-week MAPE": f"{np.mean(np.abs(y[end:end+13] /
yhat[end:end+13] - 1)):.1%}",
                 **{c: round(contrib[:end, i].sum() / X[:end, i].sum(), 2) for i, c in
enumerate(CHANNELS)}})
out = pd.DataFrame(rows).set_index("fit on weeks")
out.loc["truth"] = ["", *[round(truth["roi"][c], 2) for c in CHANNELS]]
print(out.to_string())
```

```
##              next-13-week MAPE    tv  social  search  display
## fit on weeks
## 1–104                     2.0%  2.20    1.44    6.58    29.61
## 1–117                     1.5%  2.25    1.45    6.88    41.00
## 1–130                     1.9%  2.21    1.32    7.39    24.86
## 1–143                     1.4%  2.26    1.51    7.35    16.36
## truth                           2.43    2.04    2.89     0.68
```

The out-of-sample forecasts are excellent in every window, around 2% error. On that basis alone, the model would pass most validation reviews. The stability check is more revealing: display's ROI jumps around wildly between windows, which correctly flags it as unidentified. Search, however, is perfectly stable at more than twice its true value. **Stability catches unidentified channels; it does not catch confounded ones**, because confounding is a stable feature of the data.

11.3 Calibrating with experiments

Suppose a geo test on search ran for 12 weeks (weeks 121 to 132) and measured an ROI of 2.83 with a standard error of 0.4 over that window, on backend revenue. That is exactly the kind of evidence that can correct a confounded channel.

There are two common ways to bring it into a Bayesian MMM:

- **As a prior on the channel's ROI.** Simple, and the approach Meridian is built around. The weakness is that the experiment measured the ROI in one window at one spend level, while the prior applies to the whole period.
- **As an extra likelihood term.** The model computes the ROI it implies for the test window and spend level, and that implied value is compared with the measurement. This is how PyMC-Marketing's lift-test calibration works, and it is what `build_mmm` does when given a `lift` argument (the last block of the model code in Chapter 10).

```
lift = {"channel": "search", "start": 120, "end": 132, "roi": 2.83, "se": 0.4}
model, _ = build_mmm(df, CHANNELS, lift=lift)
```

A second, independent fix targets the cause rather than the symptom. Search is confounded because its spend responds to demand the model cannot see. If an observable proxy for that demand exists, adding it as a control blocks the confounding path (Chapter 3). Here we use a **category-level** demand index, such as category search interest from a trends tool. The choice of proxy matters: *branded* search volume would be a bad control, because our own media drive it, which makes it a mediator rather than a confounder.

We fitted four versions of the model: the base model, the model calibrated with the geo test, the model with the demand proxy, and the model with both.

```
import arviz as az
names = {"base": "Base", "calibrated": "Geo test cal.",
         "proxy": "Demand proxy", "proxy_calibrated": "Proxy + cal."}
roi = {k: az.from_netcdf(f"_cache/mmm_{k}.nc").posterior["roi"].stack(s=("chain",
"draw")).values
       for k in names}
tab = pd.DataFrame({names[k]: [f"{np.median(r[i]):.2f} [{np.quantile(r[i], .05):.2f},
{np.quantile(r[i], .95):.2f}]"
                               for i in range(4)] for k, r in roi.items()},
index=CHANNELS)
tab["truth"] = [round(truth["roi"][c], 2) for c in CHANNELS]
print("ROI, posterior median [90% interval]:\n" + tab.to_string())
```

```
## ROI, posterior median [90% interval]:
##                       Base      Geo test cal.       Demand proxy       Proxy + cal.
truth
## tv       2.27 [2.18, 2.36]  2.24 [2.15, 2.34]  2.30 [2.23, 2.37]  2.30 [2.23, 2.38]
2.43
## social   1.52 [0.76, 2.60]  1.78 [0.88, 3.00]  1.68 [0.99, 2.60]  1.71 [1.01, 2.66]
2.04
## search   6.54 [5.78, 7.49]  4.74 [4.28, 5.21]  3.36 [2.20, 4.67]  3.09 [2.47, 3.73]
2.89
## display  1.04 [0.08, 3.39]  1.23 [0.10, 3.73]  1.07 [0.11, 3.35]  1.07 [0.09, 3.26]
0.68
```

```
import matplotlib.pyplot as plt
i = CHANNELS.index("search")
fig, ax = plt.subplots(figsize=(7, 2.6))
yy = np.arange(4)[::-1]
for y_, (k, r) in zip(yy, roi.items()):
    lo, med, hi = np.quantile(r[i], [0.05, 0.5, 0.95])
    ax.errorbar(med, y_, xerr=[[med - lo], [hi - med]], fmt="o", capsize=4,
color="#1F4E79")
ax.axvline(truth["roi"]["search"], ls="--", color="#C8102E", label="True search ROI")
ax.set_yticks(yy, list(names.values())); ax.set_xlabel("Search ROI");
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

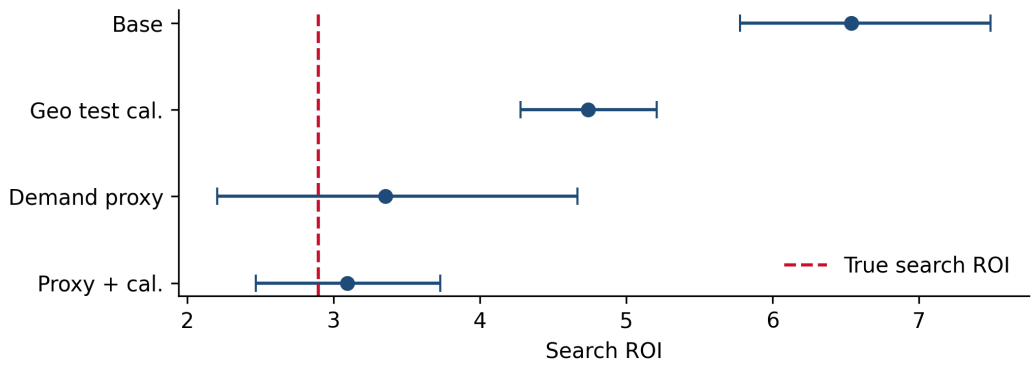


Figure 11.1: Search ROI under four model specifications (median and 90% interval) against the truth (dashed).

11.4 What calibration did and did not do

Calibration alone helped, but not enough. The geo test pulled search from about 6.5 to under 5, a large correction, yet the interval still excludes the truth. The reason is instructive. The experiment constrains what the model says about weeks 121 to 132. In the other 144 weeks, nothing has changed: search spend still moves with unobserved demand, and the model can still credit it with that demand. It reconciles the two by fitting the test window as

well as it can while keeping search high elsewhere. A single short experiment cannot overrule three years of confounded variation.

The demand proxy addressed the cause. With a control for category demand, the search estimate falls to the right neighborhood and the interval widens to an honest width that contains the truth. Blocking the confounding path is more powerful than correcting the estimate after the fact.

Both together were best. The proxy removes most of the bias, and the experiment then sharpens the estimate. The result is close to the truth with a usefully narrow interval.

Display remains unmeasured in every version. No amount of modeling fixes a channel with no variation. It needs its own experiment.

Two practical conclusions follow. First, treat calibration as part of model building, not as a patch: when an experiment and the model disagree strongly, ask why the model is wrong, and fix the specification if you can, rather than only adding the experiment as a constraint. Second, one experiment per channel is a starting point. Repeated tests across seasons and spend levels constrain the model in more places and leave it less room to hide bias outside the test windows.

> **Pitfall.**
> Calibrating an MMM to platform-attributed results, or to lift studies that measured a different outcome. Calibration transfers whatever the experiment measured into the model. If the experiment measured pixel conversions in another market, the model is now calibrated to that.

11.5 Calibrating well

- **Match the conditions.** Use the experiment's window, spend level, market and outcome definition. A likelihood term on the test window does this naturally; an ROI prior on the whole period does not.
- **Inflate uncertainty for transport.** If conditions differ (different season, spend level or market), widen the experiment's standard error to reflect the gap, as in the bias allowances of Chapter 14.
- **Prefer go-dark and scale-back tests for the channels the model is least sure about**, and heavy-up tests where the decision is about growth, so that the calibration covers the part of the response curve the decision depends on.
- **Keep a log** of every experiment used for calibration, with its conditions, so that the model's evidence base is auditable.

11.6 Other validation worth doing

Parameter recovery on simulated data. Everything in this part of the book is an example of it: simulate data with known truth that resembles your real situation (spend patterns, correlations, noise) and check whether the model recovers the truth. If it cannot recover known effects in simulation, it will not recover unknown ones in reality. Meta's siMMMulator automates this.

Prior sensitivity. Refit with meaningfully different reasonable priors. Channels whose estimates move a lot are prior-driven, which should be stated in any readout.

Business sense checks. Base share of the outcome, ROIs against benchmarks from your own past experiments, and the implied effect of switching a channel off should all be plausible to someone who knows the business. Implausible results usually mean a missing control.

11.7 Checklist

Checklist

- Has the model been refit on several windows, and are unstable channels flagged as unidentified?
- Is every channel's estimate compared with an experiment where one exists?
- For channels whose spend responds to demand, is there a demand proxy that is a confounder (category level) rather than a mediator (brand level)?
- Are experiments used for calibration matched to the model's outcome, market and spend level, with uncertainty inflated where they are not?
- Has parameter recovery been checked on data simulated to resemble the real situation?
- Is there an experiment plan for channels that remain unidentified?

11.8 Further reading

- *Calibration with experiments* (Juan Orduz) and the PyMC-Marketing lift-test calibration notebook.
- *mmm-eval* (Mutinex). An open-source framework for evaluating MMMs.
- *siMMMulator* (Meta) for simulating data with known truth.
- *Estimating the long-term base in MMM* (Niall Oulton) for the separation of base and media over longer horizons.

Full links are in Appendix A.

Chapter 12 Response curves and budget optimization

> **The decision this chapter supports.**
> How much to spend in total and how to split it across channels, based on marginal returns rather than averages, with an honest view of how much a recommended reallocation is likely to gain and how much it could lose.

12.1 Average returns do not set budgets

A channel's ROI from an MMM is an average over everything spent in it. Budget decisions are about the next euro, or the last one, and with diminishing returns those are worth less than the average. The **response curve** shows expected revenue as a function of spend; its slope is the **marginal ROI**. Two rules follow, and they are worth stating plainly because they are so often ignored:

1. **Move money from channels with low marginal ROI to channels with high marginal ROI**, until marginal returns are equal or a constraint binds. This is the logic behind classic results such as the Dorfman-Steiner condition on optimal advertising.
2. **Grow the total budget while the marginal return beats the cost of capital**, measured on profit, not revenue. If gross margin is 60%, a euro of spend has to bring back about 1.67 euros of revenue at the margin just to break even.

A channel with an excellent average ROI can be a poor place for the next euro if it is near saturation, and a channel with a mediocre average can be the best place for it if it is far from saturation.

12.2 Response curves from the posterior

Each posterior draw of the Bayesian MMM defines one response curve per channel. We use the best specification from Chapter 11, the model that includes the category demand proxy and is calibrated with the search geo test, and compute the steady-state weekly revenue that a constant weekly spend level would produce. (For a flighted channel such as TV, a steady weekly level is an approximation; budget tools usually also model flighting patterns.)

```
import sys; sys.path.insert(0, ".")
import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, arviz as az
import matplotlib.pyplot as plt
from pms_sim import simulate, CHANNELS, TRUTH, logistic_saturation
from pms_mmm import lagged, L_MAX

df, truth = simulate()
X = df[CHANNELS].to_numpy(); x_scale = X.max(0); y_scale = df["revenue"].mean()
Xlag = lagged(X / x_scale)
current = X.mean(0)                                  # average weekly spend per
channel
rng = np.random.default_rng(0)

def load_draws(name, k=500):
    post = az.from_netcdf(f"_cache/mmm_{name}.nc").posterior
    get = lambda v: post[v].stack(s=("chain", "draw")).values           # (channel,
draws)
    idx = rng.choice(get("beta").shape[1], k, replace=False)
    d = {v: get(v)[:, idx] for v in ["decay", "lam", "beta"]}
    w = d["decay"][None] ** np.arange(L_MAX)[:, None, None]             # (lag,
channel, draw)
    adst = np.einsum("lwc,lcs->wcs", Xlag, w)
    d["norm"] = logistic_saturation(adst, d["lam"][None]).mean(0)       # rescaling
used in the model
    d["carry"] = w.sum(0)                                               # steady-state
adstock multiplier
    return d

def weekly_revenue(spend, d):
    """Expected weekly revenue per channel (channel, draw) at constant weekly spend
levels."""
    a = (np.asarray(spend)[:, None] / x_scale[:, None]) * d["carry"]
    return d["beta"] * y_scale * logistic_saturation(a, d["lam"]) / d["norm"]

def true_weekly_revenue(spend):
    out = []
    for i, c in enumerate(CHANNELS):
        p = TRUTH[c]
        a = spend[i] / x_scale[i] * (p["decay"] ** np.arange(L_MAX)).sum()
        out.append(p["beta"] * logistic_saturation(a, p["lam"]))
    return np.array(out)

draws = {name: load_draws(name) for name in ["base", "calibrated",
"proxy_calibrated"]}
```

```
d = draws["proxy_calibrated"]
fig, axes = plt.subplots(2, 2, figsize=(8, 4.4))
for i, (c, ax) in enumerate(zip(CHANNELS, axes.ravel())):
    grid = np.linspace(0, 2.0 * current[i], 60)
    curves = np.array([weekly_revenue(np.where(np.arange(4) == i, g, 0.0), d)[i] for g
in grid])
    lo, med, hi = np.quantile(curves, [0.05, 0.5, 0.95], axis=1)
    ax.fill_between(grid / 1e3, lo / 1e3, hi / 1e3, alpha=0.2)
    ax.plot(grid / 1e3, med / 1e3, lw=2, label="Model")
    ax.plot(grid / 1e3, [true_weekly_revenue(np.where(np.arange(4) == i, g, 0.0))[i] /
1e3 for g in grid],
            ls="--", color="#C8102E", label="Truth")
    ax.axvline(current[i] / 1e3, ls=":", color="#6B6B6B")
    ax.set_title(c); ax.set_xlabel("Weekly spend (EUR k)"); ax.set_ylabel("Revenue
(EUR k)")
axes[0, 0].legend(fontsize=8)
plt.tight_layout()
plt.show()
```

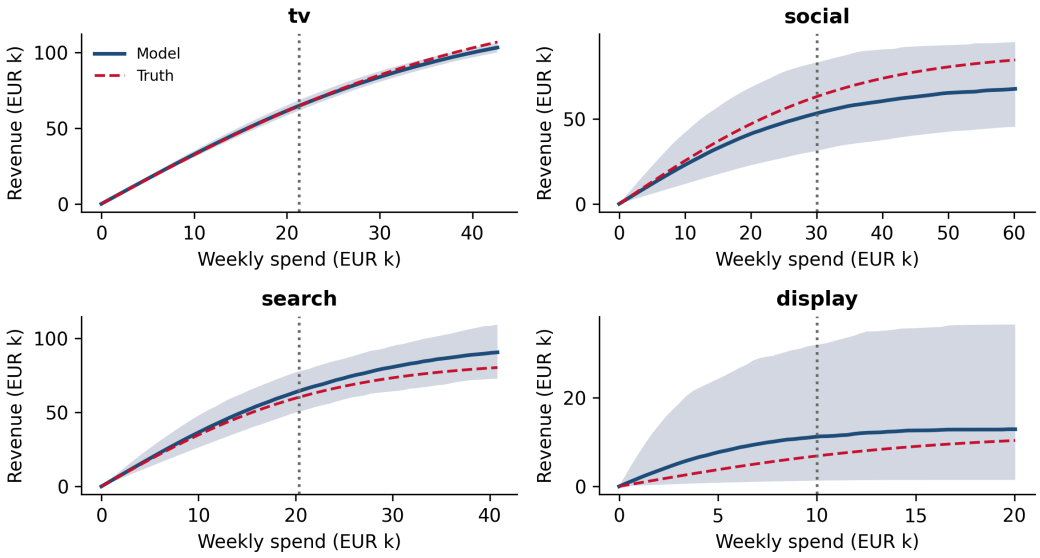


Figure 12.1: Weekly response curves from the calibrated model with demand proxy (median and 90% band), the true curves (dashed), and current average spend (dotted).

12.3 Marginal ROI at current spend

```
def marginal_roi(spend, d, step=0.01):
    up = weekly_revenue(spend * (1 + step), d)
    return (up - weekly_revenue(spend, d)) / (spend[:, None] * step)

rows = {}
for name, d in draws.items():
    m = marginal_roi(current, d)
    rows[name] = [f"{np.median(m[i]):.2f} [{np.quantile(m[i], .05):.2f},
{np.quantile(m[i], .95):.2f}]"
                  for i in range(4)]
true_m = (true_weekly_revenue(current * 1.01) - true_weekly_revenue(current)) /
(current * 0.01)
tbl = pd.DataFrame(rows, index=CHANNELS)
tbl["truth"] = true_m.round(2)
print("Marginal ROI at current spend, median [90% interval]:\n" + tbl.to_string())
```

```
## Marginal ROI at current spend, median [90% interval]:
##                         base         calibrated   proxy_calibrated  truth
## tv       2.44 [2.36, 2.53]  2.41 [2.32, 2.50]  2.48 [2.40, 2.54]   2.58
## social   0.83 [0.41, 1.25]  0.93 [0.48, 1.48]  0.90 [0.54, 1.30]   1.31
## search   5.97 [5.44, 6.47]  4.62 [4.11, 5.04]  2.11 [1.41, 2.71]   1.82
## display  0.24 [0.00, 1.69]  0.28 [0.00, 2.11]  0.24 [0.00, 1.44]   0.52
```

Compare these with the average ROIs in Chapter 10. For social, search and display, the marginal return is well below the average: the next euro buys much less than the typical one. Search is the starkest case. Its true marginal ROI is under 2 against an average near 3, and the uncalibrated model puts it near 6. Only the model that both controls for demand and is calibrated with the geo test gets close.

TV is the exception, and an instructive one: its marginal ROI at a steady average spend level is slightly above its average ROI. TV actually ran in heavy flights, which push each flight further up the saturation curve than the same budget spread evenly would. The response curve answers "what if we spent this much every week", which is a different plan from the one that generated the data. Curves for flighted channels need to be read with the flighting pattern in mind.

12.4 Optimizing the allocation

With the total weekly budget held fixed, we search for the allocation that maximizes expected revenue, averaging over posterior draws, with each channel kept between 50% and 150% of its current level. The bounds are not a technicality: outside the range of spend the model has seen, the curves are extrapolations driven by the functional form and the priors, and an unconstrained optimizer will happily exploit that.

```
from scipy.optimize import minimize

budget = current.sum()

def optimize(d):
    # Optimize over multipliers of current spend: well-scaled for the optimizer
    obj = lambda m: -weekly_revenue(m * current, d).sum(0).mean() / y_scale
    r = minimize(obj, np.ones(4), method="SLSQP", bounds=[(0.5, 1.5)] * 4,
                 constraints={"type": "eq", "fun": lambda m: (m * current).sum() /
budget - 1})
    return r.x * current

results = {}
for name, d in draws.items():
    alloc = optimize(d)
    model_gain = weekly_revenue(alloc, d).sum(0) - weekly_revenue(current, d).sum(0)
# per draw
    true_gain = true_weekly_revenue(alloc).sum() - true_weekly_revenue(current).sum()
    results[name] = {**{c: alloc[i] / current[i] - 1 for i, c in enumerate(CHANNELS)},
                     "forecast gain": model_gain.mean(),
                     "P(gain>0)": (model_gain > 0).mean(),
                     "true gain": true_gain}
out = pd.DataFrame(results).T
fmt = out.copy()
for c in CHANNELS: fmt[c] = out[c].map("{:+.0%}".format)
for c in ["forecast gain", "true gain"]: fmt[c] = out[c].map("{:,.0f}".format)
fmt["P(gain>0)"] = out["P(gain>0)"].map("{:.0%}".format)
print("Change in weekly spend by channel; weekly revenue gain (EUR):\n" +
fmt.to_string())
```

```
## Change in weekly spend by channel; weekly revenue gain (EUR):
##                     tv social search display forecast gain P(gain>0) true gain
## base              +46%   -50%   +50%    -50%        55,217      100%     6,826
## calibrated        +40%   -46%   +50%    -50%        40,720      100%     6,945
## proxy_calibrated  +50%   -37%   +27%    -50%        15,583       99%    11,371
```

```
gains = {m: {k: float(out.loc[m, k]) for k in ["forecast gain", "true gain"]} for m in
out.index}
```

This table is the core lesson of the chapter. Each row is the recommendation a model would make with the same budget, scored twice: by the model's own forecast, and by what the reallocation would truly deliver.

The **uncalibrated** model believes search has a high marginal return, so it pushes search to its upper bound. It forecasts a weekly gain of 55,217 EUR and is certain the gain is positive. Following its advice would truly gain 6,826 EUR a week. That is still a gain, mostly because

moving money from social and display into TV happens to be right, but it is a fraction of what was promised.

The model **calibrated with the geo test alone** still overstates search, still pushes it to the bound, and still overpromises: a forecast of 40,720 EUR against a true 6,945 EUR.

The model that **controls for demand and is calibrated** moves less money into search and more into TV. It forecasts 15,583 EUR and truly delivers 11,371 EUR: the largest real gain of the three, and the only forecast close to reality.

An optimizer is an amplifier. It takes whatever the model believes most confidently and acts on it at the boundary of what it is allowed to do. If that belief is a confounded coefficient, the optimizer turns a measurement error into a budget error. This is why optimization should come last, after calibration, and why the constraints should be tight until the curves have been tested.

> **Pitfall.**
> Running a budget optimizer on an uncalibrated MMM and presenting the result as a plan. The channels most likely to be overstated (those whose spend follows demand) are exactly the ones an optimizer will push hardest.

12.5 Setting the total budget

The same machinery answers the level question: increase the total budget in steps, re-optimize the split at each step, and track the marginal return of the last euro. Stop where it falls below the break-even threshold on profit (1.67 at a 60% margin in the example above), or below the return available from other uses of the money. Report the answer as a range, because the marginal return's uncertainty grows quickly as spend moves away from historical levels.

12.6 Turning a recommendation into a test

The most defensible way to act on an optimizer is to treat its output as a hypothesis. Move part of the way toward the recommended allocation, in a way that can be measured: a heavy-up in a subset of regions for the channel the model wants to grow, a scale-back in regions for the channel it wants to cut (Chapter 6). The result either confirms the curve, in which case you move further, or corrects it, in which case the model is recalibrated. Over a few cycles this converges on an allocation that rests on measured marginal returns rather than on functional-form assumptions.

12.7 Checklist

Checklist

- Is the decision based on marginal ROI at current spend, not average ROI?
- Is the break-even threshold set on profit (margin-adjusted), not revenue?
- Are the curves from a calibrated model, and are channels with confounded or prior-dominated estimates flagged?
- Are optimizer bounds kept within the historically observed spend range?
- Is the expected gain reported with its uncertainty and the probability of a loss?
- Is the recommendation implemented as a measurable step (heavy-up or scale-back test) rather than all at once?

12.8 Further reading

- *Dorfman-Steiner theorem* for the classic economics of optimal advertising budgets.
- *Budgeting with Bayesian models* (Bolt) for optimizers built on Bayesian MMMs.
- PyMC-Marketing and Meridian budget optimizer documentation.

Full links are in Appendix A.

Chapter 13 Attribution and its limits

> **The decision this chapter supports.**
> What multi-touch attribution can and cannot tell you, which questions it is still good for, and why no path-based model, rule-based or data-driven, answers the incrementality question.

13.1 What attribution does

Attribution takes the conversion paths users followed (the sequence of ad and channel touches before converting or not) and divides credit for conversions among the touched channels. Rule-based models use a fixed rule: last touch, first touch, linear, position-based, time decay. Data-driven models learn weights from the paths: Markov chain removal effects, Shapley values, and the proprietary models inside ad platforms and analytics tools.

All of them share one property that determines what they can do. **They distribute observed conversions among the touches that preceded them.** None of them compares what happened with what would have happened without a channel. In the vocabulary of Chapter 1, they produce attributed numbers, not incremental ones. The data-driven models are more sophisticated about the distribution, but they distribute the same pool.

13.2 Worked example: five attribution models against the truth

We simulate 200,000 users with different baseline purchase intent and four channels:

- **Social** prospecting reaches users at random and has the largest true effect (+1.0 percentage point).
- **Display** retargeting and **email** target users who already show intent; their true effects are small (+0.1 and +0.2 points).
- **Search** is reached mostly by high-intent users who look the brand up, with a moderate true effect (+0.4 points).

Each touch gets a time, so every user has an ordered path. Social tends to come early, search late.

```
import numpy as np, pandas as pd, itertools, math
from collections import Counter

rng = np.random.default_rng(13)
n = 200_000
CH = ["social", "display", "email", "search"]
intent = rng.beta(1, 25, n)                                  # baseline conversion
probability
exposed = {
    "social":  rng.random(n) < 0.30,                         # random reach
    "display": rng.random(n) < np.clip(intent * 5 + 0.05, 0, 1),   # selected on
intent
    "email":   rng.random(n) < np.clip(intent * 4, 0, 1),
    "search":  rng.random(n) < np.clip(intent * 10, 0, 1),
}
lift = {"social": 0.010, "display": 0.001, "email": 0.002, "search": 0.004}
u = rng.random(n)

def converts(off=None):
    p = intent + sum(lift[c] * exposed[c] for c in CH if c != off)
    return u < p

y = converts()
true_incr = {c: y.sum() - converts(off=c).sum() for c in CH}

# Ordered paths from touch times: social early, search late
window = {"social": (0, .6), "display": (.3, .9), "email": (.2, 1), "search": (.6, 1)}
times = np.column_stack([np.where(exposed[c], rng.uniform(*window[c], n), np.inf) for
c in CH])
order, n_touch = np.argsort(times, axis=1), np.isfinite(times).sum(1)
paths = [tuple(CH[j] for j in order[i, :n_touch[i]]) for i in range(n)]

print(f"Conversions: {y.sum():,};  conversions with at least one touch: {(y & (n_touch
> 0)).sum():,};  "
      f"truly incremental: {sum(true_incr.values()):,}")
```

```
## Conversions: 8,696;  conversions with at least one touch: 7,382;  truly
incremental: 959
```

Now the rule-based models, then the two common data-driven ones. The **Markov** model treats paths as a chain of states from start to conversion or drop-out; a channel's removal effect is how much the conversion probability falls if paths through it are redirected to drop-out. The **Shapley** model treats channels as players in a cooperative game whose value is the number of conversions achieved by each coalition of channels.

```
res = pd.DataFrame(index=CH)
res["true incremental"] = pd.Series(true_incr)
last, first, linear = (pd.Series(0.0, index=CH) for _ in range(3))
for p, conv in zip(paths, y):
    if conv and p:
        last[p[-1]] += 1; first[p[0]] += 1
        for c in p: linear[c] += 1 / len(p)
res["last touch"], res["first touch"], res["linear"] = last, first, linear

# Markov chain: transition counts between start, channels, conversion and drop-out
states = ["start"] + CH + ["conv", "null"]
ix = {s: i for i, s in enumerate(states)}
T = np.zeros((len(states), len(states)))
for p, conv in zip(paths, y):
    seq = ["start", *p, "conv" if conv else "null"]
    for a, b in zip(seq[:-1], seq[1:]):
        T[ix[a], ix[b]] += 1

def p_conversion(removed=None):
    P = T.copy()
    if removed:                                  # redirect transitions into
`removed` to drop-out
        r = ix[removed]; P[:, ix["null"]] += P[:, r]; P[:, r] = 0
    P = np.divide(P, P.sum(1, keepdims=True), out=np.zeros_like(P), where=P.sum(1,
keepdims=True) > 0)
    tr = [ix[s] for s in ["start"] + CH]         # transient states
    return np.linalg.solve(np.eye(len(tr)) - P[np.ix_(tr, tr)], P[np.ix_(tr,
[ix["conv"]])])[0, 0]

removal = pd.Series({c: 1 - p_conversion(c) / p_conversion() for c in CH})
touched_conv = (y & (n_touch > 0)).sum()
res["markov"] = removal / removal.sum() * touched_conv

# Shapley: v(S) = conversions of users whose set of touched channels is contained in S
conv_by_set = Counter(frozenset(p) for p, conv in zip(paths, y) if conv and p)
v = lambda S: sum(k for s, k in conv_by_set.items() if s <= S)
shapley = {}
for c in CH:
    others = [o for o in CH if o != c]
    shapley[c] = sum(math.factorial(r) * math.factorial(len(CH) - r - 1) /
math.factorial(len(CH))
                     * (v(frozenset(S) | {c}) - v(frozenset(S)))
                     for r in range(len(others) + 1) for S in
itertools.combinations(others, r))
res["shapley"] = pd.Series(shapley)
print(res.round(0).astype(int).to_string())
```

```
##          true incremental  last touch  first touch  linear  markov  shapley
## social                564         882         2758    1698    1682     1698
## display                51        1087         1750    1584    1613     1584
## email                  68         861         1189    1075    1130     1075
## search                276        4552         1685    3025    2957     3025
```

```
import matplotlib.pyplot as plt
ratio = res.drop(columns="true incremental").div(res["true incremental"], axis=0)
fig, ax = plt.subplots(figsize=(7, 3))
ratio.plot.bar(ax=ax, width=0.8, rot=0)
ax.axhline(1, color="#333", lw=1); ax.set_yscale("log")
ax.set_ylabel("Credit / true incremental")
ax.legend(fontsize=8, ncol=5, loc="upper center", bbox_to_anchor=(0.5, -0.15))
plt.tight_layout()
plt.show()
```

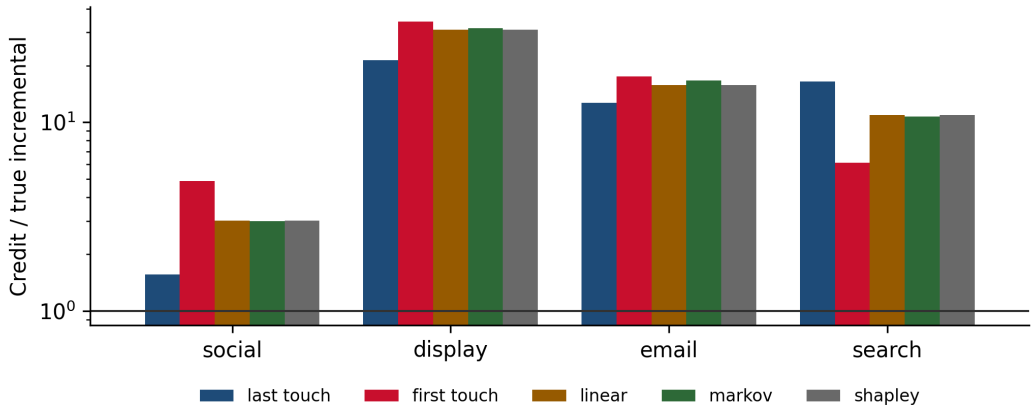


Figure 13.1: Credit per channel under each model, as a multiple of the channel's true incremental conversions. A value of 1 would be correct.

13.3 What the example shows

Every model hands out far more credit than there is incrementality. All five distribute the conversions of touched users, around seven and a half thousand, while fewer than a thousand were caused by the advertising. Most of what attribution credits would have happened anyway. Using attributed conversions to judge whether channels pay for themselves will overstate returns across the board.

Every model gets the ranking wrong. The channel that truly drives the most incremental conversions is social. Last touch ranks it third. The linear, Markov and Shapley models move it up to second, but all of them still put search first, because high-intent users who were going to convert anyway search for the brand right before converting. Display and email,

with tiny true effects, receive credit worth fifteen to thirty times what they deliver, because they target people who are already converting.

Data-driven is not the same as causal. The Markov and Shapley models are more defensible than last touch, but they learn from the same paths, and the paths are shaped by targeting. A channel aimed at high-intent users will look valuable in any model that learns from who converts after seeing it.

Shapley here is exactly linear attribution. With the common coalition definition used above (a coalition is credited with the conversions of users whose touches fall within it), the Shapley value splits each conversion equally among the channels in its path. That is a known result, and it means a "game theory" model can be simpler than it sounds. Other value functions give different answers, but none of them introduces a counterfactual.

> **Pitfall.**
> Replacing last-touch with a data-driven model and treating the problem as solved. Data-driven attribution reduces the most obvious biases of rule-based models, but it remains a correlational allocation of observed conversions. It does not measure incrementality, and it inherits whatever selection the targeting creates.

13.4 What attribution is still good for

Attribution is the wrong tool for budget decisions between channels. It is still useful for questions where the selection bias is roughly constant across the things being compared:

- **Within-channel optimization**: comparing campaigns, ad sets, keywords or creatives inside one channel with similar targeting.
- **Journey analysis**: which channels typically appear early and late, how long paths are, where users drop off.
- **Fast operational signals**: daily monitoring of trends and anomalies, ahead of slower methods.
- **Feeding models**: path data can inform priors on how channels interact, as long as nobody reads the credit as incremental.

When attribution numbers are reported to decision makers, convert them with an incrementality factor from an experiment (Chapter 8) wherever one exists, and say clearly when one does not.

13.5 Signal loss and cookieless approaches

Path-based attribution also depends on stitching touches to the same user, which is exactly what signal loss breaks (Chapter 2). Paths become shorter and more fragmented, and the channels whose touches are hardest to observe (upper-funnel video, cross-device journeys, users who decline consent) lose credit systematically. Aggregate approaches are the response: regression-based attribution, which models conversions on channel activity over time (Google's open-source *rba* is an example), is essentially a small MMM, and inherits both the strengths and the confounding problems of Chapter 9.

13.6 Checklist

Checklist

- Is the attribution output being used to compare channels or set budgets? If so, stop and look for incrementality evidence.
- Are attributed conversions labeled as attributed in every report, never as incremental?
- For each channel, is its targeting correlated with existing intent (retargeting, CRM, branded search)? Expect over-credit.
- Where an incrementality factor exists, is it applied, with its date and conditions?
- How much of the journey is invisible because of consent, devices or platforms, and which channels does that hurt?

13.7 Further reading

- *How to game attribution models* (Admend, archived).
- Markov chain attribution: the `ChannelAttribution` package and the tutorials.
- Shapley value attribution: `shap` / `shapiq` for Shapley values in machine learning more broadly.
- The `mta` Python package for further model variants.

Full links are in Appendix A.

Chapter 14 Triangulation: deciding under contradictory evidence

> **The decision this chapter supports.**
> What to believe about a channel when the MMM, an experiment, a platform lift study and attribution all say different things, how to turn that into a budget decision, and which test to run next.

14.1 The situation you are actually in

By this point in the book, each method has been shown to answer a specific question under specific assumptions. In practice you rarely have a single clean answer. For a typical large channel you might have an MMM posterior, one geo test from last spring, a platform lift study from a different market, and the platform's attributed numbers updated daily. They disagree. The decision cannot wait for them to agree.

Triangulation is the discipline of combining these sources in a way that respects what each one measures, and of using the disagreement itself as a guide to what to measure next.

14.2 Every estimate has two kinds of error

When you combine estimates, it is tempting to weight them by their reported precision. That is right only if each estimate is measuring the same quantity without bias. In practice each source has two kinds of error:

- **Sampling error**, the uncertainty the method reports: a confidence interval, a posterior interval.
- **Transport and bias error**, which the method does not report: the gap between what it measured and the quantity you need for the decision. A geo test ran in four regions for six weeks at one spend level; the MMM coefficient may be confounded; the lift study counted pixel conversions in another market; attribution measures something else entirely.

A workable approach is to give each source an explicit allowance for the second kind of error and weight by total variance, \(1 / (\text{se}^2 + b^2)\), where \(b\) is your judgment of how far the source could be from the target quantity for reasons its own interval does not capture. The allowances are judgments, and they should be written down and argued over, just like priors (Chapter 4).

14.3 Worked example: what is search really worth?

Take the search channel from Part III. The evidence on its ROI, in euros of revenue per euro of spend:

```
import numpy as np, pandas as pd
from scipy import stats

ev = pd.DataFrame({
    "estimate": [7.1, 2.8, 3.4, 9.5],
    "reported se": [0.6, 0.4, 0.7, 0.3],
    "bias allowance": [3.0, 0.5, 1.0, 6.0],
    "why": ["MMM: search spend follows demand, likely confounded upward",
            "Geo test: 12 weeks, backend revenue, normal spend level",
            "Platform lift study: pixel conversions, another market",
            "Attributed ROI: not an incrementality measure"],
}, index=["MMM", "Geo test", "Lift study", "Attribution"])

def combine(est, var):
    w = 1 / var
    return (w * est).sum() / w.sum(), np.sqrt(1 / w.sum()), w / w.sum()

naive_mu, naive_sd, naive_w = combine(ev["estimate"], ev["reported se"] ** 2)
mu, sd, w = combine(ev["estimate"], ev["reported se"] ** 2 + ev["bias allowance"] **
2)
ev["weight (naive)"], ev["weight (with allowance)"] = naive_w.round(2), w.round(2)
print(ev.drop(columns="why").to_string())
```

```
##              estimate  reported se  bias allowance  weight (naive)  weight (with
allowance)
## MMM               7.1          0.6             3.0            0.13
0.03
## Geo test          2.8          0.4             0.5            0.28
0.75
## Lift study        3.4          0.7             1.0            0.09
0.21
## Attribution       9.5          0.3             6.0            0.50
0.01
```

```
print(f"\nNaive combination:            ROI {naive_mu:.2f} (sd {naive_sd:.2f})")
```

```
##
## Naive combination:            ROI 6.75 (sd 0.21)
```

```
p_above = 1 - stats.norm.cdf(1, mu, sd)
print(f"Combination with allowances:  ROI {mu:.2f} (sd {sd:.2f}),  "
      f"P(ROI > 1) {'> 99%' if p_above > 0.99 else f'= {p_above:.0%}'}")
```

```
## Combination with allowances:  ROI 3.12 (sd 0.56),  P(ROI > 1) > 99%
```

The naive combination is dominated by the most precise number, attribution, which is also the one that measures the wrong thing. It lands far above every incrementality estimate. With explicit allowances, the experiments carry almost all the weight, attribution carries essentially none, and the MMM contributes a little. The combined estimate sits near the geo test, with an honest standard deviation.

The allowances do most of the work here, which is the point. A precise estimate of the wrong quantity should not dominate a noisy estimate of the right one, and the only way to stop that is to state how wrong each source could be.

14.4 From belief to decision

The decision on the table is whether to increase search spend by 20,000 EUR a month. That needs a **marginal** return, not the average one estimated above (Chapter 12). Search saturates quickly, so suppose the response curve implies the marginal ROI at current spend is only about 40% of the average. Then:

```
draws = stats.norm.rvs(mu, sd, size=100_000, random_state=1) * 0.4   # marginal ROI
draws
extra = 20_000
profit = extra * (draws - 1)
print(f"Marginal ROI: median {np.median(draws):.2f}, 90% interval "
      f"[{np.quantile(draws, .05):.2f}, {np.quantile(draws, .95):.2f}]")
```

```
## Marginal ROI: median 1.25, 90% interval [0.89, 1.62]
```

```
print(f"Expected monthly profit from +{extra:,} EUR: {profit.mean():,.0f} EUR;  "
      f"P(loss) = {(profit < 0).mean():.0%};  "
      f"expected loss if wrong: {-profit[profit < 0].mean():,.0f} EUR")
```

```
## Expected monthly profit from +20,000 EUR: 5,007 EUR;  P(loss) = 13%;  expected loss
if wrong: 2,227 EUR
```

The average ROI is comfortably above break-even, yet at the margin the picture is tighter: the expected value of the extra spend is positive, but there is a real chance it loses money. This is the gap between average and marginal returns from Chapter 1, in euros. Whether to proceed depends on how reversible the decision is and on what it would cost to reduce the uncertainty first. A spend increase that can be rolled back in a month is a cheap bet; a commitment that locks in budget for a year is not.

14.5 Diagnosing disagreement

When sources disagree, the pattern of disagreement usually points to a cause. Chapter 1 listed the common ones; with Parts II and III behind us they can be made specific.

Pattern	Most likely cause	What to do
MMM high, experiment lower	Spend follows demand (search, retargeting) or is collinear with season	Calibrate the MMM; add a demand proxy; treat MMM as upper bound
MMM low or prior-dominated, experiment positive	Too little spend variation, collinearity with another channel	Calibrate; create variation deliberately (flighting, geo splits)
Lift study high, geo test lower	Different outcome (pixel versus backend), market, or audience	Compare outcome definitions; run the geo test on the platform's outcome too
Geo test high, lift study lower	Cross-device or offline conversions the platform cannot see	Feed backend conversions into the lift study
Attribution far above everything	Normal: attribution counts touched, not caused, conversions	Apply an incrementality factor; stop using raw attribution for budgets
Two experiments disagree	Different conditions (season, spend level, creative), or noise	Check MDEs; the difference may be within noise; otherwise test conditions explain it

14.6 Deciding what to test next

A measurement program has limited testing capacity: each experiment costs money (holdout revenue), time and attention. Chapter 1 suggested prioritizing by spend, uncertainty and the chance that the result changes a decision. With posteriors in hand, that can be computed.

```
ch = pd.DataFrame({
    "monthly spend": [90_000, 130_000, 88_000, 43_000],
    "ROI mean": [2.3, 1.8, 2.9, 1.2],
    "ROI sd": [0.15, 0.6, 1.1, 1.4],
}, index=["tv", "social", "search", "display"])
ch["P(wrong side of 1)"] = stats.norm.cdf(-np.abs(ch["ROI mean"] - 1) / ch["ROI sd"])
ch["EUR at stake"] = ch["monthly spend"] * ch["ROI sd"]
ch["priority"] = (ch["EUR at stake"] * (0.05 + ch["P(wrong side of 1)"])).round(-2)
print(ch.sort_values("priority", ascending=False).round(2).to_string())
```

```
##          monthly spend  ROI mean  ROI sd  P(wrong side of 1)  EUR at stake
priority
## display          43000       1.2    1.40                0.44       60200.0
29700.0
## social          130000       1.8    0.60                0.09       78000.0
11000.0
## search           88000       2.9    1.10                0.04       96800.0
8900.0
## tv               90000       2.3    0.15                0.00       13500.0
700.0
```

The score multiplies how much money the uncertainty touches by how likely the uncertainty is to flip the go/no-go call (with a floor, because even a channel safely above break-even can be misallocated at the margin). TV, measured precisely, scores lowest. Display scores high despite its small budget, because nobody knows whether it pays back at all. Social and search follow. The output is a ranked testing roadmap, and it changes every time a result comes in.

14.7 A measurement scorecard

The last tool in this part is the simplest: one row per channel, maintained over time, showing what evidence exists and how much it can be trusted.

Channel	Best incrementality evidence	Date	Agreement across sources	Confidence	Next step
TV	MMM, strong flighting variation	Current	MMM and brand search response agree	High	Re-test after creative change
Social	Geo test, platform lift study	6 and 3 months old	Within each other's intervals	Medium	Heavy-up test to measure marginal return
Search	Geo test	4 months old	MMM far above experiment	Medium (experiment) / Low (MMM)	Calibrate MMM; repeat test in peak season
Display	None	n/a	MMM posterior is mostly prior	Low	Go-dark geo test

The scorecard makes three things visible to everyone: which numbers rest on experiments and which on models alone, how old the evidence is, and what the team plans to do about the weakest rows. It is also the best defense against the most common failure of measurement programs, which is a single number per channel presented without any sense of how well it is known.

14.8 Checklist

Checklist

- Does each source have a stated bias allowance as well as its reported uncertainty?
- Is attribution kept out of incrementality combinations, or given a large allowance?
- Is the decision made on marginal returns, with probability and size of loss reported?
- Is the reversibility of the decision considered alongside expected value?
- Is the testing roadmap prioritized by money at stake and the chance of flipping a decision?
- Is there a channel scorecard with evidence type, age and confidence, kept up to date?

14.9 Further reading

- *Google's Measurement Playbook* for a vendor's version of the triangulation logic.

Full links are in Appendix A.

Chapter 15 Cohorts, retention, LTV and payback

The decision this chapter supports.
How much an acquired customer is worth, how long it takes to earn back what it cost to acquire, and therefore how much each channel can afford to pay per customer. These numbers turn acquisition measurement from "cheapest conversions" into "most valuable conversions".

15.1 Why cohorts

Blended metrics such as "monthly churn" or "average revenue per customer" mix customers acquired at different times, through different channels, at different ages. A business acquiring fast will show low blended churn simply because most of its customers are new, and new customers have not had time to leave. A business whose acquisition is slowing will show rising churn with no change in customer behavior at all.

A **cohort** is a group of customers acquired in the same period (and, for marketing purposes, usually through the same channel). Tracking each cohort's activity by months since acquisition separates the effect of customer age from the effect of calendar time. The standard view is the cohort triangle: one row per cohort, one column per month since acquisition, with the most recent cohorts having the fewest observed months.

15.2 Worked example: two channels, twelve cohorts

We simulate twelve monthly cohorts of 800 customers from each of two channels. Each customer has their own monthly probability of leaving, drawn from a beta distribution; the channels differ in that distribution.

- **Search** brings customers with moderate, fairly uniform churn: Beta(1.2, 5.0), mean about 19% a month.
- **Social** brings a more mixed group: many who leave quickly and a loyal core. Beta(0.7, 1.6), mean about 30% a month.

Each active customer contributes 25 EUR of margin per month. We observe each cohort up to the present, so the oldest cohort has twelve months of history and the newest one month.

```
import numpy as np, pandas as pd
from scipy.special import betaln
from scipy.optimize import minimize

rng = np.random.default_rng(15)
CH = {"search": (1.2, 5.0), "social": (0.7, 1.6)}     # Beta(a, b) of monthly churn
probability
N_COHORTS, SIZE, MARGIN, HORIZON = 12, 800, 25.0, 36

def survival(t, a, b):
    """sBG survival: P(still active at month t), t = 0, 1, 2, ..."""
    return np.exp(betaln(a, b + t) - betaln(a, b))

rows = []
for c, (a, b) in CH.items():
    for k in range(N_COHORTS):
        theta = rng.beta(a, b, SIZE)                 # each customer's churn
probability
        months_active = rng.geometric(theta)         # months until they leave (>= 1)
        for m in range(N_COHORTS - k):               # months observed so far for this
cohort
            rows.append({"channel": c, "cohort": k, "month": m, "active":
(months_active > m).sum()})
cohorts = pd.DataFrame(rows)

tri = cohorts.query("channel == 'social'").pivot(index="cohort", columns="month",
values="active")
print("Social cohorts, share still active by month since acquisition:")
```

```
## Social cohorts, share still active by month since acquisition:
```

```
print(tri.div(tri[0], axis=0).iloc[:, :9].round(2).to_string(na_rep=""))
```

```
## month     0     1     2     3     4     5     6     7     8
## cohort
## 0       1.0  0.69  0.55  0.45  0.40  0.34  0.32  0.29  0.27
## 1       1.0  0.70  0.55  0.46  0.40  0.35  0.33  0.30  0.28
## 2       1.0  0.73  0.57  0.46  0.39  0.34  0.29  0.27  0.25
## 3       1.0  0.69  0.53  0.44  0.38  0.34  0.30  0.28  0.26
## 4       1.0  0.69  0.54  0.44  0.38  0.33  0.30  0.26
## 5       1.0  0.69  0.54  0.44  0.37  0.33  0.30
## 6       1.0  0.67  0.53  0.46  0.41  0.36
## 7       1.0  0.72  0.56  0.48  0.41
## 8       1.0  0.73  0.57  0.48
## 9       1.0  0.71  0.56
## 10      1.0  0.69
## 11      1.0
```

15.3 Heterogeneity makes churn fall over time

Look along any row of the triangle: the share of customers leaving each month falls as the cohort ages. Nobody's behavior is changing. Each customer has a fixed churn probability, but the customers most likely to leave do so first, so the survivors are increasingly the loyal ones. Any retention model that assumes a constant churn rate will get this wrong, and in a predictable direction.

The **shifted-beta-geometric (sBG)** model captures exactly this: each customer has a constant churn probability, and those probabilities vary across customers according to a beta distribution. It has two parameters and fits cohort data by maximum likelihood, with right-censoring handled naturally (customers still active at the last observed month contribute what we know about them: that they survived at least that long).

```
def cohort_arrays(c):
    d = cohorts[cohorts.channel == c]
    return [g.sort_values("month")["active"].to_numpy() for _, g in
d.groupby("cohort")]

def fit_sbg(arrays):
    def nll(log_ab):
        a, b = np.exp(log_ab)
        ll = 0.0
        for n in arrays:
            t = np.arange(1, len(n))
            p_churn = np.exp(betaln(a + 1, b + t - 1) - betaln(a, b))   # P(leave in
month t)
            ll += ((n[:-1] - n[1:]) * np.log(p_churn)).sum() + n[-1] *
np.log(survival(len(n) - 1, a, b))
        return -ll
    return np.exp(minimize(nll, [0.0, 0.0], method="Nelder-Mead").x)

t = np.arange(HORIZON)
res = {}
for c, (a_true, b_true) in CH.items():
    arrays = cohort_arrays(c)
    a, b = fit_sbg(arrays)
    # Naive alternative: constant churn estimated from the first three months of
retention
    first3 = np.mean([n[m + 1] / n[m] for n in arrays for m in range(min(3, len(n) -
1))])
    # Bootstrap over cohorts for an interval on the sBG-based LTV
    boot = [MARGIN * survival(t, *fit_sbg([arrays[i] for i in rng.integers(0,
len(arrays), len(arrays))])).sum()
            for _ in range(100)]
    res[c] = {"fitted a, b": f"{a:.2f}, {b:.2f}", "true a, b": f"{a_true}, {b_true}",
              "LTV true": MARGIN * survival(t, a_true, b_true).sum(),
              "LTV sBG": MARGIN * survival(t, a, b).sum(),
              "sBG 90% low": np.quantile(boot, 0.05), "sBG 90% high":
np.quantile(boot, 0.95),
              "LTV constant churn": MARGIN * (first3 ** t).sum()}
ltv = pd.DataFrame(res).T
num = ["LTV true", "LTV sBG", "sBG 90% low", "sBG 90% high", "LTV constant churn"]
ltv[num] = ltv[num].astype(float).round(0).astype(int)
print(f"36-month margin per acquired customer (EUR):\n{ltv.to_string()}")
```

```
## 36-month margin per acquired customer (EUR):
##        fitted a, b true a, b  LTV true  LTV sBG  sBG 90% low  sBG 90% high  LTV
constant churn
## search  1.13, 4.66  1.2, 5.0       229      235          228           241
147
## social  0.76, 1.79  0.7, 1.6       213      205          199           213
107
```

The sBG model recovers both channels' true lifetime value within a few percent from at most twelve months of data, and its interval covers the truth. The constant-churn extrapolation, the kind of calculation many dashboards do, underestimates lifetime value by a third to a half, and it underestimates social, the more heterogeneous channel, the most.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 2.9))
for c, color in zip(CH, ["#1F4E79", "#C8102E"]):
    arrays = cohort_arrays(c)
    obs = [np.sum([n[m] for n in arrays if len(n) > m]) / np.sum([n[0] for n in arrays
if len(n) > m])
           for m in range(N_COHORTS)]
    a, b = fit_sbg(arrays)
    first3 = np.mean([n[m + 1] / n[m] for n in arrays for m in range(min(3, len(n) -
1))])
    ax.plot(range(N_COHORTS), obs, "o", ms=4, color=color)
    ax.plot(t, survival(t, a, b), color=color, lw=2, label=f"{c}: sBG")
    ax.plot(t, first3 ** t, color=color, ls="--", lw=1.2, label=f"{c}: constant
churn")
ax.axvline(N_COHORTS - 0.5, color="#6B6B6B", lw=0.8)
ax.set_xlabel("Months since acquisition")
ax.set_ylabel("Share still active")
ax.legend(fontsize=8, ncol=2)
plt.tight_layout()
plt.show()
```

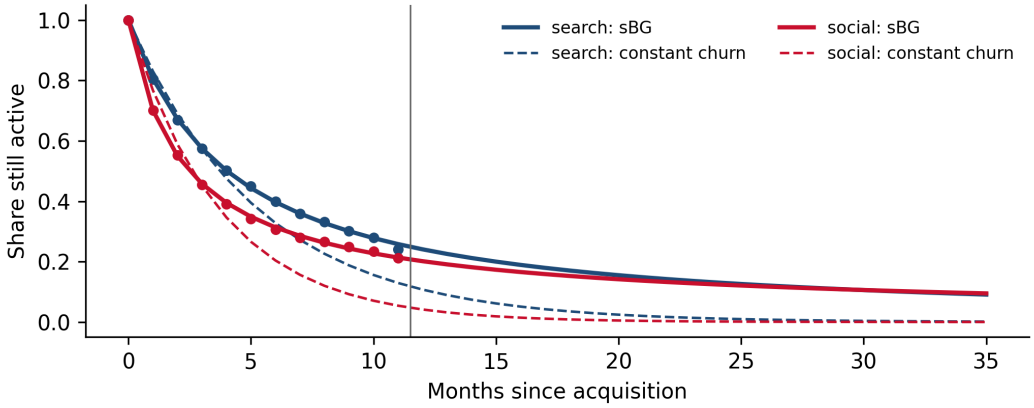


Figure 15.1: Retention by month since acquisition: observed (dots, pooled cohorts), sBG fit and projection (solid), constant-churn projection (dashed).

15.4 Payback and what a channel can afford

Lifetime value only matters relative to acquisition cost. Two numbers turn it into a decision:

- **Payback period**: months until cumulative margin per acquired customer exceeds the cost of acquiring them. It governs cash: a business that has to fund acquisition from its balance sheet cares how long the money is out.
- **Maximum affordable CAC**: the lifetime value (over a horizon the business is willing to count, discounted if the horizon is long) minus any required margin of safety. It is the ceiling for bids and channel targets.

```
cac = {"search": 180, "social": 140}
out = []
for c in CH:
    a, b = fit_sbg(cohort_arrays(c))
    cum = MARGIN * np.cumsum(survival(t, a, b))
    out.append({"channel": c, "CAC": cac[c], "12-month value": cum[11], "36-month
LTV": cum[-1],
                "payback (months)": int(np.argmax(cum >= cac[c])) + 1 if cum[-1] >=
cac[c] else None,
                "LTV / CAC": cum[-1] / cac[c]})
print(pd.DataFrame(out).set_index("channel").round(1).to_string())
```

```
##          CAC  12-month value  36-month LTV  payback (months)  LTV / CAC
## channel
## search   180           147.9         234.6                19        1.3
## social   140           124.9         205.1                16        1.5
```

Social's customers leave faster in the first months, which a retention chart alone would hold against it. But they are cheaper to acquire and their loyal core keeps paying, so social pays back sooner and returns more per euro than search. Retention curves and acquisition costs have to be read together.

Both channels are thin, though: each euro of acquisition cost returns only 1.3 to 1.5 euros of margin over three years, and a third or more of that is projection beyond what has been observed. These are also **attributed** acquisition costs. For budgeting, the CAC should be the incremental one from Parts II and III, which is usually higher, and at that cost neither channel may pay back within the horizon. Whether to keep acquiring at these levels depends on the business's cash position and how much it trusts the long tail of the projection.

> **Pitfall.**
> Judging acquisition channels on cost per customer while their customers have very different value. A channel with twice the CAC can be the better buy if its customers stay twice as long. Wherever possible, set channel targets on value per euro, not customers per euro.

15.5 Practical choices

Margin, not revenue. Lifetime value should be built from contribution margin per period (revenue minus variable costs such as payment processing, support and incentives), or it overstates what a customer is worth.

Discounting and horizon. A euro earned in year three is worth less than one earned now, and projections that far out are uncertain. Most teams cap the horizon (24 or 36 months) and discount at the business's cost of capital.

Definition of "active". For subscription businesses, active means paying. For businesses where customers can lapse and return (most transaction-based products, including many fintech products where an account stays open but goes unused), a contractual churn model like sBG does not fit; non-contractual models such as BG/NBD and Pareto/NBD model purchase frequency and the probability that a customer is still "alive" from transaction histories. PyMC-Marketing includes Bayesian versions of these.

Early prediction. Waiting a year to learn a cohort's value is too slow for marketing decisions. Two approaches help: fit a retention model to the first few months (as above, the model extrapolates far better than averages), and predict value from early behavior (activation steps, first transactions) with a model trained on older cohorts.

Measuring value effects of marketing. When an experiment changes who is acquired (an incentive, a new channel), its effect on value must be read at a horizon long enough for value to show: a fixed cohort horizon such as 60 or 90 days after acquisition, not two weeks.

> **Pitfall.**
> Treating an LTV/CAC ratio of 3 as a universal target. The right ratio depends on margin, payback speed, cost of capital and how much of the LTV is projection rather than observation. Use the ratio as a summary, and make decisions on payback and marginal returns.

15.6 Checklist

Checklist

- Are retention and value analyzed by cohort (acquisition month and channel), not as blended averages?
- Does the retention model allow churn to fall as cohorts age, rather than assuming a constant rate?
- Is lifetime value built from margin, over a stated horizon, with discounting where the horizon is long?
- Is the uncertainty of projected LTV reported, and how much of it is projection beyond observed months?
- Are channel CAC targets set on value, and on incremental rather than attributed acquisition cost?
- For experiments that change customer mix, is value read at a long enough horizon?

15.7 Further reading

- PyMC-Marketing CLV quickstart for BG/NBD, Pareto/NBD and related models.
- *LTVision* (Meta) for predicting customer value from early behavior.
- Mobile Dev Memo's posts on LTV, ROAS and the marketing P&L, including *How much data is needed to predict LTV*.
- *Retention with BART* (Juan Orduz) for flexible cohort retention models.

Full links are in Appendix A.

Chapter 16 Churn and survival analysis

> **The decision this chapter supports.**
> How long customers stay, what is associated with staying longer, which of those associations are causal enough to act on, and which customers a retention budget should actually be spent on.

16.1 Churn is a time-to-event problem

"What is our churn rate?" sounds simple and is not, because at any moment most customers have not churned *yet*. Their lifetimes are **censored**: we know they lasted at least this long, not how long they will last. Methods that ignore censoring give systematically wrong answers, and the most common mistakes all come from that.

Survival analysis is the toolkit built for this. Its central object is the **survival function** \(S(t)\), the probability that a customer is still active \(t\) months after joining, and its companion the **hazard**, the probability of leaving at \(t\) given survival until then.

16.2 Worked example: the cost of ignoring censoring

We simulate 6,000 business customers who joined over the past 24 months. Each has an unobserved level of intent (how much they need the product), which lowers their risk of leaving. Customers with higher intent are also more likely to **activate** (complete onboarding and start using the product in earnest), and activation itself also reduces churn, but by less than it appears. Some customers are on a **pro** plan, which reduces churn too. Observation stops today, so recent joiners have short, censored histories.

```
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")
from statsmodels.duration.survfunc import SurvfuncRight, survdiff
from statsmodels.duration.hazard_regression import PHReg

rng = np.random.default_rng(16)
n = 6_000
intent = rng.normal(0, 1, n)                                          # unobserved
activated = (rng.random(n) < 1 / (1 + np.exp(-(-0.2 + 1.4 * intent)))).astype(int)
pro = (rng.random(n) < 0.3).astype(int)
joined = rng.integers(0, 24, n)                                       # month of sign-
up

TRUE_HR = {"activated": np.exp(-0.3), "pro": np.exp(-0.5)}
hazard = 0.05 * np.exp(-0.5 * intent - 0.3 * activated - 0.5 * pro)  # monthly hazard
lifetime = rng.exponential(1 / hazard)                                # months until
churn
observed_for = 24 - joined
time = np.minimum(lifetime, observed_for)
churned = (lifetime <= observed_for).astype(int)
df = pd.DataFrame({"time": time, "churned": churned, "activated": activated, "pro":
pro})

km = SurvfuncRight(df["time"], df["churned"])
print(f"Share of customers who have churned so far: {churned.mean():.0%}")
```

```
## Share of customers who have churned so far: 38%
```

```
print(f"Average lifetime of churned customers:      {time[churned == 1].mean():.1f}
months")
```

```
## Average lifetime of churned customers:      6.5 months
```

```
print(f"Kaplan–Meier median lifetime:               {km.quantile(0.5):.1f} months")
```

```
## Kaplan–Meier median lifetime:               17.7 months
```

```
print(f"True median lifetime (unobservable):        {np.median(lifetime):.1f} months")
```

```
## True median lifetime (unobservable):        17.2 months
```

Both naive numbers are wrong. “Share churned so far” depends on how recently customers joined, not just on how long they stay. “Average lifetime of churned customers” only counts those who already left, who are by construction the short-lived ones; it understates lifetime

by more than half. The **Kaplan-Meier** estimator uses every customer for as long as they were observed and recovers the true median.

16.3 Comparing groups

Kaplan-Meier curves by group are the first thing to look at. Here, activated versus not activated:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 2.8))
for a, label in [(0, "Not activated"), (1, "Activated")]:
    s = SurvfuncRight(df.loc[df.activated == a, "time"], df.loc[df.activated == a,
"churned"])
    ax.step(s.surv_times, s.surv_prob, where="post", lw=2, label=label)
ax.set_xlabel("Months since sign-up")
ax.set_ylabel("Share still active")
ax.set_ylim(0, 1)
ax.legend()
plt.tight_layout()
plt.show()
```

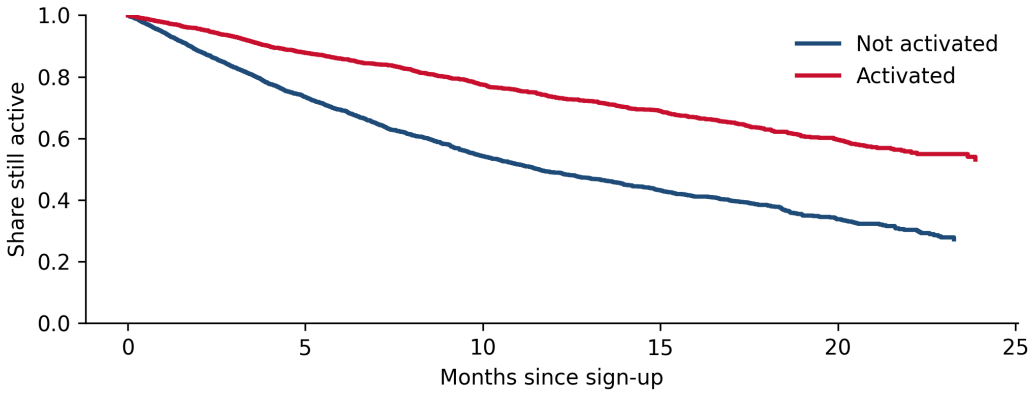


Figure 16.1: Kaplan-Meier survival curves by activation status.

```
stat, p = survdiff(df["time"], df["churned"], df["activated"])
print(f"Log-rank test: chi2 = {stat:.0f}, p {'< 0.001' if p < 0.001 else f'=
{p:.3f}'}")
```

```
## Log-rank test: chi2 = 339, p < 0.001
```

The gap is large and unmistakably real. The question that matters for the business is a different one: if we get more customers to activate, by how much will churn fall?

16.4 Cox regression, and why its hazard ratios are not effects

The **Cox proportional hazards** model relates the hazard to customer characteristics. Each coefficient becomes a hazard ratio: a hazard ratio of 0.7 means the risk of leaving at any given time is 30% lower.

```
def hazard_ratios(cols):
    fit = PHReg(df["time"], df[cols], status=df["churned"]).fit()
    ci = np.exp(fit.conf_int())
    return pd.DataFrame({"HR": np.exp(fit.params), "95% low": ci[:, 0], "95% high":
ci[:, 1]}, index=cols)

hr = hazard_ratios(["activated", "pro"])
hr["true causal HR"] = [TRUE_HR["activated"], TRUE_HR["pro"]]
print(hr.round(2).to_string())
```

```
##              HR  95% low  95% high  true causal HR
## activated  0.44     0.41      0.48            0.74
## pro        0.68     0.61      0.75            0.61
```

```
df["intent"] = intent                       # only possible in a simulation
print("\nWith the confounder included:")
```

```
##
## With the confounder included:
```

```
print(hazard_ratios(["activated", "pro", "intent"]).round(2).to_string())
```

```
##              HR  95% low  95% high
## activated  0.71     0.64      0.78
## pro        0.67     0.61      0.73
## intent     0.61     0.58      0.64
```

The Cox model says activation cuts the hazard by more than half, with a tight interval. The true causal effect is a reduction of about a quarter. The difference is confounding, in exactly the sense of Chapter 3: customers who need the product more both activate more and stay longer, and the model credits activation with part of the effect of need. When the confounder is included (possible here only because we simulated it), the estimate falls back to the truth. The pro plan estimate is close to the truth, because plan choice was independent of intent in this simulation; in real data it rarely is. (It is slightly attenuated even so: omitting a strong predictor pulls Cox hazard ratios toward 1 even when it is unrelated to the feature, a property known as non-collapsibility.)

This matters because activation is a lever. A team that reads the observational hazard ratio will forecast a churn reduction twice as large as an onboarding improvement can deliver. The way to measure the causal effect of activation is to randomize something that changes activation (an onboarding flow, a nudge sequence, a setup call) and measure retention by cohort over a horizon long enough for churn to show (Chapter 5).

> **Pitfall.**
> Reading hazard ratios from a churn model as the effect of changing that feature. Features such as activation, product usage, number of integrations or support contacts are partly symptoms of customer need. Their association with retention overstates what pushing them will achieve.

16.5 Predicting churn versus preventing it

Churn prediction models (logistic regression, gradient boosting, survival models) answer "who is likely to leave?". Retention budgets need a different answer: "whose decision to leave will our intervention change?". Those are not the same people.

- **Lost causes**: very high risk, will leave whatever you do.
- **Sure things**: low risk, will stay whatever you do.
- **Persuadables**: the customers whose behavior the offer changes, often at moderate risk.

Targeting the highest predicted risk spends most of the budget on lost causes. Targeting by **uplift**, the estimated difference the intervention makes for each customer, requires an experiment: randomly give the offer to some customers and not others, then model the difference.

16.5.1 Worked example: risk targeting versus uplift targeting

20,000 customers are in a randomized retention test: half receive an offer. Each customer's 90-day churn risk depends on three features, and the offer's effect is largest for customers at moderate risk. The budget allows the offer to go to 20% of the base in future. We compare four ways of choosing that 20%.

```
from sklearn.linear_model import LogisticRegression

N = 20_000
X = np.column_stack([rng.normal(0, 1, N), rng.normal(0, 1, N), rng.poisson(2, N)])
p0 = 1 / (1 + np.exp(-(-1.7 + 1.1 * X[:, 0] - 0.8 * X[:, 1] + 0.35 * X[:, 2])))  #
churn risk, no offer
tau = 0.5 * p0 * (1 - p0)                                  # true churn reduction from
the offer
offer = rng.random(N) < 0.5                                # randomized test
churn = rng.random(N) < np.where(offer, p0 - tau, p0)

F = np.column_stack([X, X ** 2])
m_control = LogisticRegression(max_iter=2000).fit(F[~offer], churn[~offer])
m_offer = LogisticRegression(max_iter=2000).fit(F[offer], churn[offer])
risk = m_control.predict_proba(F)[:, 1]
uplift = risk - m_offer.predict_proba(F)[:, 1]             # two-model ("T-learner")
uplift estimate

k = int(0.2 * N)
rows = []
for name, score in [("Random 20%", rng.random(N)), ("Highest predicted risk", risk),
                    ("Highest predicted uplift", uplift), ("Oracle (true uplift)",
tau)]:
    top = np.argsort(-score)[:k]
    rows.append({"targeting": name, "customers saved": round(tau[top].sum()),
                 "avg risk of targeted": round(p0[top].mean(), 2)})
print(f"Average 90-day churn risk without offer: {p0.mean():.0%}")
```

```
## Average 90-day churn risk without offer: 33%
```

```
print(pd.DataFrame(rows).set_index("targeting").to_string())
```

```
##                          customers saved  avg risk of targeted
## targeting
## Random 20%                           323                  0.33
## Highest predicted risk               383                  0.72
## Highest predicted uplift             480                  0.48
## Oracle (true uplift)                 491                  0.49
```

Targeting the highest-risk customers beats random selection, but only modestly: it concentrates the budget on people whose average risk is above 70%, many of whom leave regardless. Targeting by estimated uplift saves about a quarter more customers with the same budget, by aiming at customers of moderate risk, and comes close to the oracle that knows each customer's true response. In businesses where the highest-risk customers are truly lost causes, risk targeting can do worse than random. The uplift model needed nothing

exotic: two logistic regressions fitted on the randomized test. What it did need was the randomized test.

16.6 Practical notes

- **Define churn precisely.** For subscriptions, cancellation or non-renewal. For usage-based products, a period of inactivity (for example, no transactions for 90 days), chosen so that few customers who pass it ever come back.
- **Check proportional hazards.** Cox models assume each feature's effect is constant over time. Activation, for instance, often matters a lot early and little later; plotting scaled Schoenfeld residuals or fitting separate models for early and late tenure reveals this.
- **Separate voluntary and involuntary churn.** Failed payments and compliance closures have different drivers and different fixes from customers choosing to leave.
- **Measure retention interventions with holdouts and long horizons.** A retention offer that delays churn by a month looks like a success at 30 days and a cost at 120.

16.7 Checklist

Checklist

- Are churn and lifetime estimated with methods that handle censoring (Kaplan-Meier, Cox, parametric survival), not averages over churned customers?
- Is the churn definition explicit and stable, with voluntary and involuntary churn separated?
- Are hazard ratios presented as associations unless the feature was randomized?
- Are retention offers targeted by uplift from a randomized test, rather than by predicted risk?
- Is the effect of retention interventions read at a horizon long enough to distinguish prevention from delay?

16.8 Further reading

- *Kaplan-Meier survival analysis in Python*.
- *CausalML* (Uber) and *EconML* (py-why) for uplift and heterogeneous treatment effect models.

Full links are in Appendix A.

Chapter 17 Segmentation

> **The decision this chapter supports.**
> How to group customers so that the groups lead to different, measurable actions (different messages, offers, channels or service levels), and how to tell a useful segmentation from a merely tidy one.

17.1 What a segmentation is for

A segmentation is useful when three things hold: customers in different segments **respond differently** to something you can do, the segments are **stable** enough to act on for months rather than days, and you can **identify** each customer's segment from data you actually have at the moment of acting. A segmentation that fails any of these is a description, not a tool.

Two broad approaches exist, and most good segmentations combine them.

- **Managerial (rule-based) segments** are defined by business logic: plan, tenure band, size, activity level, RFM scores. They are transparent, stable and easy to operationalize.
- **Statistical segments** come from clustering on behavioral or attitudinal data. They can find structure nobody thought to look for, but they are sensitive to choices that look technical and are not.

17.2 Worked example: a customer base with four true segments

We simulate 5,000 business customers drawn from four groups with different behavior: many small, low-volume **micro** businesses; **growing** businesses with more transactions and products; a smaller group of **established** high-volume customers; and **dormant** customers who have stopped transacting. In real data the true groups are unknown; here they let us check what each method recovers.

```
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score, silhouette_score

rng = np.random.default_rng(17)
specs = {   # n, transactions/month, value per transaction (EUR), tenure (months),
products
    "micro": (2000, 8, 60, 6, 1.3), "growing": (1500, 40, 250, 14, 2.5),
    "established": (700, 120, 900, 40, 3.8), "dormant": (800, 1, 150, 24, 1.5)}
parts = []
for seg, (n, tx, val, ten, prod) in specs.items():
    t = rng.poisson(tx * rng.lognormal(0, 0.35, n))
    parts.append(pd.DataFrame({
        "true_segment": seg, "tx_month": t, "volume_month": t * val * rng.lognormal(0,
0.45, n),
        "tenure": np.clip(rng.normal(ten, ten * 0.35, n), 1, None).round(),
        "products": np.clip(rng.poisson(prod, n), 1, 6),
        "days_since_tx": np.where(seg == "dormant", rng.integers(40, 180, n),
rng.integers(0, 20, n))}))
cust = pd.concat(parts, ignore_index=True)
FEATS = ["tx_month", "volume_month", "tenure", "products", "days_since_tx"]
print(cust.groupby("true_segment")[FEATS].median().round(0).to_string())
```

```
##               tx_month  volume_month  tenure  products  days_since_tx
## true_segment
## dormant            1.0         124.0    24.0       1.0          110.0
## established      120.0      110951.0    39.0       4.0            9.0
## growing           39.0        9988.0    14.0       2.0            9.0
## micro              8.0         457.0     6.0       1.0            9.0
```

17.3 RFM: the managerial baseline

RFM scores each customer on **recency** (time since last activity), **frequency** (how often they transact) and **monetary** value, usually in quintiles, and combines the scores into named groups. It takes an hour to build, everyone understands it, and it maps directly onto actions: reactivate the lapsed, protect the valuable, grow the frequent-but-small.

```
q = lambda s, asc=True: pd.qcut(s.rank(method="first", ascending=asc), 5,
labels=False) + 1
cust["R"] = q(cust["days_since_tx"], asc=False)          # 5 = most recent
cust["F"] = q(cust["tx_month"])
cust["M"] = q(cust["volume_month"])

def rfm_group(r):
    if r.R <= 2 and r.F <= 2: return "Lapsed"
    if r.M == 5 and r.F >= 4: return "Top value"
    if r.F >= 3: return "Active core"
    return "Low activity"
cust["rfm"] = cust.apply(rfm_group, axis=1)
print(pd.crosstab(cust["rfm"], cust["true_segment"]).to_string())
```

```
## true_segment  dormant  established  growing  micro
## rfm
## Active core         0            0     1204    800
## Lapsed            800            0        0    352
## Low activity        0            0        0    848
## Top value           0          700      296      0
```

RFM catches every dormant customer and puts all established ones in its top-value group. Its weaknesses are typical. The lapsed group also contains a few hundred micro businesses that are merely infrequent, because quintile cut-offs are relative: with many low-frequency customers, the bottom quintiles contain active ones too. Fixed business thresholds ("no transaction in 30 days") often work better than quintiles for recency. And it mixes micro and growing businesses, which overlap on transaction-based scores. For many decisions (who to win back, who gets an account manager) that is good enough, and its simplicity is worth a lot.

17.4 Clustering, and the choice that decides everything

K-means is the default clustering method: it groups customers so that each is close to its group's center. "Close" is measured in the units of the features, and that is where most segmentations go wrong.

```
raw = KMeans(4, n_init=10, random_state=0).fit_predict(cust[FEATS])
Z = StandardScaler().fit_transform(np.log1p(cust[FEATS]))         # log, then
standardize
scaled = KMeans(4, n_init=10, random_state=0).fit_predict(Z)

print(f"Agreement with true segments (adjusted Rand index; 1 = perfect, 0 = chance):")
```

```
## Agreement with true segments (adjusted Rand index; 1 = perfect, 0 = chance):
```

```
print(f"  raw features:                {adjusted_rand_score(cust.true_segment,
raw):.2f}")
```

```
##   raw features:                0.19
```

```
print(f"  log-transformed, standardized: {adjusted_rand_score(cust.true_segment,
scaled):.2f}")
```

```
##   log-transformed, standardized: 0.81
```

```
print("\nRaw-feature clusters versus truth:")
```

```
##
## Raw-feature clusters versus truth:
```

```
print(pd.crosstab(cust["true_segment"], raw).to_string())
```

```
## col_0            0    1   2    3
## true_segment
## dormant        800    0   0    0
## established     50  381  59  210
## growing       1489   11   0    0
## micro         2000    0   0    0
```

On raw features, monthly volume (in thousands of euros) dwarfs everything else (products, measured from 1 to 6), so the clustering is really just a split of the established customers by volume, with the other three groups lumped together. After a log transform (volumes are heavily skewed) and standardization, the clusters line up well with the true segments. Feature scaling is not a preprocessing detail; it *is* the definition of similarity, and it should be chosen deliberately: which differences between customers should count as large?

> **Pitfall.**
> Clustering on raw, unscaled features, or on features with heavy skew. The feature with the largest numeric range decides the segmentation, and the result looks like a discovery.

17.5 How many segments?

Statistical criteria help rule out bad choices, but they do not pick the right number. Two useful diagnostics are the **silhouette score** (how well separated the clusters are) and **stability** (whether fitting on two different bootstrap samples of the data gives the same segments).

```
rows = []
for k in range(2, 8):
    labels = KMeans(k, n_init=10, random_state=0).fit_predict(Z)
    stab = []
    for b in range(10):
        a1 = KMeans(k, n_init=5, random_state=b).fit(Z[rng.integers(0, len(Z),
len(Z))]).predict(Z)
        a2 = KMeans(k, n_init=5, random_state=50 + b).fit(Z[rng.integers(0, len(Z),
len(Z))]).predict(Z)
        stab.append(adjusted_rand_score(a1, a2))
    rows.append({"k": k, "silhouette": silhouette_score(Z, labels, sample_size=2000,
random_state=0),
                 "bootstrap stability": np.mean(stab),
                 "agreement with truth": adjusted_rand_score(cust.true_segment,
labels)})
print(pd.DataFrame(rows).set_index("k").round(2).to_string())
```

```
##    silhouette  bootstrap stability  agreement with truth
## k
## 2        0.35                 0.99                  0.52
## 3        0.41                 0.98                  0.68
## 4        0.34                 0.95                  0.81
## 5        0.35                 0.94                  0.79
## 6        0.33                 0.91                  0.62
## 7        0.33                 0.82                  0.58
```

The silhouette score prefers three clusters, not the true four, and stability is high for anything from two to five. This is typical. Different numbers of segments are all statistically defensible, and the choice between them is a business decision: how many distinct treatments can the organization actually run, and does splitting a segment lead to a different action? Stability is still worth checking, because a segmentation that changes when the data is resampled will change from one quarter to the next.

17.6 Mixed data

Real customer data mixes numeric features with categorical ones (sector, legal form, country, acquisition channel). Euclidean distance does not handle categories well; one-hot encoding lets a single categorical feature with many levels dominate. **Gower distance** combines a scaled difference for numeric features with a simple match/mismatch for categorical ones, and pairs naturally with hierarchical clustering or k-medoids. For large bases, cluster a sample and assign the rest to the nearest medoid.

17.7 From segments to actions

The test of a segmentation is whether it changes what you do. Three habits keep it honest.

- **Profile segments on what you did not cluster on**: lifetime value, channel mix, support usage, response to past campaigns. Segments that differ only on their input features add little.
- **Test segment-specific treatments.** If a segment is supposed to respond to a different message, run the experiment that shows it does. Chapter 16 showed that predicted risk is a poor guide to who responds; the same holds for segments.
- **Monitor drift.** Customers move between segments as they grow, lapse or change products. Track the flows, because the movements (micro to growing, growing to dormant) are often more actionable than the segments themselves.

17.8 Checklist

> **Checklist**
>
> - Does each segment lead to a different, feasible action, and can a customer's segment be identified when the action happens?
> - Has a simple managerial segmentation (such as RFM) been tried as a baseline?
> - Were features transformed and scaled deliberately, with the choice of what counts as "different" written down?
> - Is the number of segments justified by actionability, with stability checked rather than silhouette alone?
> - Are categorical features handled with an appropriate distance (Gower) rather than raw one-hot encoding?
> - Are segments profiled on outcomes that were not clustering inputs, and are segment-specific treatments tested?

17.9 Further reading

- *Gower distance* (Rui Vieira).

Full links are in Appendix A.

Chapter 18 Brand measurement

> **The decision this chapter supports.**
> Whether brand-building activity is working, which leading indicators to track while its slower effects build up, and what growth through brand actually looks like in customer data.

18.1 The measurement problem with brand

Most of the methods in Parts II and III measure effects that appear within days or weeks. Brand activity is meant to work over months and years: making the brand easier to think of and easier to choose when the need arises. Short-window experiments and standard MMMs mostly miss that, and teams that judge brand spend on short-window performance metrics tend to cut it.

Brand measurement therefore relies on three complementary kinds of evidence:

Evidence	Examples	Strength	Weakness
Attitudinal surveys	Prompted and unprompted awareness, consideration, brand lift studies	Directly measures mental availability	Stated, not behavioral; small samples; slow and costly to track
Behavioral leading indicators	Branded search volume, share of search, direct traffic, brand-term click share	Cheap, frequent, behavioral	Contaminated by own paid search, PR, seasonality; correlation only
Long-window causal tests	Geo tests with long post-periods on brand and business outcomes; MMM with long-term components	Causal	Expensive, slow, low power

None of these alone settles the question. Together, and tracked consistently over time, they show whether brand investment is building the base that performance marketing then converts.

18.2 Share of search

Share of search is a brand's share of all branded searches in its category: searches for your brand name divided by searches for your brand plus your main competitors. Work published

through the IPA found that share of search tends to lead market share, often by several months, across a range of categories. It has become popular because it is cheap, available weekly, and comparable across markets.

Two technical points matter more than they seem.

- **Use the share, not the volume.** Raw branded search volume rises and falls with category demand and seasonality. Dividing by the category total removes most of that.
- **Correlate changes, not levels.** Two trending series correlate strongly at every lag, which says nothing about which leads which.

18.2.1 Worked example: finding a lead that is really there

We simulate six years of monthly data in which a brand's share of search grows after a campaign and market share follows six months later.

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(18)
M, LEAD = 72, 6
t = np.arange(M)
sos = 0.20 + 0.08 / (1 + np.exp(-(t - 30) / 5)) + np.cumsum(rng.normal(0, 0.004, M))
market_share = np.r_[np.full(LEAD, 0.18), 0.9 * sos[:-LEAD]] + rng.normal(0, 0.004, M)

def lagged_corr(x, y, k):
    return np.corrcoef(x[:len(x) - k], y[k:])[0, 1]

lags = range(13)
levels = [lagged_corr(sos, market_share, k) for k in lags]
changes = [lagged_corr(np.diff(sos), np.diff(market_share), k) for k in lags]
print(pd.DataFrame({"levels": levels, "monthly changes": changes},
                   index=pd.Index(lags, name="lead of SoS
(months)")).round(2).T.to_string())
```

```
## lead of SoS (months)    0     1     2     3     4     5     6     7     8     9
10    11    12
## levels                0.95  0.96  0.97  0.98  0.99  0.99  1.00  0.99  0.98  0.98
0.97  0.96  0.94
## monthly changes       0.01  0.12  0.21  0.07  0.13  0.32  0.49 -0.01  0.14  0.20
0.10  0.02  0.13
```

```
fig, ax = plt.subplots(figsize=(7, 2.7))
ax.plot(list(lags), levels, marker="o", label="Levels")
ax.plot(list(lags), changes, marker="o", label="Monthly changes")
ax.axvline(LEAD, ls=":", color="#6B6B6B")
ax.set_xlabel("Months by which share of search leads market share")
ax.set_ylabel("Correlation")
ax.legend()
plt.tight_layout()
plt.show()
```

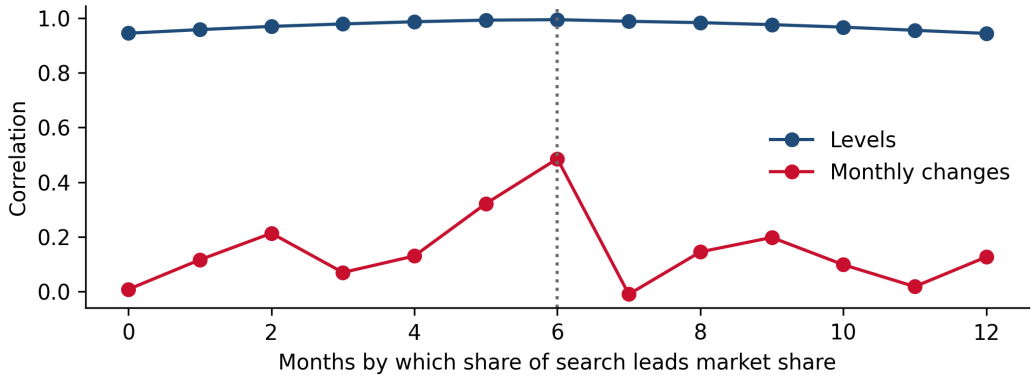


Figure 18.1: Cross-correlation between share of search and market share at different leads. Levels correlate highly at every lead; changes reveal the true six-month lead.

The level correlations sit above 0.9 at every lead: they would support almost any story. The correlations of monthly changes peak clearly at the true lead. Even then, the result is a correlation. It makes share of search a useful **leading indicator** for tracking, not proof that whatever raised it will raise market share.

> **Pitfall.**
> Reading your own branded search volume as a pure brand signal. It also rises with your paid search coverage, PR spikes, product incidents and seasonality, and it can be bought: bidding on your own brand terms changes how people search. Use share of search against competitors, and keep a change log of events that move it.

Practical notes. Search-trend tools return sampled, relative indices, which are noisy for small brands and change slightly between downloads; average several pulls or use a keyword-volume tool. Choose the competitor set once and keep it fixed. Track by country: share of search is only comparable within a market and language.

18.3 What brand growth looks like: double jeopardy

Ehrenberg-Bass research on buying behavior, built on the NBD-Dirichlet model, describes regularities that hold across categories and countries. The most useful for marketers is **double jeopardy**: smaller brands have far fewer buyers, and those buyers are also slightly less loyal. Brands differ mostly in how many people buy them (penetration), much less in how often their buyers buy.

The Dirichlet model generates this from two simple assumptions: each customer buys the category at their own steady rate (a negative binomial distribution of purchase counts), and chooses among brands with their own steady propensities, which vary across customers according to a Dirichlet distribution centered on market shares.

```
n_customers = 20_000
shares = np.array([0.35, 0.25, 0.18, 0.12, 0.10])
rate = rng.gamma(1.5, 4 / 1.5, n_customers)                # category purchases per
year (mean 4)
purchases = rng.poisson(rate)
propensity = rng.dirichlet(6 * shares, size=n_customers)    # each customer's brand
preferences
buys = np.array([rng.multinomial(k, p) for k, p in zip(purchases, propensity)])

buyers = buys > 0
tbl = pd.DataFrame({
    "market share": buys.sum(0) / buys.sum(),
    "penetration": buyers.mean(0),
    "purchases per buyer": buys.sum(0) / buyers.sum(0),
    "share of category (buyers)": [(buys[buyers[:, j], j] / buys[buyers[:,
j]].sum(1)).mean() for j in range(5)],
}, index=[f"Brand {c}" for c in "ABCDE"])
print(tbl.round(2).to_string())
```

```
##          market share  penetration  purchases per buyer  share of category (buyers)
## Brand A          0.35         0.58                 2.42                        0.52
## Brand B          0.25         0.47                 2.09                        0.45
## Brand C          0.18         0.38                 1.86                        0.41
## Brand D          0.12         0.28                 1.71                        0.37
## Brand E          0.10         0.24                 1.65                        0.36
```

Brand A has three and a half times brand E's market share. Most of that gap is penetration: A reaches well over twice as many buyers as E. Purchase frequency and loyalty (share of category requirements among the brand's own buyers) differ much less. This is double jeopardy, and it holds in real panel data far beyond this simulation.

The practical implication is the main reason brand measurement matters. Growth comes mainly from reaching more category buyers, including light and occasional ones, not from making existing customers more loyal. A brand that grows should see **penetration and reach-related metrics** move first: awareness among category buyers, share of search, new-

customer counts. A loyalty program that raises purchase frequency without adding buyers usually moves market share very little.

18.4 Measuring brand effects causally

The only fully causal evidence for brand effects comes from experiments with long enough windows. Two designs work in practice.

- **Long geo tests**: run brand activity (for example TV or video) in some regions and not others for a quarter or more, and track branded search, direct traffic and business outcomes over the test and a long cooldown. Effects on branded search show up quickly; effects on acquisition take longer.
- **Halo checks in performance tests**: when testing a performance channel, also measure branded search, direct and organic traffic in test and control. A channel that looks weak on its own conversions may be lifting the others.

MMMs can include long-term components (slowly decaying adstock on brand activity, or a time-varying base that responds to brand spend), but these are among the hardest things for an MMM to identify. Treat them as hypotheses to test, not results.

18.5 Checklist

Checklist

- Is brand health tracked with a fixed set of metrics and a fixed competitor set, by market, over time?
- Is share of search used rather than raw branded search volume, and are changes rather than levels correlated?
- Is there a change log for events that move branded search (paid brand bidding, PR, incidents)?
- Are penetration and new-customer metrics tracked alongside loyalty metrics?
- Do performance experiments also record halo outcomes (branded search, direct, organic)?
- Are brand effects judged on windows long enough for them to appear?

18.6 Further reading

- *Share of search reporting* (We Discover)
- *Google's Brand Measurement Playbook*.

  Full links are in Appendix A.

Chapter 19 Pricing research and elasticities

> **The decision this chapter supports.**
> What to charge, which features customers will pay for, and whether a price change will make or lose money, using methods that range from a quick survey to a live price test.

19.1 A map of pricing methods

Pricing methods differ in what they ask, how close they come to real buying behavior, and how much they cost.

Method	What it measures	Closeness to behavior	Typical use
Van Westendorp price sensitivity meter	Ranges of prices customers find acceptable	Low (stated perceptions)	Early-stage range finding
Gabor-Granger	Stated purchase intent at a set of prices	Low to medium	Demand curve for a single product
Choice-based conjoint	Trade-offs between price and features in simulated choices	Medium	Packaging, plan design, willingness to pay per feature
Price tests (A/B, geo, new customers only)	Real conversion and revenue at different prices	High	Validating a specific change
Observational elasticity from history	Association between past prices and volume	Medium, but confounded	Context and priors only

Surveys are cheap and fast and systematically overstate willingness to buy. Live tests are accurate and constrained by fairness, regulation and customer trust. In practice, surveys narrow the options and tests decide between the final few.

19.2 Van Westendorp: finding the acceptable range

The price sensitivity meter asks four questions: at what price would the product be **so cheap** you would doubt its quality, **cheap** (a bargain), **expensive** but still worth considering, and **too expensive** to consider? Plotting the cumulative distributions of the answers gives four curves whose intersections define a range.

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(19)
n = 600
reference = rng.lognormal(np.log(20), 0.35, n)          # each respondent's sense of a fair monthly price
answers = {"too cheap": reference * rng.uniform(.35, .55, n), "cheap": reference * rng.uniform(.65, .85, n),
           "expensive": reference * rng.uniform(1.15, 1.4, n), "too expensive": reference * rng.uniform(1.6, 2.2, n)}
grid = np.linspace(3, 60, 1000)
share_at_or_above = lambda x: (x[:, None] >= grid).mean(0)    # "too cheap" and "cheap" fall as price rises
share_at_or_below = lambda x: (x[:, None] <= grid).mean(0)    # "expensive" and "too expensive" rise
tc_curve, c_curve = share_at_or_above(answers["too cheap"]), share_at_or_above(answers["cheap"])
e_curve, te_curve = share_at_or_below(answers["expensive"]), share_at_or_below(answers["too expensive"])
cross = lambda a, b: grid[np.argmin(np.abs(a - b))]
points = {"Point of marginal cheapness (too cheap = expensive)": cross(tc_curve, e_curve),
          "Optimal price point (too cheap = too expensive)": cross(tc_curve, te_curve),
          "Indifference price point (cheap = expensive)": cross(c_curve, e_curve),
          "Point of marginal expensiveness (cheap = too expensive)": cross(c_curve, te_curve)}
for k, v in points.items():
    print(f"{k:58s} EUR {v:5.2f}")
```

```
## Point of marginal cheapness (too cheap = expensive)        EUR 15.32
## Optimal price point (too cheap = too expensive)            EUR 18.41
## Indifference price point (cheap = expensive)               EUR 19.38
## Point of marginal expensiveness (cheap = too expensive)    EUR 23.08
```

```
fig, ax = plt.subplots(figsize=(7, 2.8))
for y, lab, ls in [(tc_curve, "Too cheap", "--"), (c_curve, "Cheap", "-"), (e_curve, "Expensive", "-"), (te_curve, "Too expensive", "--")]:
    ax.plot(grid, y, ls=ls, lw=1.8, label=lab)
lo, hi = points["Point of marginal cheapness (too cheap = expensive)"], points["Point of marginal expensiveness (cheap = too expensive)"]
ax.axvspan(lo, hi, color="#6B6B6B", alpha=0.1)
ax.set_xlim(5, 45)
ax.set_xlabel("Monthly price (EUR)")
ax.set_ylabel("Share of respondents")
ax.legend(fontsize=8, ncol=2)
plt.tight_layout()
plt.show()
```

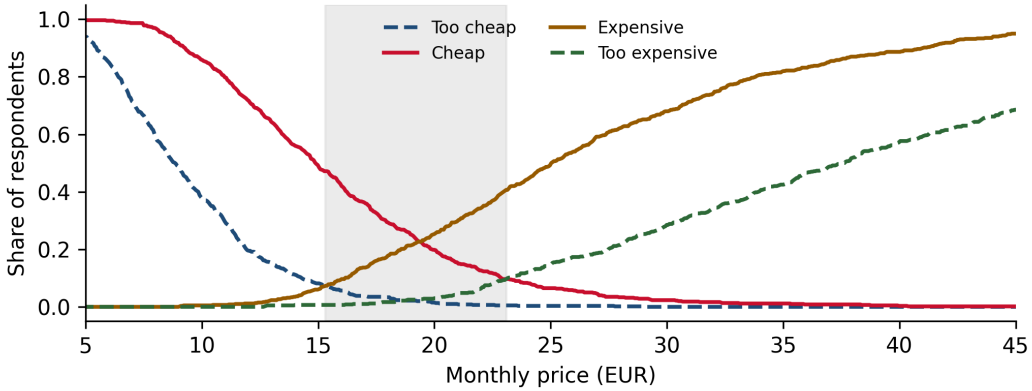


Figure 19.1: Van Westendorp curves. The acceptable range runs from the point of marginal cheapness to the point of marginal expensiveness.

The method is quick and gives a sensible range, here roughly 15 to 23 EUR a month. It does not tell you how many people will actually buy at each price, and it tends to anchor on whatever respondents already pay for similar products. Use it to rule out prices, not to pick one.

Gabor-Granger goes a step further: ask whether respondents would buy at a given price, moving the price up or down depending on the answer, and build a demand curve from the responses. Multiplying demand by price (or margin) gives a revenue- or profit-maximizing price. Stated intent overstates real purchase rates, often by a large factor, so the curve's shape is more trustworthy than its level; calibrating against a known conversion rate at the current price helps.

19.3 Choice-based conjoint: what features are worth

Conjoint analysis asks respondents to choose between product configurations that vary in price and features, several times each. Because every choice is a trade-off, the analysis can separate how much each feature adds to the appeal of a product from how much price subtracts, and so express each feature as a **willingness to pay**.

We simulate 400 respondents each making 10 choices among three plans, varying monthly price (9, 15, 25 or 35 EUR) and three features: invoicing tools, priority support and extra cards. The model is a conditional (multinomial) logit, where each alternative's utility is linear in its attributes and the probability of choosing it follows the logit formula.

```
import warnings; warnings.filterwarnings("ignore")
from statsmodels.discrete.conditional_models import ConditionalLogit

TRUE = {"price": -0.08, "invoices": 0.60, "support": 0.45, "cards": 0.30}
R, TASKS, ALTS, prices = 400, 10, 3, np.array([9, 15, 25, 35])
rows, task_id = [], 0
price_sens = rng.normal(TRUE["price"], 0.02, R)          # price sensitivity varies by
respondent
for r in range(R):
    for _ in range(TASKS):
        X = np.column_stack([rng.choice(prices, ALTS), rng.integers(0, 2, (ALTS, 3))])
        utility = price_sens[r] * X[:, 0] + X[:, 1:] @ np.array([TRUE["invoices"],
TRUE["support"], TRUE["cards"]])
        choice = np.argmax(utility + rng.gumbel(size=ALTS))
        rows += [[task_id, *X[a], int(a == choice)] for a in range(ALTS)]
        task_id += 1
cbc = pd.DataFrame(rows, columns=["task", "price", "invoices", "support", "cards",
"chosen"])

fit = ConditionalLogit(cbc["chosen"], cbc[["price", "invoices", "support", "cards"]],
groups=cbc["task"]).fit()
draws = rng.multivariate_normal(fit.params, fit.cov_params(), 5000)       # parameter
uncertainty
wtp = -draws[:, 1:] / draws[:, [0]]
out = pd.DataFrame({"WTP (EUR/month)": wtp.mean(0), "95% low": np.quantile(wtp, .025,
0),
                    "95% high": np.quantile(wtp, .975, 0),
                    "true WTP": [-TRUE[f] / TRUE["price"] for f in ["invoices",
"support", "cards"]]},
                   index=["invoices", "support", "cards"])
print(out.round(2).to_string())
```

```
##           WTP (EUR/month)  95% low  95% high  true WTP
## invoices             7.56     6.45      8.67      7.50
## support              4.14     3.10      5.20      5.62
## cards                3.18     2.13      4.24      3.75
```

Invoicing and extra cards are estimated close to their true values. Priority support is underestimated, and its interval misses the truth. The cause is a simplification in the model, not bad luck: it assumes every respondent shares one price sensitivity, while in the simulation (as in reality) price sensitivity varies from person to person, and it treats each respondent's ten choices as independent observations, which makes the intervals too narrow. This is why commercial conjoint studies use **hierarchical Bayes** or **mixed logit** models, which estimate a distribution of preferences across respondents and account for repeated choices. The simple model is fine for a first look; use the hierarchical version for decisions.

With the estimated utilities you can also run a **market simulator**: predict the share of choices a set of plans would get, against each other and against a competitor's offer, and test packaging options (which features go in which tier, at what prices) before building anything.

```
b = fit.params
def shares(plans):
    u = np.array([b["price"] * p + b["invoices"] * i + b["support"] * s + b["cards"] *
c for p, i, s, c in plans])
    e = np.exp(u - u.max())
    return e / e.sum()

competitor = (15, 1, 0, 0)
for name, ours in {"Basic 9 EUR, no features": (9, 0, 0, 0),
                   "Plus 19 EUR, invoices + cards": (19, 1, 0, 1),
                   "Pro 29 EUR, all features": (29, 1, 1, 1)}.items():
    print(f"{name:32s} share vs competitor: {shares([ours, competitor])[0]:.0%}")
```

```
## Basic 9 EUR, no features         share vs competitor: 47%
## Plus 19 EUR, invoices + cards    share vs competitor: 48%
## Pro 29 EUR, all features         share vs competitor: 37%
```

Two cautions. Conjoint respondents make hypothetical choices with no money at stake, so absolute price sensitivity tends to be understated; trust relative feature values more than the overall price level. And the design matters: unrealistic combinations, too many attributes, or prices far outside the market range all degrade the results.

19.4 Will a price change make money?

Before any test, a simple calculation shows how much volume a price change can afford to lose (or must gain). With contribution margin m (as a share of price) and a relative price change Δp, profit is unchanged if volume changes by

$$\Delta q_{\text{break-even}} = \frac{-\Delta p}{m + \Delta p}.$$

```
rows = []
for m in [0.3, 0.6, 0.8]:
    for dp in [-0.10, +0.10]:
        dq = -dp / (m + dp)
        rows.append({"margin": f"{m:.0%}", "price change": f"{dp:+.0%}",
                     "break-even volume change": f"{dq:+.1%}", "break-even
elasticity": round(dq / dp, 2)})
print(pd.DataFrame(rows).to_string(index=False))
```

```
## margin price change break-even volume change  break-even elasticity
##    30%         −10%                   +50.0%                  −5.00
##    30%         +10%                   −25.0%                  −2.50
##    60%         −10%                   +20.0%                  −2.00
##    60%         +10%                   −14.3%                  −1.43
##    80%         −10%                   +14.3%                  −1.43
##    80%         +10%                   −11.1%                  −1.11
```

At a 30% margin, a 10% price cut needs a 50% volume increase just to break even, which almost never happens. At an 80% margin, typical of software and many financial products, a 10% price increase breaks even if volume falls by no more than about 11%. Comparing these break-even elasticities with what you know about your customers' sensitivity is often enough to rule a proposed change in or out.

19.5 Elasticities from data, and price tests

Observational elasticities are confounded. Regressing volume on past prices runs into the problems of Chapter 3: prices are often changed in response to demand (discounts when demand is weak, increases when it is strong), promotions coincide with campaigns, and competitors move at the same time. Log-log regressions on historical data are useful context, not decision-grade estimates.

Price tests give the real answer, with constraints that matter more than in other experiments:

- **Fairness and regulation.** Showing different prices to similar customers at the same time can breach consumer protection rules and damages trust when discovered. Common alternatives are testing on new customers only, testing by market or region, testing over time with a clear before-and-after design, or testing packaging and plan structure rather than the price of an identical product.
- **Long outcome windows.** A higher price may convert slightly fewer customers and retain them equally well, or convert the same number and churn more later. Read price tests on revenue or margin per acquired customer over a cohort horizon (Chapter 15), not on conversion alone.
- **Mix effects.** A price change on one plan shifts customers between plans. Measure total revenue per customer across the lineup, not only the tested plan.

19.6 Checklist

Checklist

- Is the method matched to the decision: surveys for ranges and feature values, tests for final price changes?
- Are stated-intent results treated as relative signals, calibrated where possible against real conversion?
- Has the break-even volume change been calculated for the proposed price change at the actual margin?
- Are historical elasticities treated as confounded context rather than estimates?
- Does any price test respect fairness and regulatory constraints, and is it read on margin per customer over a long enough horizon, including mix effects?

19.7 Further reading

- *Van Westendorp's price sensitivity meter* and *Gabor-Granger method* (Wikipedia), the `pricesensitivitymeter` R package, and the Van Westendorp Python implementation in the library.

Full links are in Appendix A.

Chapter 20 Forecasting and anomaly detection

> **The decision this chapter supports.**
> What to expect next week, month and quarter (for targets, pacing and budgets), and how to tell, day by day, whether a movement in a key metric is a real problem or ordinary noise.

20.1 Two jobs for the same model

Forecasting turns up in marketing science in two roles. **Planning forecasts** set targets and budgets: how many sign-ups to expect next quarter, how much spend that will take, whether the plan is on pace. **Monitoring** uses a forecast as a moving baseline: if today's number is far outside what the model expected for a day like today, something may be wrong. The same model serves both, and the quality bar is the same: honest uncertainty and an evaluation on data the model has not seen.

A third role, the counterfactual forecast that estimates what would have happened without an intervention, was covered in Chapter 7. The difference is the question asked, not the method.

20.2 The data

Three years of daily sign-ups with a gentle upward trend, a strong weekly pattern (weekends are quiet), a yearly cycle, and holiday periods where activity drops.

```
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")
import statsmodels.api as sm
from statsmodels.tsa.exponential_smoothing.ets import ETSModel
import matplotlib.pyplot as plt

rng = np.random.default_rng(20)
dates = pd.date_range("2023-01-01", "2025-12-31", freq="D")
t = np.arange(len(dates))

def is_holiday(idx):
    return (((idx.month == 12) & (idx.day >= 24)) | ((idx.month == 1) & (idx.day <=
1))
            | ((idx.month == 8) & idx.day.isin(range(10, 20))))

weekday_effect = np.array([0.05, 0.08, 0.06, 0.04, 0.0, -0.45, -0.55])
log_mu = (np.log(400) + 0.0004 * t + weekday_effect[dates.dayofweek]
          + 0.12 * np.sin(2 * np.pi * dates.dayofyear / 365.25) - 0.35 *
is_holiday(dates))
signups = pd.Series(rng.negative_binomial(150, 150 / (150 +
np.exp(log_mu))).astype(float), index=dates)
```

20.3 Three forecasters and a fair test

We compare three approaches, from simplest to most structured:

- **Seasonal naive**: next week looks like last week. Surprisingly hard to beat, and the right baseline for any forecast.
- **Exponential smoothing (ETS)** with a damped trend and weekly seasonality, fitted on log sign-ups.
- **Regression** on log sign-ups with a trend, day-of-week effects, yearly Fourier terms and a holiday indicator.

The test is a **rolling-origin backtest**: at a series of dates through the final year, fit each model on everything before that date and forecast the next 28 days. Accuracy is measured with MASE (mean absolute error scaled by the error of a seasonal naive forecast in training; below 1 means better than naive) and the **coverage** of the 90% prediction interval (it should contain about 90% of actual days).

```
def design(idx):
    X = pd.DataFrame({"trend": (idx - dates[0]).days / 365}, index=idx)
    for k in range(1, 7):
        X[f"dow{k}"] = (idx.dayofweek == k).astype(float)
    for k in (1, 2, 3):
        X[f"s{k}"] = np.sin(2 * np.pi * k * idx.dayofyear / 365.25)
        X[f"c{k}"] = np.cos(2 * np.pi * k * idx.dayofyear / 365.25)
    X["holiday"] = is_holiday(idx).astype(float)
    return sm.add_constant(X, has_constant="add")

H = 28
rows = []
for origin in pd.date_range("2025-01-05", "2025-11-30", freq="28D"):
    train = signups[:origin - pd.Timedelta(days=1)]
    test = signups[origin:origin + pd.Timedelta(days=H - 1)]
    scale = np.mean(np.abs(train.values[7:] - train.values[:-7]))

    naive = np.tile(train.values[-7:], H // 7 + 1)[:H]

    ets = ETSModel(np.log1p(train[-730:]).asfreq("D"), error="add", trend="add",
damped_trend=True,
                   seasonal="add", seasonal_periods=7).fit(disp=False)
    e = ets.get_prediction(start=test.index[0],
end=test.index[-1]).summary_frame(alpha=0.1)

    reg = sm.OLS(np.log1p(train), design(train.index)).fit()
    r = reg.get_prediction(design(test.index)).summary_frame(alpha=0.1)

    for name, f, lo, hi in [("Seasonal naive", naive, None, None),
                            ("ETS", np.expm1(e["mean"]), np.expm1(e["pi_lower"]),
np.expm1(e["pi_upper"])),
                            ("Regression", np.expm1(r["mean"]),
np.expm1(r["obs_ci_lower"]), np.expm1(r["obs_ci_upper"]))]:
        f = np.asarray(f)
        rows.append({"model": name, "MASE": np.mean(np.abs(test.values - f)) / scale,
                     "MAPE": np.mean(np.abs(test.values - f) / test.values),
                     "90% interval coverage": np.nan if lo is None else
                         np.mean((test.values >= np.asarray(lo)) & (test.values <=
np.asarray(hi)))})
bt = pd.DataFrame(rows).groupby("model").mean()
print(bt.round(2).to_string(na_rep="n/a"))
```

```
##                 MASE  MAPE  90% interval coverage
## model
## ETS             1.03  0.11                   0.92
## Regression      0.74  0.08                   0.92
## Seasonal naive  1.34  0.14                    n/a
```

The regression wins here, mainly because it knows about holidays and the yearly cycle, which ETS (with only weekly seasonality) and the naive forecast do not. Both model-based forecasters produce intervals with close to nominal coverage. In real data the ranking varies, which is the point of backtesting: the right model is the one that wins on *your* history, at *your* forecast horizon, not the one that sounds most sophisticated.

> **Pitfall.**
> Evaluating a forecast on the data it was fitted on, or on a single holdout period. In-sample fit rewards overfitting, and one period can be unusually easy or hard. Use a rolling-origin backtest at the horizon you actually need.

20.4 Anomaly detection: is today's number a problem?

Monitoring dashboards often flag a metric when it moves more than some percentage from yesterday or from last week. On data with strong weekly patterns, that rule fires every Saturday and Monday, and on holidays, and people learn to ignore it. A better rule compares each day with what a forecast expected for that day, and flags residuals that are large **relative to the model's normal error**.

We take the last 120 days and inject three real problems: a sign-up outage (one day at 40% of normal), a tracking break (ten days at 80% of normal) and a bot spike (one day at 180%). The holiday period at the end of the year is a genuine, expected dip that should not be flagged.

```
window = signups.index[-120:]
observed = signups.copy()
truth = pd.Series(False, index=signups.index)
for day, factor in [("2025-09-17", 0.4), ("2025-11-04", 1.8)]:
    observed[day] *= factor; truth[day] = True
brk = pd.date_range("2025-10-08", periods=10)
observed[brk] *= 0.8; truth[brk] = True

# Model fitted on history before the monitoring window, predicting each day in it
hist = observed[:window[0] - pd.Timedelta(days=1)]
model = sm.OLS(np.log1p(hist), design(hist.index)).fit()
resid = np.log1p(observed[window]) - model.predict(design(window))
mad = np.median(np.abs(model.resid - np.median(model.resid))) * 1.4826       # robust
scale
robust_z = resid / mad

rules = {
    "Day-over-day change > 25%": (observed.pct_change().abs() > 0.25)[window],
    "Same weekday last week, change > 25%": (observed.pct_change(7).abs() > 0.25)
[window],
    "Model residual, |robust z| > 3": (robust_z.abs() > 3),
}
rows = []
for name, flag in rules.items():
    tp = (flag & truth[window]).sum()
    rows.append({"rule": name, "days flagged": int(flag.sum()), "true problems
caught": int(tp),
                 "false alarms": int((flag & ~truth[window]).sum()), "problem days":
int(truth[window].sum())})
print(pd.DataFrame(rows).set_index("rule").to_string())
```

```
##                                     days flagged  true problems caught  false
alarms  problem days
## rule
## Day-over-day change > 25%                     38                     5
33            12
## Same weekday last week, change > 25%          25                     6
19            12
## Model residual, |robust z| > 3                 6                     6
0             12
```

```
fig, ax = plt.subplots(figsize=(7, 2.8))
expected = np.expm1(model.predict(design(window)))
ax.plot(window, observed[window], lw=1, color="#333", label="Observed")
ax.plot(window, expected, lw=1.5, ls="--", label="Expected")
flagged = window[rules["Model residual, |robust z| > 3"].values]
ax.scatter(flagged, observed[flagged], color="#C8102E", zorder=3, s=20,
label="Flagged")
for d in window[truth[window].values]:
    ax.axvspan(d - pd.Timedelta(hours=12), d + pd.Timedelta(hours=12),
color="#9A5B00", alpha=0.15, lw=0)
ax.set_ylabel("Daily sign-ups")
ax.legend(fontsize=8, loc="lower left")
plt.tight_layout()
plt.show()
```

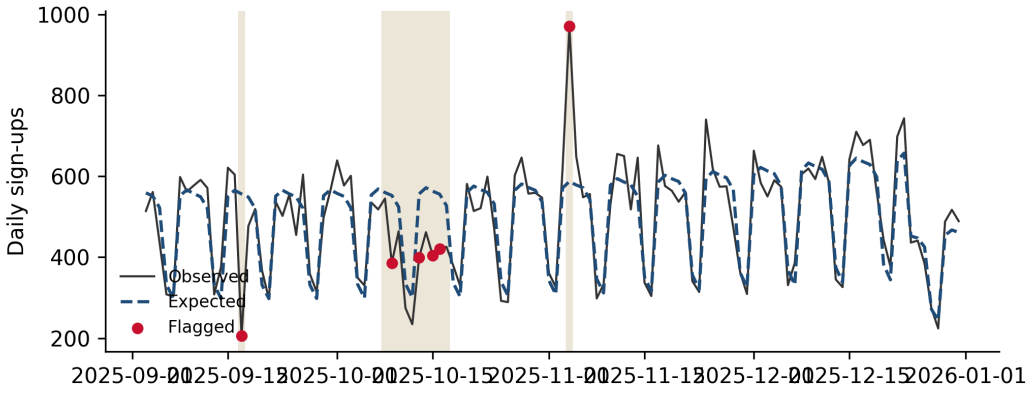


Figure 20.1: Monitoring window: observed sign-ups, the model's expectation, and days flagged by the residual rule. Injected problems are shaded.

The day-over-day rule drowns real problems in weekend alarms. The week-over-week rule is quieter but still fires on holidays and again a week after each problem, because last week's anomaly becomes this week's baseline. The model-based rule ignores the weekly pattern and the holidays, because it expects them, and flags the outage and spike. A modest tracking break is the hardest case: each day is only 20% low, so single-day thresholds catch only some of its days. Shifts that persist are better caught by change-point methods on the residuals or on ratios such as tracked to backend conversions (Chapter 2), which accumulate evidence over several days.

Good monitoring combines both: a residual rule for sharp one-day problems, and a change-point or cumulative-sum check for level shifts, run per market and per channel, with alerts routed to someone who can act.

20.5 Tools

- **Classical**: `statsmodels` (ETS, SARIMAX, regression with Fourier terms), the `forecast` and `fable` packages in R.
- **Prophet** and **NeuralProphet**: decomposable trend-seasonality-holiday models, easy to set up for business data.
- **Nixtla's statsforecast and neuralforecast**: fast implementations of many models, convenient for backtesting at scale.
- **Foundation models** such as Chronos: pretrained forecasters that can work without fitting; worth backtesting against the simple baselines before relying on them.
- **Anomaly and change-point detection**: `ruptures`, `adtk`, Kats, `Rbeast`.

20.6 Checklist

Checklist

- Is every forecast compared with a seasonal naive baseline in a rolling-origin backtest at the relevant horizon?
- Do prediction intervals achieve close to their nominal coverage in the backtest?
- Are holidays, promotions and known events in the model, so that expected dips are not treated as surprises?
- Do monitoring alerts compare each day with a model expectation, scaled by the model's normal error, rather than with yesterday?
- Are persistent level shifts checked with change-point or cumulative methods, not only single-day thresholds?
- Is each alert routed to an owner, with the change log checked before escalating?

20.7 Further reading

- *Forecasting: Principles and Practice* (Hyndman and Athanasopoulos) and its Python read-along; *skforecast*, *Greykite* (LinkedIn) and *Orbit* (Uber).
- Prophet's documentation on seasonality, holidays and regressors; *NeuralProphet*.
- *Nixtlaverse*, *Chronos* (Amazon) and its quickstart notebook, *pytorch-forecasting*, *AutoTS*, *Kats* (Meta), *modeltime* and *timetk* (R).
- `ruptures` for change points, `adtk` for anomaly detection, `stumpy` for matrix-profile pattern search, and `aeon` for time-series machine learning.

Full links are in Appendix A.

Chapter 21 Paid search

The decision this chapter supports.
How much of paid search is incremental, especially on brand terms, how to set bids and budgets on marginal rather than average cost, and which tests settle the question.

21.1 Why search needs special care

Paid search captures demand rather than creating most of it. People who search for your brand were already looking for you; people who search for the category are already in the market. That makes search look excellent in attribution (Chapter 13) and dangerous in an MMM (Chapter 9), where its spend moves with demand. The two questions that matter are how much of it is incremental, and what the next euro buys.

21.2 Brand terms: how much would you get anyway?

When you stop bidding on your own brand name, many of the people who would have clicked the ad click the organic result instead. The incremental value of brand search is the share of traffic that does *not* come back, plus whatever protection the ads give against competitors bidding on your name. Both vary by market and by how aggressively competitors bid, so it has to be measured.

A clean design is a **switchback test** across regions: randomly assign each region-week to brand ads on or off, so that every region spends some weeks in each state, and compare total brand-search visits (paid plus organic).

21.2.1 Worked example

Twenty regions, twelve weeks, brand ads on or off at random each region-week. Without ads, 80% of brand searches end in an organic click. With ads on, the ad takes half of those clicks and adds a small number of visits that would not have happened otherwise (8% of searches). So roughly five in six paid brand clicks would have arrived organically anyway.

```
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")
import statsmodels.formula.api as smf

rng = np.random.default_rng(21)
R, W = 20, 12
rows = []
size = rng.lognormal(np.log(2000), 0.5, R)
week_effect = 1 + 0.08 * np.sin(np.arange(W) / 2)
for r in range(R):
    for w in range(W):
        on = int(rng.random() < 0.5)
        searches = rng.poisson(size[r] * week_effect[w])
        organic_without_ads = rng.binomial(searches, 0.80)
        incremental = rng.binomial(searches, 0.08) if on else 0          # visits only
the ad produces
        captured = rng.binomial(organic_without_ads, 0.5) if on else 0   # would-be
organic clicks taken by the ad
        rows.append({"region": r, "week": w, "ads_on": on,
                     "paid_clicks": captured + incremental,
                     "visits": organic_without_ads + incremental})
df = pd.DataFrame(rows)
df["log_visits"] = np.log(df["visits"])

fit = smf.ols("log_visits ~ ads_on + C(region) + C(week)", df).fit(cov_type="cluster",
cov_kwds={"groups": df["region"]})
lift = np.exp(fit.params["ads_on"]) - 1
lo, hi = np.exp(fit.conf_int().loc["ads_on"]) - 1
on = df[df.ads_on == 1]
incr_visits = on["visits"].sum() * lift / (1 + lift)
print(f"Brand-search visits with ads on vs off: {lift:+.1%} (95% CI {lo:+.1%} to
{hi:+.1%})")
```

```
## Brand-search visits with ads on vs off: +9.8% (95% CI +9.0% to +10.6%)
```

```
print(f"Paid brand clicks in 'on' weeks: {on['paid_clicks'].sum():,.0f}; implied
incremental visits: {incr_visits:,.0f} "
      f"({incr_visits / on['paid_clicks'].sum():.0%} of paid clicks)")
```

```
## Paid brand clicks in 'on' weeks: 144,237; implied incremental visits: 23,557 (16%
of paid clicks)
```

Total brand-search visits are only about 10% higher with ads on, and only about one paid brand click in six is incremental. If brand CPC is 0.30 EUR, the cost per *incremental* visit is around six times that. Whether brand bidding pays depends on that incremental cost against the value of a visit, and on what competitors do in your absence. Those are empirical

questions, and the answer differs between markets: where competitors bid aggressively on your name, the incremental share is much higher.

> **Pitfall.**
> Turning brand search off nationally for two weeks and comparing with the two weeks before. Seasonality, other campaigns and competitor moves all change in the same window. Randomize across regions and weeks, and measure total brand traffic, not paid clicks.

21.3 Non-brand terms: marginal, not average

For non-brand terms, the issue is less incrementality (most non-brand clicks would not have reached you otherwise) than diminishing returns. Raising bids buys more clicks at higher prices; each extra conversion costs more than the last. Search platforms' bid simulators, or your own spend tests, trace a curve of conversions against spend. A power curve, conversions $= a \cdot \text{spend}^b$ with $b < 1$, usually fits well.

```
a, b = 0.8, 0.62                       # fitted from bid-simulator points, for example
spend = np.array([5_000, 10_000, 20_000, 30_000, 40_000, 60_000])
conv = a * spend ** b
avg_cpa = spend / conv
marg_cpa = 1 / (a * b * spend ** (b - 1))          # derivative of spend w.r.t.
conversions
print(pd.DataFrame({"monthly spend": spend, "conversions": conv.round(0),
                    "average CPA": avg_cpa.round(0), "marginal CPA":
marg_cpa.round(0)}).to_string(index=False))
```

```
##  monthly spend  conversions  average CPA  marginal CPA
##           5000        157.0         32.0          51.0
##          10000        242.0         41.0          67.0
##          20000        371.0         54.0          87.0
##          30000        477.0         63.0         101.0
##          40000        571.0         70.0         113.0
##          60000        734.0         82.0         132.0
```

The marginal CPA is always above the average CPA, by a factor of $1/b$ (here about 1.6). A team managing to an average CPA target of 60 EUR will keep raising spend until the average reaches 60, by which point the last conversions cost close to 100 EUR each. Set targets on marginal CPA, equal across campaigns and markets, and equal to what an incremental conversion is worth (Chapter 12).

21.4 Smart bidding and automated campaigns

Automated bidding and campaign types that span many inventories (such as Performance Max) optimize toward the conversions the platform can see, using signals including brand searches and remarketing audiences. They will find the cheapest *attributed* conversions, which are often the least incremental: brand-term traffic and existing customers. Practical safeguards:

- **Separate brand and non-brand** in reporting and, where the platform allows, in campaigns, so that cheap brand conversions do not subsidize the reported CPA of everything else.
- **Feed value, not just conversions**: offline conversion uploads with the system-of-record outcome and its value (Chapter 2), so the algorithm optimizes toward customers that matter.
- **Exclude existing customers** from acquisition campaigns where possible.
- **Measure with geo tests or conversion lift studies** (Chapters 6 and 8), not with the campaign's own attribution.

21.5 Checklist

Checklist

- Is brand search incrementality measured with a randomized design (regional switchback or geo holdout) on total brand traffic?
- Are brand and non-brand reported and targeted separately?
- Are bids and budgets set on marginal CPA, not average CPA?
- Do automated campaigns receive system-of-record conversions and values, and exclude existing customers?
- Is the search channel's MMM estimate treated as likely upward-biased until calibrated (Chapter 11)?

21.6 Further reading

- *Validating paid search ideas with calculus*, mackgrenfell.com
- *Paid search demand and supply model*, we-discover.com

Full links are in Appendix A.

Chapter 22 Paid social and creative fatigue

> **The decision this chapter supports.**
> Whether paid social prospecting is incremental, how to test creatives fairly, and when a creative has worn out and needs replacing.

22.1 What makes paid social different

Paid social platforms sell attention in feeds, and their delivery algorithms decide who sees which ad. Three consequences shape how it should be measured.

- **The platform chooses the audience.** Optimization toward conversions sends ads to people the algorithm predicts will convert, many of whom would have converted anyway. Attributed conversions overstate incrementality, often by a large factor (Chapters 1 and 8).
- **Creative is the main lever.** With targeting increasingly automated, the creative does most of the work of reaching the right people and persuading them. Creative testing and refresh are where much of the performance comes from.
- **Effects extend beyond the click.** Video and upper-funnel formats drive branded search, direct visits and later conversions that the platform cannot see. Lift studies on platform outcomes and geo tests on backend outcomes (Chapter 6) see different parts of this.

For incrementality, use the platform's conversion lift studies with system-of-record conversions where possible, and geo tests for the biggest budget decisions. The rest of this chapter is about creative.

22.2 Testing creatives fairly

Putting several creatives in one ad set and comparing their results is not a test. The delivery algorithm shifts spend toward whichever creative looks better in its first impressions, gives the others little delivery, and shows them to different audiences. The comparison is confounded by who saw what and when, and early noise becomes a self-fulfilling verdict.

A fair creative test needs:

- **Randomized audiences**: the platform's split-test or A/B feature, which assigns users to cells and holds delivery comparable.
- **A metric that matters**: cost per acquired customer (or better, per incremental customer), not click-through rate. Creatives that maximize clicks are often not the ones that convert.

- **Enough volume and a fixed duration**, set from the expected difference and the baseline rate (Chapter 5).

22.3 Creative fatigue

As the same people see the same creative again and again, response falls. That decline is **fatigue**, and its rate determines how often new creative is needed. It is best measured against **frequency** (average impressions per person reached), not calendar time, because a creative with a big budget wears out faster than the same creative with a small one.

22.3.1 Worked example: estimating how fast response decays

Twelve creatives run for 60 days each, launched at different times, with different budgets and different baseline appeal. Each creative's click-through rate falls exponentially with its average frequency. We estimate the decay rate across all creatives at once with a Poisson regression of clicks on frequency, with a separate baseline for each creative and impressions as exposure.

```
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")
import statsmodels.api as sm
import statsmodels.formula.api as smf

rng = np.random.default_rng(22)
K_TRUE = 0.18                                   # CTR falls by this rate per unit of
average frequency
rows = []
for c in range(12):
    base_ctr = rng.uniform(0.006, 0.015)
    daily_imps = rng.uniform(20_000, 80_000)
    audience = rng.uniform(150_000, 400_000)
    cum = 0.0
    for d in range(60):
        imps = rng.poisson(daily_imps)
        cum += imps
        reach = audience * (1 - np.exp(-cum / audience))
        freq = cum / reach                      # average frequency so far
        clicks = rng.binomial(imps, base_ctr * np.exp(-K_TRUE * (freq - 1)))
        rows.append({"creative": c, "day": d, "impressions": imps, "clicks": clicks,
"frequency": freq})
ads = pd.DataFrame(rows)

fit = smf.glm("clicks ~ C(creative) + frequency", ads, family=sm.families.Poisson(),
              offset=np.log(ads["impressions"])).fit()
k = -fit.params["frequency"]
lo, hi = -fit.conf_int().loc["frequency"][::-1]
print(f"Estimated decay per unit of frequency: {k:.3f} (95% CI {lo:.3f} to {hi:.3f});
true {K_TRUE}")
```

```
## Estimated decay per unit of frequency: 0.179 (95% CI 0.177 to 0.181); true 0.18
```

```
print(f"CTR halves after about {np.log(2) / k:.1f} additional impressions per person")
```

```
## CTR halves after about 3.9 additional impressions per person
```

```
print(f"CTR falls to 60% of launch level at an average frequency of about {1 +
np.log(1 / 0.6) / k:.1f}")
```

```
## CTR falls to 60% of launch level at an average frequency of about 3.9
```

```
import matplotlib.pyplot as plt
ads["rel_ctr"] = ads["clicks"] / ads["impressions"] / np.exp(
    fit.params["Intercept"] + fit.params.filter(like="C(creative)").reindex(
        [f"C(creative)[T.{c}]" for c in ads["creative"]]).fillna(0).to_numpy() - k)
fig, ax = plt.subplots(figsize=(7, 2.7))
ax.scatter(ads["frequency"], ads["rel_ctr"], s=6, alpha=0.4, color="#6B6B6B")
f = np.linspace(1, ads["frequency"].max(), 100)
ax.plot(f, np.exp(-k * (f - 1)), lw=2.2, color="#1F4E79", label="Estimated decay")
ax.axhline(0.6, ls=":", color="#C8102E", label="Refresh threshold (60%)")
ax.set_xlabel("Average frequency")
ax.set_ylabel("CTR relative to launch")
ax.legend()
plt.tight_layout()
plt.show()
```

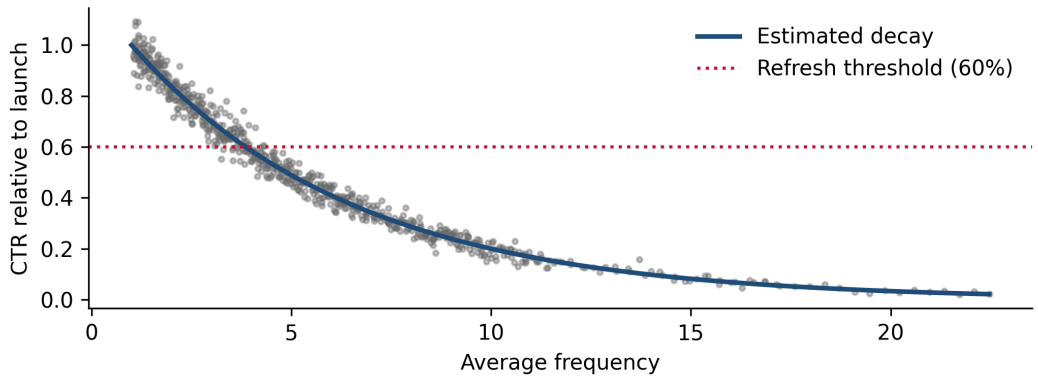


Figure 22.1: Daily CTR relative to each creative's launch level, against average frequency, with the estimated decay curve.

The pooled model recovers the true decay rate and turns it into a rule that planners can use: refresh a creative when its audience's average frequency approaches the threshold. Because the rule is in terms of frequency, it automatically schedules earlier refreshes for creatives with larger budgets or smaller audiences.

Three cautions apply. Click-through is a convenient proxy, but fatigue in conversion rate or cost per acquisition is what matters, and it is noisier; check that the two move together before relying on clicks. Fatigue rates differ by format, audience and market, so estimate them on your own data and refresh the estimate. And frequency is not only a fatigue problem: some exposure is needed before response appears at all, so the right frequency is a balance, best found with tests of different frequency caps.

Pitfall.
Declaring a winning creative from an ad set where the algorithm chose the delivery. Use the platform's split-testing feature, or a holdout design, and judge on acquisition cost, not click-through.

22.4 Checklist

Checklist

- Is paid social incrementality measured with lift studies on system-of-record outcomes, and checked with geo tests for large budgets?
- Are creative tests run with randomized audiences, on acquisition metrics, with a fixed sample size?
- Is fatigue measured against frequency rather than calendar days, with a refresh threshold derived from your own data?
- Do click-based fatigue signals agree with conversion-based ones?
- Are frequency caps tested rather than set by convention?

22.5 Further reading

- *Find unicorn ads and avoid creative fatigue* (MarTech Series)

Full links are in Appendix A.

Chapter 23 SEO measurement

The decision this chapter supports.
Whether a change to your site (titles, templates, internal linking, content, technical fixes) improved organic search traffic, given that search engines change their rankings constantly and nobody can randomize users.

23.1 Why SEO is hard to measure

Organic search traffic is driven by rankings you do not control. Search engines update their algorithms many times a year, competitors change their pages, seasonality moves demand, and your own other changes overlap. A before-and-after comparison on the pages you changed attributes all of that to your change. And you cannot randomize users, because the search engine decides what every user sees.

What you *can* randomize is **pages**. If a site has many pages of the same type (product pages, location pages, help articles, blog posts), randomly assign some to receive the change and keep the rest as controls. Both groups experience the same algorithm updates, seasonality and competition; the difference in how their traffic changes estimates the effect.

23.2 Worked example: an SEO split test

A site has 400 pages of the same template. On day 90, a new title format goes live on 200 randomly chosen pages, and truly lifts their organic clicks by 6%. On day 80, a search engine update independently lifts traffic on all pages by about 10%, and the site's traffic is seasonally rising anyway.

```
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")

rng = np.random.default_rng(23)
P, T, LAUNCH, UPDATE, LIFT = 400, 120, 90, 80, 0.06
page_base = rng.lognormal(np.log(40), 0.9, P)                    # daily clicks per
page
day = np.arange(T)
common = (1 + 0.002 * day) * (1 - 0.15 * np.isin(day % 7, [5, 6])) * np.where(day >=
UPDATE, 1.10, 1.0)
treated = np.zeros(P, bool)
treated[rng.permutation(P)[: P // 2]] = True
effect = np.where(treated[:, None] & (day[None, :] >= LAUNCH), 1 + LIFT, 1.0)
page_noise = np.exp(np.cumsum(rng.normal(0, 0.01, (P, T)), axis=1))       # pages
drift independently
clicks = rng.poisson(page_base[:, None] * common[None, :] * page_noise * effect)

pre, post = slice(50, LAUNCH), slice(LAUNCH, T)
before = clicks[:, pre].mean(1); after = clicks[:, post].mean(1)
naive = after[treated].sum() / before[treated].sum() - 1

# Difference-in-differences on page-level log ratios, with a bootstrap over pages
log_ratio = np.log((after + 1) / (before + 1))
did = lambda idx: np.exp(log_ratio[idx][treated[idx]].mean() - log_ratio[idx]
[~treated[idx]].mean()) - 1
est = did(np.arange(P))
boot = [did(rng.integers(0, P, P)) for _ in range(2000)]
print(f"Naive before/after on changed pages: {naive:+.1%}")
```

```
## Naive before/after on changed pages: +19.8%
```

```
print(f"Split test (DiD vs control pages):   {est:+.1%}  (95% CI {np.quantile(boot,
.025):+.1%} to "
      f"{np.quantile(boot, .975):+.1%}); true {LIFT:+.0%}")
```

```
## Split test (DiD vs control pages):   +5.8%  (95% CI +4.6% to +7.0%); true +6%
```

```
import matplotlib.pyplot as plt
idx = lambda m: pd.Series(clicks[m].sum(0) / clicks[m][:,
pre].sum(0).mean()).rolling(7, center=True).mean()
fig, ax = plt.subplots(figsize=(7, 2.7))
ax.plot(day, idx(treated), lw=2, label="Changed pages")
ax.plot(day, idx(~treated), lw=2, ls="--", label="Control pages")
ax.axvline(UPDATE, ls=":", color="#6B6B6B")
ax.text(UPDATE + 1, ax.get_ylim()[0] + 0.01, "algorithm update", fontsize=8)
ax.axvline(LAUNCH, color="#C8102E", lw=1)
ax.text(LAUNCH + 1, ax.get_ylim()[1] - 0.04, "change live", fontsize=8,
color="#C8102E")
ax.set_xlabel("Day")
ax.set_ylabel("Clicks (pre-period = 1)")
ax.legend(loc="upper left")
plt.tight_layout()
plt.show()
```

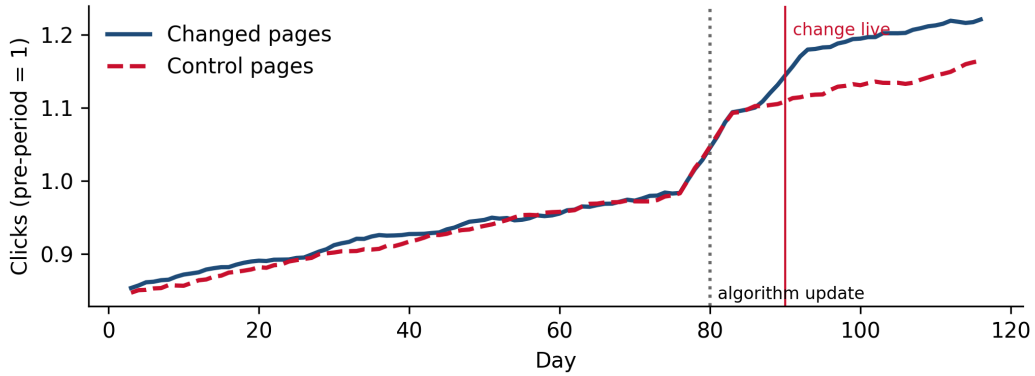


Figure 23.1: Organic clicks for changed and control pages (7-day averages, indexed to the pre-period). The algorithm update moves both groups; only the title change separates them.

The naive comparison credits the change with the algorithm update and the seasonal rise, and reports an effect several times too large. The split test differences them out and recovers the true lift with an honest interval.

23.3 Designing SEO tests

- **Page groups must be comparable**: same template, similar traffic profile. Randomize within strata of traffic (for example, pair pages by pre-period clicks and randomize within each pair) to improve balance and power.
- **Enough pages and traffic.** Page-level traffic is highly variable; dozens of pages with little traffic each will rarely detect a realistic effect. Run a placebo analysis on historical data first (pretend a change happened, as in Chapter 7) to see what effect size the design can detect.

- **Allow for crawling and indexing.** Search engines take days to weeks to re-crawl and re-rank changed pages. Define the readout window from launch plus indexing time, and check in the search console when changed pages were recrawled.
- **Watch for spillover.** Changes to internal linking or site-wide elements affect control pages too. Those changes need a site-level design (CausalImpact against other sites or markets, Chapter 7) rather than a page split.

23.4 Data caveats

Search console data is aggregated and partly anonymized: rare queries are hidden, figures are rounded, and query-level and page-level totals do not always reconcile. Use page-level clicks for tests, keep the extraction method fixed, and log search engine updates and your own site releases in the change log (Chapter 2), because they are the most common confounders in organic traffic analysis.

> **Pitfall.**
> Attributing an organic traffic change to an SEO project without a comparison group. Algorithm updates, seasonality and competitor changes routinely move organic traffic by more than any single change you make.

23.5 Checklist

> **Checklist**
> - Can the change be tested by randomizing comparable pages into changed and control groups?
> - Is randomization stratified by pre-period traffic, and has a placebo analysis shown the design can detect a realistic effect?
> - Does the readout window allow for crawling and re-ranking?
> - Are site-wide changes analyzed with a site-level quasi-experiment instead of a page split?
> - Are search engine updates and site releases in the change log?

23.6 Further reading

- *Measuring SEO effectiveness using causal impact analysis* (WordLift)
- *CausalImpact for SEO* (JC Chouinard)

Full links are in Appendix A.

Chapter 24 CRM and email holdouts

> ***The decision this chapter supports.***
> *Whether lifecycle messaging (email, push, in-app, SMS) actually changes customer behavior, which programs and messages to keep, and how to measure them when opens and clicks no longer mean what they used to.*

24.1 Why CRM measurement is easy to get wrong

CRM messages go to people who are already customers or prospects, and the people who engage with them are disproportionately the ones who were going to act anyway. "Conversions from users who clicked the email" is the purest form of the attribution problem from Chapter 13: engaged users click and convert, and the email takes the credit.

Engagement metrics have also degraded. Mail clients that pre-fetch images (Apple's Mail Privacy Protection is the best-known example) register opens that never happened, so open rates are inflated and no longer comparable over time or across audiences. Clicks are more reliable but still measure engagement, not effect.

The good news is that CRM is the easiest channel to randomize. You control who receives each message, so every program can have a holdout.

24.2 Worked example: attributed versus incremental

An onboarding email program goes to 50,000 new customers, with 20% randomly held out for the program's duration. The outcome is activation (first real use of the product) within 30 days. Customers differ in underlying intent; high-intent customers activate more often and click emails more often. The program's true effect is a modest increase in activation.

```
import numpy as np, pandas as pd

rng = np.random.default_rng(24)
n = 50_000
intent = rng.normal(0, 1, n)
holdout = rng.random(n) < 0.20
p_base = 1 / (1 + np.exp(-(-0.8 + 1.2 * intent)))           # activation without
emails
TRUE_EFFECT = 0.03                                          # +3 points for those who
receive the program
activated = rng.random(n) < np.where(holdout, p_base, np.clip(p_base + TRUE_EFFECT, 0,
1))
clicked = ~holdout & (rng.random(n) < 1 / (1 + np.exp(-(-1.5 + 1.0 * intent))))

attributed = (activated & clicked).sum()                    # "activated after
clicking an email"
rate_send, rate_hold = activated[~holdout].mean(), activated[holdout].mean()
incremental = (rate_send - rate_hold) * (~holdout).sum()
se = np.sqrt(rate_send * (1 - rate_send) / (~holdout).sum() + rate_hold * (1 -
rate_hold) / holdout.sum())
print(f"Activation: send group {rate_send:.1%}, holdout {rate_hold:.1%}, "
      f"difference {rate_send - rate_hold:+.1%} (95% CI {rate_send - rate_hold -
1.96*se:+.1%} to "
      f"{rate_send - rate_hold + 1.96*se:+.1%})")
```

```
## Activation: send group 37.5%, holdout 35.3%, difference +2.2% (95% CI +1.2% to
+3.3%)
```

```
print(f"Attributed activations (clicked, then activated): {attributed:,}")
```

```
## Attributed activations (clicked, then activated): 4,673
```

```
print(f"Incremental activations (vs holdout):           {incremental:,.0f}")
```

```
## Incremental activations (vs holdout):           895
```

```
print(f"Attributed / incremental:                       {attributed /
incremental:.1f}x")
```

```
## Attributed / incremental:                       5.2x
```

The program works: activation is about two points higher in the send group (the true effect is three), and the holdout comparison says so with a usable interval. But "activated after clicking" overstates its contribution several times over, because clickers are the customers most likely to activate anyway. A

program judged on attributed numbers looks like a large win; judged on the holdout, it is a real but modest one. That difference decides how much to invest in it, and whether a more ambitious program would pay.

24.3 Holdout designs for CRM

- ***Per-campaign holdouts**: a random 5% to 20% of each send's audience receives nothing. Good for testing individual campaigns, but many small campaigns each have little power.*
- ***Program-level holdouts**: a random share of customers excluded from an entire program (onboarding, re-engagement) for its full duration. Measures the program's total effect, which is usually what decisions are about.*
- ***Global holdout**: a small, persistent random group (often 2% to 5%) that receives no marketing messages at all, refreshed periodically. Measures the cumulative effect of all CRM over months, including fatigue and unsubscribes, which campaign-level tests miss.*
- ***Message tests within the send group**: A/B tests between versions, with the holdout still in place, so you learn both which version is better and whether either beats nothing.*

Size the holdout from the effect you need to detect. Activation effects of a few percentage points need thousands of customers in the holdout; effects on revenue per customer, which is far noisier, need more (Chapter 5).

```
from scipy import stats
z = stats.norm.ppf(0.975) + stats.norm.ppf(0.8)
p = 0.35
rows = []
for audience in [10_000, 50_000, 200_000]:
    for share in [0.05, 0.10, 0.20]:
        n_h, n_s = audience * share, audience * (1 - share)
        rows.append({"audience": f"{audience:,}", "holdout share": f"{share:.0%}",
                     "MDE (points)": round(100 * z * np.sqrt(p * (1 - p) * (1 / n_h +
1 / n_s)), 2)})
tbl = pd.DataFrame(rows).pivot(index="audience", columns="holdout share", values="MDE
(points)")
print(tbl.loc[["10,000", "50,000", "200,000"], ["5%", "10%", "20%"]].to_string())
```

```
## holdout share    5%   10%   20%
## audience
## 10,000         6.13  4.45  3.34
## 50,000         2.74  1.99  1.49
## 200,000        1.37  1.00  0.75
```

With small audiences and small holdouts, only large effects are detectable. Many CRM programs have true effects of one or two points; for them, a 5% holdout on 10,000 customers is unlikely to show

anything, and the honest conclusion from a null result is "we could not detect an effect smaller than about six points".

> ***Pitfall.***
> *Using open rates or "conversions after click" to evaluate CRM. Opens are inflated by privacy features; clicks select engaged users. Use randomized holdouts and measure the outcome itself.*

24.4 Guardrails and long-term effects

Messages have costs that a single campaign readout misses: unsubscribes and spam complaints remove people from future reach, and message fatigue can lower response to everything that follows. Track unsubscribe and complaint rates as guardrail metrics in every test, and use the global holdout to see the net effect of the whole program over months. A campaign that wins its own test while raising unsubscribes can still lose overall.

Also watch for ***targeting by uplift rather than by engagement*** *(Chapter 16): the customers most likely to respond to a re-engagement message are not always the least engaged ones, and some highly engaged customers react negatively to extra messages.*

24.5 Checklist

> ***Checklist***
>
> - *Does every CRM program have a randomized holdout, sized from the effect that matters?*
> - *Is there a persistent global holdout measuring the cumulative effect of all messaging?*
> - *Are results read on the outcome (activation, transactions, revenue), not on opens or post-click conversions?*
> - *Are unsubscribes, complaints and long-run engagement tracked as guardrails?*
> - *Are message A/B tests run with a no-message holdout still in place?*

24.6 Further reading

- *CausalML and EconML for uplift modeling of message targeting.*

Full links are in Appendix A.

Chapter 25 Readouts that change decisions

> ***The decision this chapter supports.***
> *How to write up an analysis or experiment so that the people who decide can act on it correctly: what to say first, how to state uncertainty without hedging, and how to report results that are weak, null or mixed.*

25.1 The readout is part of the method

An analysis that is right but misread has the same effect as one that is wrong. Most measurement failures in practice are not statistical errors but communication ones: a non-significant result reported as a small win, an interval dropped on the way into a slide, an observational estimate presented with the authority of an experiment, a caveat buried on page four that would have changed the decision.

The structure below works for experiment readouts, model results and ad-hoc "did it work?" answers alike.

25.2 A structure that works

1. ***The verdict and how sure you are, in one or two sentences.*** *"The heavy-up paid back with an iROAS of about 1.4, but the test cannot rule out a loss (95% interval 0.3 to 2.4)." The reader who stops here should still make the right decision.*
2. ***What we did.*** *Design (experiment, quasi-experiment, model, observational), dates, markets, sample, primary metric and its window. Two to five lines.*
3. ***Results.*** *The primary metric with its interval, then secondary metrics and guardrails, in a table. Every number has a source.*
4. ***What we checked.*** *Data quality, sample ratio, pre-period fit, placebo tests, sensitivity. One line each, with the outcome.*
5. ***Caveats that could change the conclusion.*** *Not a list of every limitation; only those that would change what the reader should do.*
6. ***Recommendation.*** *What to do, who owns it, by when, and what would change the recommendation.*

The order matters. Leading with method and caveats and ending with the answer protects the analyst and fails the reader.

25.3 Saying how sure you are

Uncertainty is part of the result, not a disclaimer. A few conventions keep it precise without hedging every sentence.

- ***Always give the interval with the estimate**, and say what the interval is (95% confidence, 90% credible). "Lift 3.8% (95% CI 0.8% to 6.6%)".*
- ***Say what the result rules out.** "The test rules out a lift above 7%" is often more useful than the point estimate.*
- ***Report null results as results.** "We could not detect an effect; the test could have detected effects of 4% or more" is a finding. "A small positive lift (not significant)" is not; it invites the reader to treat noise as signal.*
- ***Name the design's strength.** Experiments, quasi-experiments and models deserve different levels of confidence (Chapter 3). Say which one this is in the first paragraph.*
- ***Disclose the analysis path.** How many metrics and segments were examined, whether the test was read before its planned end, and whether the analysis plan changed. These change how much the result should be trusted (Chapter 5).*

A simple language ladder helps keep wording consistent across a team:

Evidence	*Wording*
Interval excludes zero, design is an experiment	*"X increased Y by about Z (interval)"*
Interval excludes zero, quasi-experimental or model-based	*"X is associated with an increase of about Z; the design cannot rule out [named confounders]"*
Interval includes zero, test well powered	*"We found no effect larger than [MDE]"*
Interval includes zero, test underpowered	*"The test was too small to tell; effects below [MDE] were undetectable"*

25.4 Showing uncertainty in charts

Bar charts of point estimates hide exactly what the reader needs. Show intervals.

```python
import numpy as np
import matplotlib.pyplot as plt

segments = ["DE", "IT", "FR", "ES"]
est = np.array([4.1, 2.6, 1.9, -0.8])
half = np.array([2.2, 3.1, 2.9, 3.4])            # half-width of 95% interval

fig, (a1, a2) = plt.subplots(1, 2, figsize=(8, 2.7), sharey=False)
a1.bar(segments, est, color="#1F4E79")
a1.set_title("Point estimates only")
a1.set_ylabel("Lift (%)")
y = np.arange(4)[::-1]
a2.errorbar(est, y, xerr=half, fmt="o", capsize=4, color="#1F4E79")
a2.axvline(0, color="#6B6B6B", lw=0.8)
a2.set_yticks(y, segments)
a2.set_title("With 95% intervals")
a2.set_xlabel("Lift (%)")
plt.tight_layout()
plt.show()
```

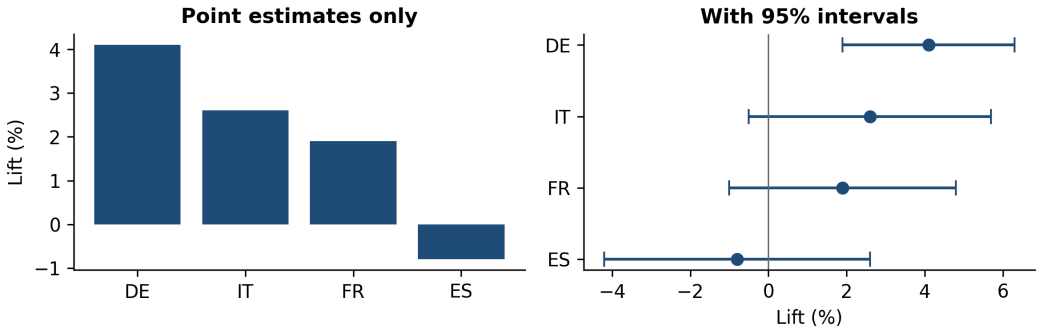


Figure 25.1: The same results twice. Left: bars of point estimates suggest a clear ranking. Right: estimates with 95% intervals show that only one effect is distinguishable from zero and the ranking is uncertain.

The left chart invites a ranking of markets and a decision to cut Spain. The right chart shows that only Germany's effect is distinguishable from zero, that the four intervals overlap heavily, and that a country ranking would mostly reflect noise. With four countries examined, the chance of at least one spurious "significant" result is already about one in five; a cut by country should be labeled as exploratory unless it was planned.

Other habits worth keeping: put the decision threshold (break-even, target CPA) on the chart; avoid dual axes; label units; and keep the same scale across charts that will be compared.

25.5 An example: two versions of the same readout

Version A. *"The Q2 heavy-up test in 8 regions showed a +3.8% lift in sign-ups ($p = 0.01$), delivering 1,040 incremental conversions and an iROAS of 1.39. Recommend rolling out the higher spend level*

nationally."

Version B. *"The heavy-up increased sign-ups in the test regions by about 4% (95% interval 1% to 7%), so the extra spend did drive some conversions. At 120 EUR per conversion, that is an iROAS of about 1.4, but the interval runs from 0.3 to 2.4, so we cannot rule out that the extra spend lost money. Design: stratified randomized geo test, 8 of 40 regions, 30 days, backend sign-ups, synthetic control with placebo-based intervals; pre-period fit 3.4% MAPE. Recommendation: keep the higher level in the test regions for another 30 days to narrow the interval, rather than rolling out nationally now (owner: channel lead; decision date: end of next month). A national roll-out at this spend level would be a bet with roughly a one-in-four chance of losing money."*

Version A is shorter and sounds more decisive, and it would lead to a decision the evidence does not support. Version B gives the same headline, the uncertainty that matters, and a recommendation proportionate to it.

25.6 Checklist

> ***Checklist***
>
> - *Does the readout open with the verdict and the confidence in it?*
> - *Is the design named in the first paragraph, with observational results labeled as such?*
> - *Does every estimate have an interval, a sample size, a time window and a source?*
> - *Are null results reported with the effect size that could have been detected?*
> - *Are the number of comparisons and any interim looks disclosed?*
> - *Do charts show intervals and decision thresholds?*
> - *Does the recommendation have an owner, a date and a statement of what would change it?*

25.7 Further reading

- *Data visualization: Fundamentals of Data Visualization (Claus Wilke), Data to viz, and the Python and R graph galleries.*
- *Statistical Thinking for the 21st Century (Poldrack) on communicating uncertainty.*
- *The Google measurement playbook for examples of readout conventions in industry.*

Full links are in Appendix A.

Chapter 26 The analyst's toolkit

> ***The decision this chapter supports.***
> *Which tools and working habits make measurement work reliable and repeatable, and where newer tools (including AI assistants) help and where they need care.*

26.1 Habits before tools

Tools change every year; a few habits outlast them and prevent most of the errors described in this book.

- ***Write the plan before the data.*** *Hypothesis, design, metric, window, decision rule (Chapter 5). A dated plan is the best defense against analyzing until something looks good.*
- ***Simulate before you analyze.*** *Every worked example in this book used data with a known truth. Do the same for your own designs: simulate data that looks like yours, with an effect you choose, and check that the method recovers it and how often it would detect it. It costs an afternoon and regularly saves a quarter.*
- ***Keep a change log*** *of tracking, naming, pricing, promotion and platform changes (Chapter 2).*
- ***Make analyses reproducible.*** *Code in version control, fixed random seeds, pinned package versions, and a record of which data snapshot an analysis used. The test: could someone else rerun it six months from now and get the same number?*
- ***Separate exploration from confirmation.*** *Exploratory slicing generates hypotheses; confirmatory analyses test pre-specified ones. Label which is which in every readout.*

26.1.1 Simulation-based power, in a dozen lines

The simulation habit generalizes the power calculations used throughout the book. Write a function that generates one dataset under the design and returns the analysis result; run it many times with and without an effect.

```python
import numpy as np

def simulate_once(rng, effect, n_regions=30, n_treated=10, days=28, noise=0.08):
    """One geo test: daily outcomes per region, multiplicative effect in treated
regions."""
    base = rng.lognormal(np.log(100), 0.6, n_regions)
    treated = rng.permutation(n_regions) < n_treated
    pre = base * rng.lognormal(0, noise, n_regions)
    post = base * rng.lognormal(0, noise, n_regions) * np.where(treated, 1 + effect,
1.0)
    log_ratio = np.log(post / pre)
    return log_ratio[treated].mean() - log_ratio[~treated].mean()

def power(effect, sims=2000, seed=0):
    rng = np.random.default_rng(seed)
    null = np.array([simulate_once(rng, 0.0) for _ in range(sims)])
    alt = np.array([simulate_once(rng, effect) for _ in range(sims)])
    crit = np.quantile(np.abs(null), 0.95)          # 5% false positive rate under the
null
    return (np.abs(alt) > crit).mean()

for e in [0.02, 0.05, 0.08]:
    print(f"True lift {e:.0%}: power {power(e):.0%}")
```

```
## True lift 2%: power 6%
## True lift 5%: power 20%
## True lift 8%: power 42%
```

The answer is sobering, and typical: with 30 regions, 10 treated, and region-level noise of this size, even an 8% lift is detected less than half the time. Finding that out takes seconds before a test, and a quarter of wasted spend after one. More regions, a longer pre-period, or a bigger spend change would be needed.

The same pattern works for any design in this book: change `simulate_once` *to match how your data is generated and how you will analyze it. Because the null distribution comes from the same simulation, the false positive rate is correct by construction, even for analyses with no textbook formula.*

26.2 The core stack

Task	Typical tools	Notes
Data access	SQL on the warehouse (Databricks, BigQuery, Snowflake)	Build the measurement panel once (Chapter 2) and reuse it
Analysis	Python (`pandas`, `statsmodels`, `scipy`) or R (`tidyverse`, `fixest`)	Either is fine; use what the team can review
Bayesian modeling	PyMC, Stan, NumPyro; PyMC-Marketing, Meridian for MMM	Chapters 10 to 12
Experiments	Platform tools, in-house assignment, GeoLift, matched_markets, CausalPy	Chapters 5 to 7
Forecasting	statsmodels, Prophet, statsforecast, Chronos	Always backtest against a seasonal naive (Chapter 20)
Notebooks and reports	Jupyter, marimo, Quarto, R Markdown, bookdown	Reports generated from code stay in sync with the analysis
Exploration	`sweetviz`, `DataExplorer`, PyGWalker, `naniar`	Quick profiling of new data before modeling
Dashboards	Looker Studio, Tableau, internal BI tools	Show intervals and thresholds, not only point estimates

Spreadsheets still have a place: quick break-even calculations, sharing small tables, and teaching. The library's Google Sheets and Excel resources include regression, logistic regression, conjoint and pricing workbooks that are useful for explaining methods to non-analysts.

26.3 AI assistants in measurement work

Large language model assistants are now part of most analysts' workflow, and they are genuinely useful for drafting SQL and code, explaining unfamiliar methods, reviewing code for errors, generating simulation scaffolding, and writing first drafts of readouts. Text-to-SQL tools and AI analyst agents connected to a warehouse can answer routine questions quickly.

They also fail in ways that matter specifically for measurement:

- ***Plausible but wrong definitions.*** *An assistant will choose an industry-standard definition of a metric when your company's definition differs. Check generated queries against the metric dictionary.*
- ***Silent analytical choices.*** *Generated analyses make decisions (filters, windows, outlier handling, which comparison to make) without flagging them. Each one can change the answer.*

- ***Confident causal language.*** *Assistants readily describe correlations as effects and before-and-after comparisons as impact. Apply the same scrutiny as to any analyst's draft (Chapter 25).*
- ***Data handling.*** *Customer and financial data should only go into tools approved for it.*

The practical rule is the same as for any junior collaborator: use them to go faster, review everything, and verify every number that will reach a decision against a source you trust.

26.4 Building a measurement team's library

The resource library behind this book (Appendix A) is an example of a habit worth copying: a shared, organized collection of methods, tools, case studies and code that the team adds to as it learns. Pair it with an internal record of your own experiments and their results, with designs, dates, outcomes and what changed afterwards. Over time, that record becomes the most valuable measurement asset you have: priors for new models, benchmarks for new tests, and evidence of what actually works in your market.

26.5 Checklist

> ***Checklist***
>
> - *Is there a written plan, dated before the data, for every confirmatory analysis?*
> - *Was the design checked by simulation, including its power and false positive rate?*
> - *Is the analysis reproducible: code in version control, seeds fixed, package versions and data snapshot recorded?*
> - *Are AI-generated queries and analyses reviewed against the metric dictionary and checked for silent choices?*
> - *Is there a shared record of past experiments and results that new work can draw on?*

26.6 Further reading

- *R for Data Science*
- *Python for Data Analysis (Wes McKinney)*
- *An Introduction to Statistical Learning*
- *Forecasting: Principles and Practice*

Full links are in Appendix A.

A Resource library

This appendix collects the public resources the further-reading sections draw on, grouped by the part of the book they support. Within each group, links keep the order of the original library. Where a link's own text was just its address, the entry shows the site and path instead. Links to private workspaces (internal Slack, Notion and app sessions) have been removed. Some links point to shared Google Drive or Colab files whose access depends on the owner's sharing settings.

A.1 Part I. Foundations

Measurement Playbook

- *Google's Measurement Playbook* (thinkwithgoogle.com)

Signals

- *How does IDFA deprecation impact ad prices?* (mobiledevmemo.com)

Causal Inference

- *EconML*
- *Causal Impact Google* (google.github.io)
- *Causal Impact Google presentation* (youtube.com)
- *tfp-causalimpact*
- *SEO Title Tag Optimization at Etsy*
- *pycausalimpact*
- *Causal Py* (causalpy.readthedocs.io)
- *Market Matching* (cran.r-project.org)
- *Market Matching GitHub* (github.com)
- *Causal Inference Mixtape* (mixtape.scunning.com)
- *Difference in Differences, Causal Impact, Synthetic Controls* (towardsdatascience.com)
- *Causal Inference Course* (bradyneal.com)
- *Statistical Inference with R book* (moderndive.com)
- *Causal Analysis with PyMC* (pymc-labs.io)
- *Causal Inference Book* (appliedcausalinference.github.io)
- *Bootstrap Random Walks for Causal Inference Analysis on Time Series*
- *Dagitty (DAG tool)* (dagitty.net)
- *Causal Inference for The Brave and True*

- *r-causal.org*
- *pysyncon*
- *causalfm-toolkit*

Regression

- *Some useful equations for nonlinear regression in R*
- *Interpret Logistic Regression Coefficients*
- *Interpret the Logistic Regression Intercept*
- *Logistic Regression in R (with Categorical Variables)*
- *Understanding logistic regression using predicted probabilities*

Statistics

- *An Introduction to Statistical Learning*
- *Forecasting: Principles and Practice*
- *Introduction to Modern Statistics*
- *Introduction to Advertising Measurement*
- *Bayes Rules!*
- *Statistical Thinking for the 21st Century*

Marketing and Data Analytics Resources

- *Marketing Analytics with R (github.com)*
- *Course (coursera.org)*
- *Marketing Analytics in R - Datacamp Course (datacamp.com)*
- *Marketing Analytics using R and Excel - Udemy Course (udemy.com)*
- *R Marketing Analysis Packages (github.com)*
- *Hands-on Data Science for Marketing (github.com)*
- *Python for Marketing Research (python-marketing-research.github.io)*
- *Elements of Data Science*
- *Python for Data Analysis*

A.2 Part II. Experiments

A/B tests

- *Mobile ad creative: how to produce and deploy advertising creative at scale (mobiledevmemo.com)*
- *A/B Test Calculator (cxl.com)*

- *abtest.design*

Geo tests

- *Meta GeoLift (facebookincubator.github.io)*
- *Commuting Zones package (github.com)*
- *Google Geo Experiment (github.com)*
- *Synthetic Control TidySynth (github.com)*
- *Google Meridian Geox*

A.3 Part III. Models

Marketing mix modeling

- *Google Meridian MMM*
- *Meta Robyn (facebookexperimental.github.io)*
- *Facebook Prophet (facebook.github.io)*
- *Bayesian MMM (getrecast.com)*
- *MMM checklist (getrecast.com)*
- *Seasonality (getrecast.com)*
- *Creative (getrecast.com)*
- *MMM datasets (forecastegy.com)*
- *Adstock (from Recast) (getrecast.com)*
- *Econometrics in Python (saxifrage.xyz)*
- *Econometrics in GSheets (saxifrage.xyz)*
- *Geo Model (saxifrage.xyz)*
- *Data checklist (saxifrage.xyz)*
- *HelloFresh example (engineering.hellofresh.com)*
- *Media Effect Estimation with Orbit's KTR Model*
- *pymc-marketing*
- *Better budgeting with Bayesian models: Bolt's story with PyMC-Marketing*
- *Calibration with experiments (juanitorduz.github.io)*
- *MaMiMo GitHub (github.com)*
- *mostlytidyMMM*
- *Estimating Long-term base (medium.com)*
- *Prophetverse MMM*

- *mmm-eval*
- *siMMMulator*

Diminishing Returns/Cost Curves/Response Curves

- *Diminishing Returns model (saxifrage.xyz)*
- *One piece of calculus that can change the way you advertise (mackgrenfell.com)*
- *Marginal CPA (mackgrenfell.com)*

Budget Optimization

- *Dorfman-Steiner theorem (mpra.ub.uni-muenchen.de)*

Attribution

- *How to game attribution models (web.archive.org)*
- *The fastshap R package*
- *SHAP: Shapley Additive Explanations*
- *Marketing Channel Attribution With Markov Models In R*
- *Shapley Value vs. Markov Model in marketing attribution*
- *ChannelAttribution*
- *MTA python package (github.com)*

A.4 Part IV. Customers

Cohort Analysis

- *Cohort Retention Analysis with BART*

Lifetime Value (LTV, CLV)

- *Customer Lifetime Value (CLV) in Python (PyMC)*
- *Marketing P&L using LTV and ROAS (mobiledevmemo.com)*
- *How much data is needed to predict LTV?(mobiledevmemo.com)*

Churn Analysis and Prediction

- *Kaplan-Meier Survival Analysis in Python*
- *Churn Analysis in R*
- *Customer Churn Prediction in Telecommunication*

Segmentation

- *Gower distance*

Decile Analysis

- *The Lost Art of Decile Analysis (medium.com)*

A.5 Part V. Brand, pricing and demand

Brand Measurement

- *Google's Brand Measurement Playbook (thinkwithgoogle.com)*

Share of Search

- *Branded Share of Search Reporting*

Share of Brand

- *User Guide on R Package NBDdirichlet*
- *Dirichlet Model (empgens.com)*

Halo Effect

- *Halo Effect (saxifrage.xyz)*

Pricing Analysis

- *Van Westendorp's Price Sensitivity Meter*
- *Gabor–Granger method*
- *Multilevel Elasticities for a Single SKU (juanitorduz.github.io)*

Revenue Growth Analytics (Pricing/Margin)

- *Price Elasticity: What It Is & How to Calculate It*

Choice Modeling

- *Conjoint Analysis*

Factor Analysis for Market Research

- *Factor Analysis (researchoptimus.com)*

Forecasting, time series and anomaly detection

- *Uber Orbit (github.com)*

- *Meta Prophet*
- *PyTorch Forecasting Documentation*
- *Forecasting: Principles and Practice* (otexts.com)
- *skforecast*
- *Nixtlaverse*
- *Greykite*
- *Chronos*
- *Anomaly Detection Toolkit (ADTK)*

A.6 Part VI. Channel playbooks

Paid search

- *Validating paid search ideas with calculus* (mackgrenfell.com)
- *Paid Search Demand and Supply Model* (we-discover.com)

Creative Fatigue

- *How to Find Unicorn Ads and Avoid Facebook Creative Fatigue*

SEO

- *Google Search Updates* (status.search.google.com)
- *Google Search Console for Python*
- *Measuring SEO effectiveness using Causal Impact Analysis*
- *CausalImpact for SEO*

Virality (word of mouth)

- *Word of Mouth Coefficient* (reforge.com)
- *How to calculate Word of Mouth coefficient* (reforge.com)
- *Facebook viral marketing: When and why do apps "jump the shark?"*

User Behavior Analysis

- *Retentioneering Documentation*

A.7 Part VII. Practice and tools

Exploratory Analysis

- *correlationfunnel library*
- *Data Explorer library (boxuancui.github.io)*
- *naniar library (naniar.njtierney.com)*

Data Visualizations

- *From Data to Viz*
- *Python Graph Gallery*
- *R Graph Gallery*
- *Fundamentals of Data Visualization*
- *Modern Data Visualization with R*

R Language

- *Statistical Inference via Data Science: A ModernDive into R and the Tidyverse (moderndive.com)*
- *R for Data Science (r4ds.had.co.nz)*
- *An Introduction to Machine Learning with R (lgatto.github.io)*
- *ggplot2: Elegant Graphics for Data Analysis*
- *Big Book of R*

Sentiment Analysis / Text Analysis

- *Supervised Machine Learning for Text Analysis in R (smltar.com)*
- *Text Mining with R (Book) (tidytextmining.com)*

B Glossary

Adstock
: *The carryover of an advertising effect into later periods. Geometric adstock carries a fixed share of each period's effect into the next (Chapter 9).*

Attributed conversions
: *Conversions a tracking rule assigns to a channel because the converting user was exposed to it. Not the same as incremental conversions (Chapters 1 and 13).*

Average versus marginal return
: *Average return is total outcome per unit of total spend; marginal return is the extra outcome from the next unit of spend. Budget decisions depend on the marginal return (Chapter 12).*

Backtest (rolling-origin)
: *Evaluating a forecast by repeatedly fitting on data up to a date and forecasting the following period, then moving the date forward (Chapter 20).*

Calibration (of an MMM)
: *Using experiment results to constrain a model's estimates, as a prior or an additional likelihood term (Chapter 11).*

Censoring
: *Incomplete observation of a time-to-event outcome, such as customers who have not churned yet. Methods that ignore it are biased (Chapter 16).*

Cohort
: *A group of customers acquired in the same period, usually also by the same channel, tracked by time since acquisition (Chapter 15).*

Collider
: *A variable caused by two others. Conditioning on it creates a spurious association between them (Chapter 3).*

Confounder
: *A common cause of the treatment and the outcome. Failing to adjust for it biases the estimated effect (Chapter 3).*

Conjoint analysis
: *A survey method in which respondents choose between product profiles, used to estimate the value of features and willingness to pay (Chapter 19).*

Counterfactual
: *What would have happened without the treatment. Every causal estimate compares an observed outcome with an estimated counterfactual (Chapter 3).*

CUPED
: *A variance-reduction technique for experiments that adjusts outcomes using pre-experiment values of the same metric (Chapter 5).*

Difference-in-differences (DiD)

Comparing the change over time in a treated group with the change in a comparison group, under the assumption of parallel trends (Chapter 7).

Divergence (MCMC)

A warning from Hamiltonian Monte Carlo samplers that the posterior geometry was not explored reliably, often fixed by reparameterization (Chapter 10).

Double jeopardy

The empirical regularity that smaller brands have fewer buyers who are also slightly less loyal (Chapter 18).

Geo experiment

An experiment that randomizes geographic regions rather than individuals to treatment and control (Chapter 6).

Guardrail metric

A metric that must not deteriorate in a test, such as unsubscribes or complaints, even if the primary metric improves (Chapters 5 and 24).

Hazard ratio

In survival models, the ratio of the instantaneous risk of the event between two groups. An association unless the grouping was randomized (Chapter 16).

Holdout

A randomly selected group deliberately not exposed to a treatment, used as the counterfactual (Chapters 8 and 24).

iROAS, iCPA

Incremental return on ad spend and incremental cost per acquisition: outcomes caused by the spend, per unit of spend, and spend per caused acquisition.

Incrementality factor

The ratio of incremental to attributed conversions measured in an experiment, used to adjust platform reporting (Chapter 8).

Intent-to-treat (ITT)

An effect estimated by comparing groups as randomized, regardless of whether each unit actually received the treatment (Chapter 8).

Lifetime value (LTV)

The expected margin a customer generates over a defined horizon, often discounted (Chapter 15).

Marketing mix model (MMM)

A regression model of a business outcome on media activity and other drivers over time, used for allocation and budget decisions (Chapters 9 to 12).

Mediator

A variable on the causal path from treatment to outcome. Adjusting for it removes part of the effect being measured (Chapter 3).

Minimum detectable effect (MDE)

The smallest true effect a design detects with a stated probability (usually 80%) at a stated significance level (Chapter 5).

Multiple comparisons

The inflation of false positives when many metrics, segments or tests are examined (Chapter 5).

Parallel trends

The assumption that treated and comparison groups would have followed parallel paths without treatment (Chapter 7).

Peeking

Checking an experiment's result repeatedly and stopping when it looks significant, which inflates the false positive rate (Chapter 5).

Placebo test

Running an analysis where no effect should exist (a fake treatment date or fake treated units) to check the method's false positive rate (Chapters 6 and 7).

Posterior predictive check

Comparing data simulated from a fitted Bayesian model with the observed data. Necessary but not sufficient evidence of a good model (Chapter 10).

Prior predictive check

Simulating from the priors alone to check that they imply plausible outcomes before fitting (Chapters 4 and 10).

Response curve

Expected outcome as a function of spend in a channel, whose slope is the marginal return (Chapter 12).

Sample ratio mismatch (SRM)

A statistically significant difference between the planned and observed split of units in an experiment, indicating a broken assignment or logging (Chapter 5).

Saturation

Diminishing returns to spend within a period (Chapter 9).

Share of search

A brand's share of branded search volume within its category, used as a leading indicator of market share (Chapter 18).

Synthetic control

A counterfactual built as a weighted combination of untreated units that tracks the treated unit before treatment (Chapters 6 and 7).

Triangulation

Combining evidence from several methods, with explicit allowances for each one's biases, to reach a decision (Chapter 14).

Uplift

The individual-level difference a treatment makes to an outcome. Targeting by uplift differs from targeting by predicted risk or engagement (Chapter 16).

C All checklists in one place

Every chapter ends with a checklist. They are collected here for use as a pre-flight list before an analysis, a test or a readout. Each heading links back to the chapter that explains the items.

What marketing science is for

Chapter 1.

- *What decision will this analysis inform, and who will make it?*
- *What would we do if the answer came out high? If it came out low? If both are the same, stop.*
- *Is the question about attributed, incremental or marginal performance? Is the method able to answer that question?*
- *Which evidence layer does the method belong to, and does a higher layer already disagree with it?*
- *What precision is needed for the decision, and can the planned design deliver it?*
- *Is there a tracking change, naming change or promotion in the window that would break a comparison?*

Data and signal foundations

Chapter 2.

- *Is the outcome taken from the system of record, not from tracking?*
- *Are tracked and backend outcomes stored side by side, with their ratio monitored?*
- *Is there a change log, and does any entry overlap the analysis window?*
- *Is unmapped spend below an agreed threshold for every week in the window?*
- *Have currency, time zone and duplicate checks been run?*
- *Does every channel you want to measure have enough spend variation to learn from?*
- *Are all channels included, including offline ones, so that omitted spend does not bias the rest?*

Causal thinking for marketers

Chapter 3.

- *Have you written down the counterfactual: what exactly would have happened without the treatment?*
- *Have you drawn the causal graph, including how the budget was set?*
- *Which variables are confounders (adjust) and which are mediators or colliders (do not adjust)?*

- *Does the analysis filter on anything that happens after treatment?*
- *Where on the identification ladder does the method sit, and is the decision large enough to justify climbing higher?*
- *Does the readout state the design, the uncertainty and the confounders that remain?*

Bayesian reasoning in practice

Chapter 4.

- *Are priors set and documented before fitting, with a source for each informative prior?*
- *Has a prior predictive check confirmed that the priors imply plausible outcomes?*
- *Has a sensitivity analysis shown whether a different reasonable prior changes the decision?*
- *When combining a model and an experiment, do they measure the same quantity (spend level, period, market, outcome)?*
- *Was the decision rule (thresholds on probability and expected loss) fixed before looking at results?*
- *Does the readout report probability of loss and expected loss, not just a point estimate?*

A/B tests that hold up

Chapter 5.

- *Is there a written plan with hypothesis, unit, primary metric and window, MDE, sample size, duration and decision rule, dated before launch?*
- *Does the MDE correspond to an effect size worth acting on? If not, redesign or do not run the test.*
- *Does the test run in whole weeks and long enough for the outcome window?*
- *Was the sample ratio checked, and were A/A tests run on this setup?*
- *Is the decision metric read only at the planned end, or under a pre-planned sequential design?*
- *How many comparisons does the readout make, and are they corrected or labeled as exploratory?*
- *Is a null result reported with the effect size that could have been detected?*

Geo experiments

Chapter 6.

- *Is the geo unit the smallest one at which media and outcomes can both be separated, with spillover considered?*
- *Was assignment randomized (ideally stratified), and is the method fixed in advance?*
- *Does a pre-test power analysis on historical data give an MDE worth testing for?*

- *Is the treatment type (go-dark, heavy-up, scale-back) matched to the decision (average versus marginal return)?*
- *Is delivery verified by region, so that the treatment actually happened as designed?*
- *Is the outcome taken from the system of record, with a cooldown window for lasting effects?*
- *Is the result reported as an iROAS range from placebo-based or model-based intervals, with the MDE?*

Quasi-experiments: DiD, CausalImpact and synthetic control

Chapter 7.

- *Is there a plausible comparison that was not affected by the intervention?*
- *For DiD: are event-study pre-period coefficients flat, and are standard errors clustered at the unit level?*
- *For staggered rollouts: is a staggered-adoption estimator used instead of plain two-way fixed effects?*
- *For CausalImpact: are the controls unaffected, chosen on pre-period grounds only, and fixed before looking?*
- *Do placebo-in-time (and, where possible, placebo-in-space) tests come out null?*
- *Does the change log show anything else hitting the treated unit in the window?*
- *Does the readout name the design as quasi-experimental and list the threats that remain?*

Reading platform lift studies

Chapter 8.

- *Is the outcome the platform's tracked conversion or your system-of-record conversion?*
- *Is the result intent-to-treat or per reached user, and does any comparison mix the two?*
- *How many conversions were in the holdout, and what MDE did the design have?*
- *Which confidence or credibility level is reported?*
- *Did budget, bidding or creative change during the study?*
- *Is the incrementality factor computed with its interval and stored with the conditions it was measured under (platform, campaign type, market, attribution setting, date)?*
- *If the result is applied to another market, campaign or spend level, is that stated as an assumption?*

Marketing mix modeling end to end

Chapter 9.

- *Is the outcome from the system of record, and are all major non-media drivers in the base?*
- *Does every modeled channel have meaningful spend variation (check the coefficient of variation), and are channels without it grouped or handled with priors?*
- *Is there a channel whose spend responds to demand (search, retargeting)? Treat its estimate as an upper bound until tested.*
- *Are adstock and saturation choices justified per channel type?*
- *Is the ratio of channels to observations reasonable, or is geo-level data available?*
- *Is anyone treating R^2 or MAPE as validation?*

Bayesian MMM in practice

Chapter 10.

- *Is every prior written down with a reason, and does the prior predictive check imply plausible ROIs and media shares?*
- *Were divergences, R-hat and effective sample sizes checked before reading results?*
- *Does the posterior predictive check cover the data at roughly the nominal rate?*
- *For each channel, how does the posterior compare with the prior? Which channels are data-driven and which are prior-driven?*
- *Which channels' spend responds to demand? Are their estimates flagged as likely upward-biased until tested?*

Validating and calibrating MMM

Chapter 11.

- *Has the model been refit on several windows, and are unstable channels flagged as unidentified?*
- *Is every channel's estimate compared with an experiment where one exists?*
- *For channels whose spend responds to demand, is there a demand proxy that is a confounder (category level) rather than a mediator (brand level)?*
- *Are experiments used for calibration matched to the model's outcome, market and spend level, with uncertainty inflated where they are not?*
- *Has parameter recovery been checked on data simulated to resemble the real situation?*
- *Is there an experiment plan for channels that remain unidentified?*

Response curves and budget optimization

Chapter 12.

- *Is the decision based on marginal ROI at current spend, not average ROI?*
- *Is the break-even threshold set on profit (margin-adjusted), not revenue?*
- *Are the curves from a calibrated model, and are channels with confounded or prior-dominated estimates flagged?*
- *Are optimizer bounds kept within the historically observed spend range?*
- *Is the expected gain reported with its uncertainty and the probability of a loss?*
- *Is the recommendation implemented as a measurable step (heavy-up or scale-back test) rather than all at once?*

Attribution and its limits

Chapter 13.

- *Is the attribution output being used to compare channels or set budgets? If so, stop and look for incrementality evidence.*
- *Are attributed conversions labeled as attributed in every report, never as incremental?*
- *For each channel, is its targeting correlated with existing intent (retargeting, CRM, branded search)? Expect over-credit.*
- *Where an incrementality factor exists, is it applied, with its date and conditions?*
- *How much of the journey is invisible because of consent, devices or platforms, and which channels does that hurt?*

Triangulation: deciding under contradictory evidence

Chapter 14.

- *Does each source have a stated bias allowance as well as its reported uncertainty?*
- *Is attribution kept out of incrementality combinations, or given a large allowance?*
- *Is the decision made on marginal returns, with probability and size of loss reported?*
- *Is the reversibility of the decision considered alongside expected value?*
- *Is the testing roadmap prioritized by money at stake and the chance of flipping a decision?*
- *Is there a channel scorecard with evidence type, age and confidence, kept up to date?*

Cohorts, retention, LTV and payback

Chapter 15.

- *Are retention and value analyzed by cohort (acquisition month and channel), not as blended averages?*

- *Does the retention model allow churn to fall as cohorts age, rather than assuming a constant rate?*
- *Is lifetime value built from margin, over a stated horizon, with discounting where the horizon is long?*
- *Is the uncertainty of projected LTV reported, and how much of it is projection beyond observed months?*
- *Are channel CAC targets set on value, and on incremental rather than attributed acquisition cost?*
- *For experiments that change customer mix, is value read at a long enough horizon?*

Churn and survival analysis

Chapter 16.

- *Are churn and lifetime estimated with methods that handle censoring (Kaplan-Meier, Cox, parametric survival), not averages over churned customers?*
- *Is the churn definition explicit and stable, with voluntary and involuntary churn separated?*
- *Are hazard ratios presented as associations unless the feature was randomized?*
- *Are retention offers targeted by uplift from a randomized test, rather than by predicted risk?*
- *Is the effect of retention interventions read at a horizon long enough to distinguish prevention from delay?*

Segmentation

Chapter 17.

- *Does each segment lead to a different, feasible action, and can a customer's segment be identified when the action happens?*
- *Has a simple managerial segmentation (such as RFM) been tried as a baseline?*
- *Were features transformed and scaled deliberately, with the choice of what counts as "different" written down?*
- *Is the number of segments justified by actionability, with stability checked rather than silhouette alone?*
- *Are categorical features handled with an appropriate distance (Gower) rather than raw one-hot encoding?*
- *Are segments profiled on outcomes that were not clustering inputs, and are segment-specific treatments tested?*

Brand measurement

Chapter 18.

- *Is brand health tracked with a fixed set of metrics and a fixed competitor set, by market, over time?*
- *Is share of search used rather than raw branded search volume, and are changes rather than levels correlated?*
- *Is there a change log for events that move branded search (paid brand bidding, PR, incidents)?*
- *Are penetration and new-customer metrics tracked alongside loyalty metrics?*
- *Do performance experiments also record halo outcomes (branded search, direct, organic)?*
- *Are brand effects judged on windows long enough for them to appear?*

Pricing research and elasticities

Chapter 19.

- *Is the method matched to the decision: surveys for ranges and feature values, tests for final price changes?*
- *Are stated-intent results treated as relative signals, calibrated where possible against real conversion?*
- *Has the break-even volume change been calculated for the proposed price change at the actual margin?*
- *Are historical elasticities treated as confounded context rather than estimates?*
- *Does any price test respect fairness and regulatory constraints, and is it read on margin per customer over a long enough horizon, including mix effects?*

Forecasting and anomaly detection

Chapter 20.

- *Is every forecast compared with a seasonal naive baseline in a rolling-origin backtest at the relevant horizon?*
- *Do prediction intervals achieve close to their nominal coverage in the backtest?*
- *Are holidays, promotions and known events in the model, so that expected dips are not treated as surprises?*
- *Do monitoring alerts compare each day with a model expectation, scaled by the model's normal error, rather than with yesterday?*
- *Are persistent level shifts checked with change-point or cumulative methods, not only single-day thresholds?*
- *Is each alert routed to an owner, with the change log checked before escalating?*

Paid search

Chapter 21.

- *Is brand search incrementality measured with a randomized design (regional switchback or geo holdout) on total brand traffic?*
- *Are brand and non-brand reported and targeted separately?*
- *Are bids and budgets set on marginal CPA, not average CPA?*
- *Do automated campaigns receive system-of-record conversions and values, and exclude existing customers?*
- *Is the search channel's MMM estimate treated as likely upward-biased until calibrated (Chapter 11)?*

Paid social and creative fatigue

Chapter 22.

- *Is paid social incrementality measured with lift studies on system-of-record outcomes, and checked with geo tests for large budgets?*
- *Are creative tests run with randomized audiences, on acquisition metrics, with a fixed sample size?*
- *Is fatigue measured against frequency rather than calendar days, with a refresh threshold derived from your own data?*
- *Do click-based fatigue signals agree with conversion-based ones?*
- *Are frequency caps tested rather than set by convention?*

SEO measurement

Chapter 23.

- *Can the change be tested by randomizing comparable pages into changed and control groups?*
- *Is randomization stratified by pre-period traffic, and has a placebo analysis shown the design can detect a realistic effect?*
- *Does the readout window allow for crawling and re-ranking?*
- *Are site-wide changes analyzed with a site-level quasi-experiment instead of a page split?*
- *Are search engine updates and site releases in the change log?*

CRM and email holdouts

Chapter 24.

- *Does every CRM program have a randomized holdout, sized from the effect that matters?*
- *Is there a persistent global holdout measuring the cumulative effect of all messaging?*

- *Are results read on the outcome (activation, transactions, revenue), not on opens or post-click conversions?*
- *Are unsubscribes, complaints and long-run engagement tracked as guardrails?*
- *Are message A/B tests run with a no-message holdout still in place?*

Readouts that change decisions

Chapter 25.

- *Does the readout open with the verdict and the confidence in it?*
- *Is the design named in the first paragraph, with observational results labeled as such?*
- *Does every estimate have an interval, a sample size, a time window and a source?*
- *Are null results reported with the effect size that could have been detected?*
- *Are the number of comparisons and any interim looks disclosed?*
- *Do charts show intervals and decision thresholds?*
- *Does the recommendation have an owner, a date and a statement of what would change it?*

The analyst's toolkit

Chapter 26.

- *Is there a written plan, dated before the data, for every confirmatory analysis?*
- *Was the design checked by simulation, including its power and false positive rate?*
- *Is the analysis reproducible: code in version control, seeds fixed, package versions and data snapshot recorded?*
- *Are AI-generated queries and analyses reviewed against the metric dictionary and checked for silent choices?*
- *Is there a shared record of past experiments and results that new work can draw on?*